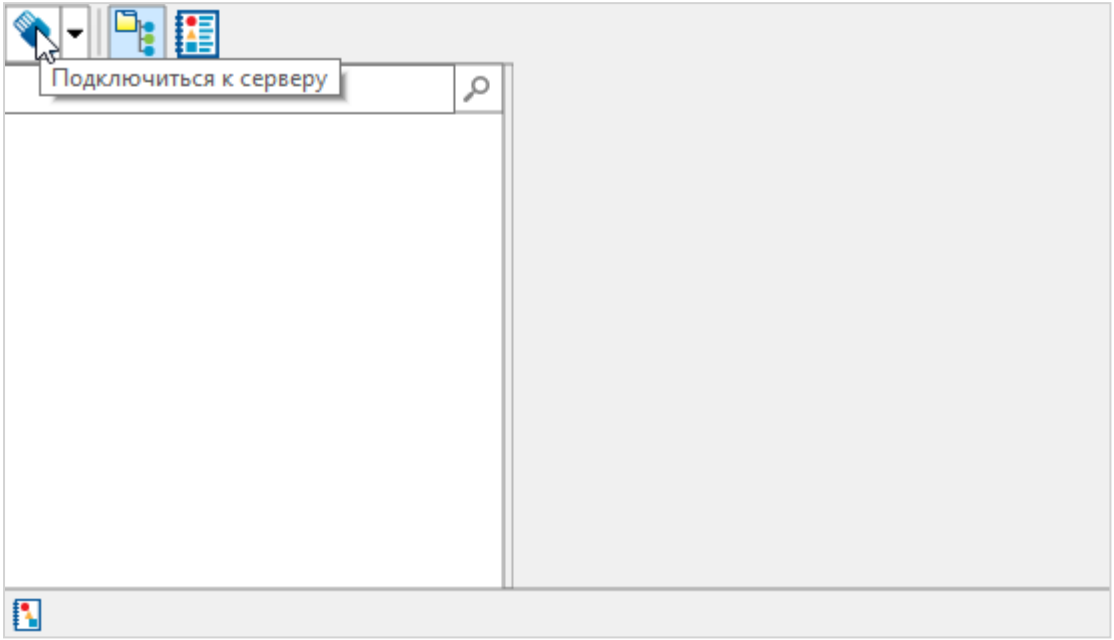
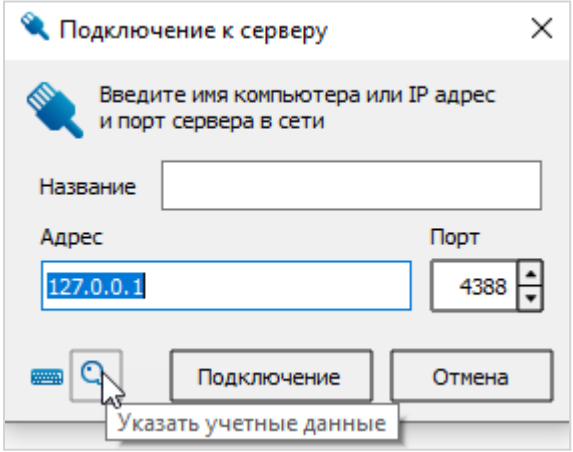
1.5. Подключение к серверу

Для подключения к нужному серверу нажмите кнопку **Подключиться к серверу** на панели инструментов.



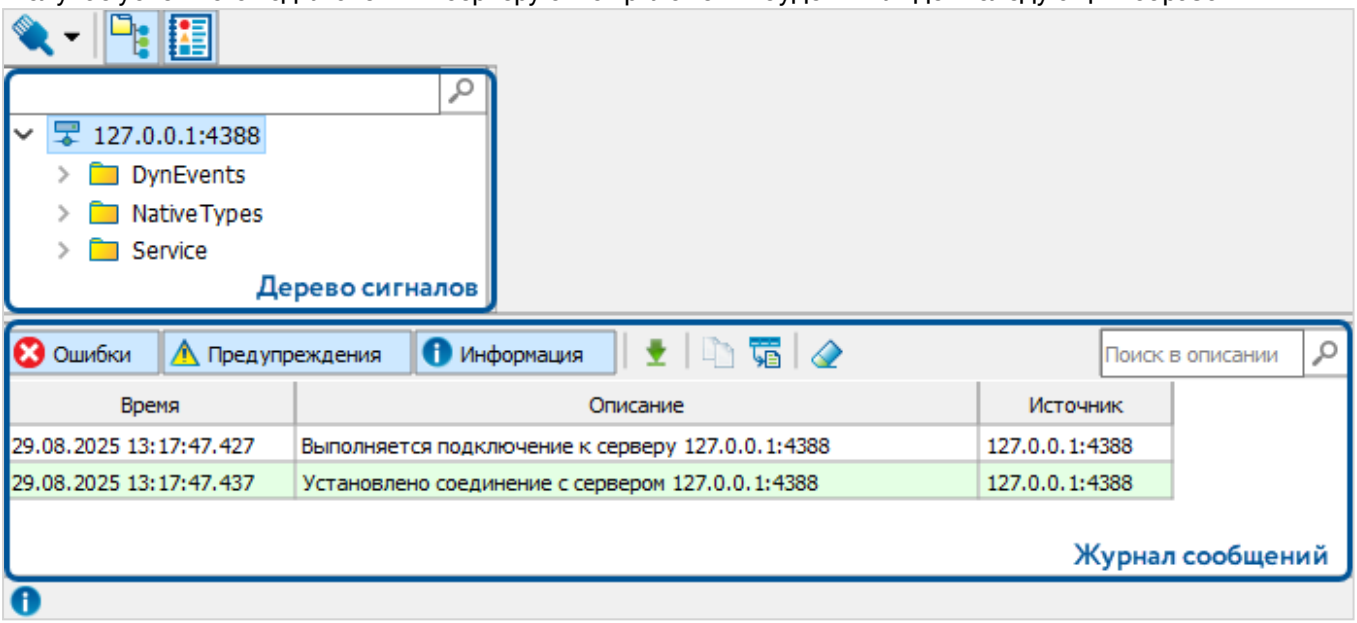
В открывшемся диалоговом окне укажите адрес и порт сервера. Если сервер защищен паролем, следует нажать кнопку **Указать учетные данные**, и в открывшемся окне ввести пароль. Название подключения вводить не обязательно - оно используется для удобства выбора подключения из списка подключений в дальнейшем.



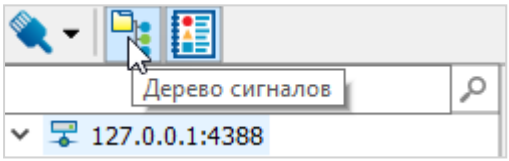
Если необходимо, можно подключиться к нескольким серверам за одну сессию[1]. Для этого подключитесь к каждому серверу по очереди.

[1]Сессией считается процесс взаимодействия пользователя с приложением. Сессия начинается с момента открытия приложения, и завершается в момент закрытия приложения.

В случае успешного подключения к серверу окно приложения будет выглядеть следующим образом.

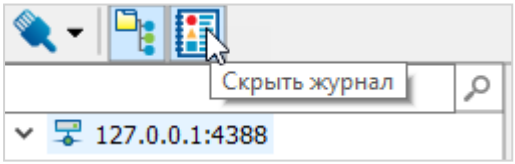


› В дереве сигналов отображаются сигналы серверов, к которым произведено подключение. Дерево сигналов можно скрыть, нажав кнопку **Дерево сигналов** на панели инструментов.



› В правой части окна можно открыть окно просмотра сигнала и инспектор сигналов - это позволит просмотреть и изменить значение сигнала. Подробнее об этом - в разделе 1.6. Просмотр и изменение значений сигналов (стр. 12).

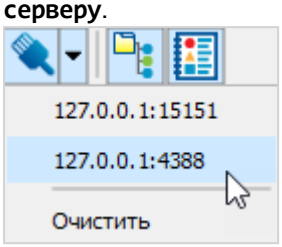
› В журнал записываются сообщения об ошибках, предупреждениях и информационные сообщения. Журнал можно скрыть, нажав кнопку **Скрыть журнал** на панели инструментов.



ОБРАТИТЕ ВНИМАНИЕ

Папка **Service** содержит служебные сигналы текущей конфигурации Alpha.Server. Подробнее служебные сигналы описаны в документе на Alpha.Server, раздел **«Конфигурирование Alpha.Server»**.

Чтобы каждый раз не вводить адрес и порт нужного сервера, достаточно подключиться к нему один раз, а затем подключаться, выбрав нужный сервер из выпадающего списка рядом с кнопкой **Подключиться к серверу**.



В списке отображаются до десяти последних подключений. Чтобы очистить список, нажмите **Очистить**.

1.6. Просмотр и изменение значений сигналов

Сигналы сервера, к которому произведено подключение, отображены в дереве сигналов. Просмотреть информацию о сигнале и изменить его значение можно в редакторе сигнала или инспекторе сигналов.

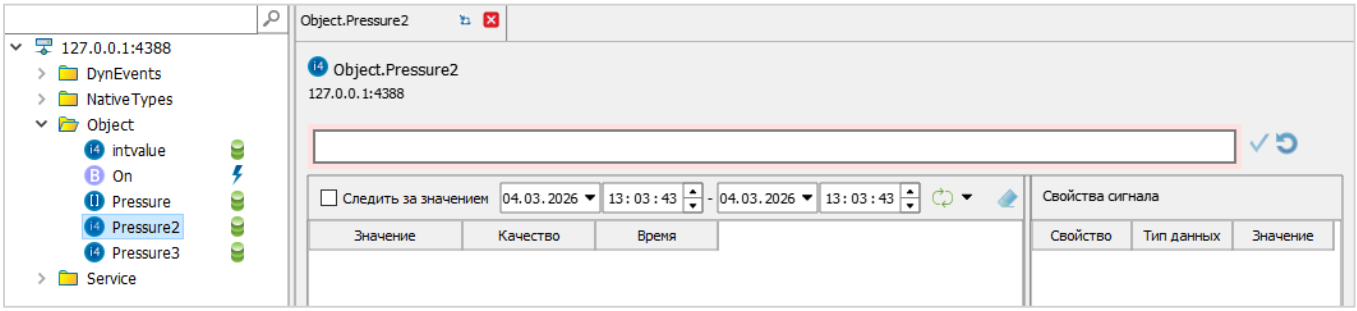
> **!** ВАЖНО
> Будьте внимательны при изменении значений сигналов, так как это может привести к непредсказуемым последствиям для технологического процесса.

Просмотр и изменение значений в редакторе сигнала

Чтобы открыть редактор сигнала, кликните левой кнопкой мыши на сигнале в дереве сигналов. Откроется окно, в котором можно посмотреть:

- полный путь до сигнала;
- источник подключения;
- значение сигнала;
- свойства сигнала.

В этом окне также можно изменить значение сигнала, следить за его изменением или просмотреть историю значений сигнала.

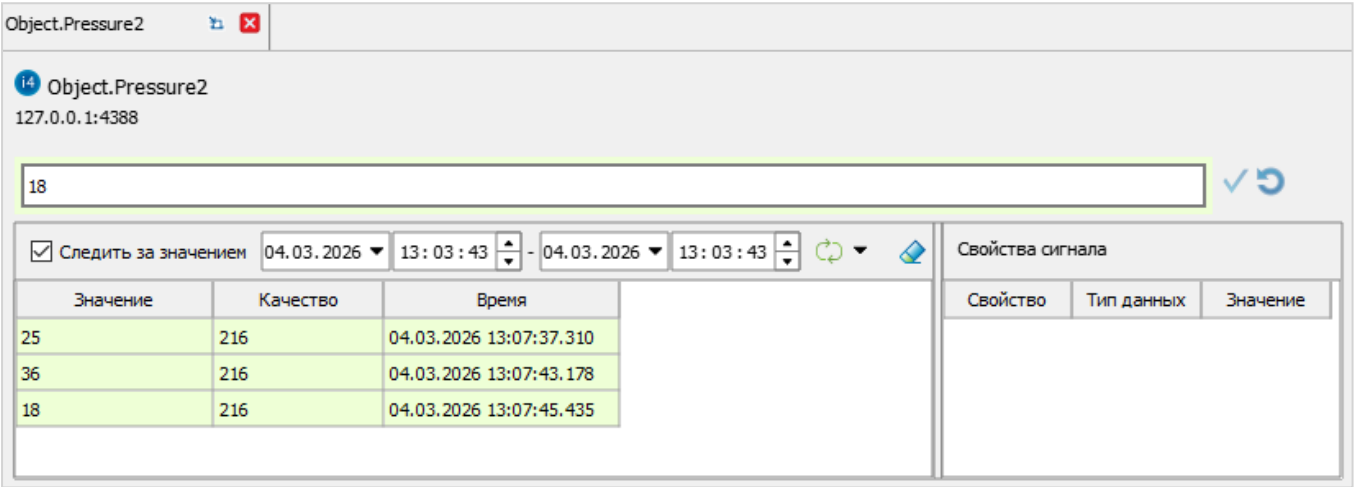


Чтобы изменить значение сигнала, введите новое значение в поле ввода и нажмите **Записать значение** ✓.

- Если значение введено с ошибкой или находится вне диапазона допустимых значений, во всплывающей подсказке появится соответствующее предупреждение.
- Чтобы отменить ввод, нажмите клавишу Esc.

Под полем ввода значения располагаются две таблицы.

- В таблице слева можно следить за изменением значения сигнала, установив флаг **Следить за значением**, либо запросить историю значений сигнала за определенный период. Для этого следует указать временной период в полях ввода и выполнить запрос исторических значений сигнала, нажав кнопку **Загрузить историю значений** (флаг **Следить за значением** при этом будет снят автоматически). При вводе некорректных границ периода поля ввода выделятся красной рамкой. Чтобы очистить таблицу, нажмите .
- В таблице справа можно просмотреть пользовательские свойства сигнала. Пользовательские свойства - это свойства с номерами в диапазоне от **«1000000»** до **«2000000»**, тип и значения которых вы можете определить. Подробнее о добавлении сигнала и его свойств можно почитать в документации на Alpha.Server → Конфигурирование Alpha.Server.

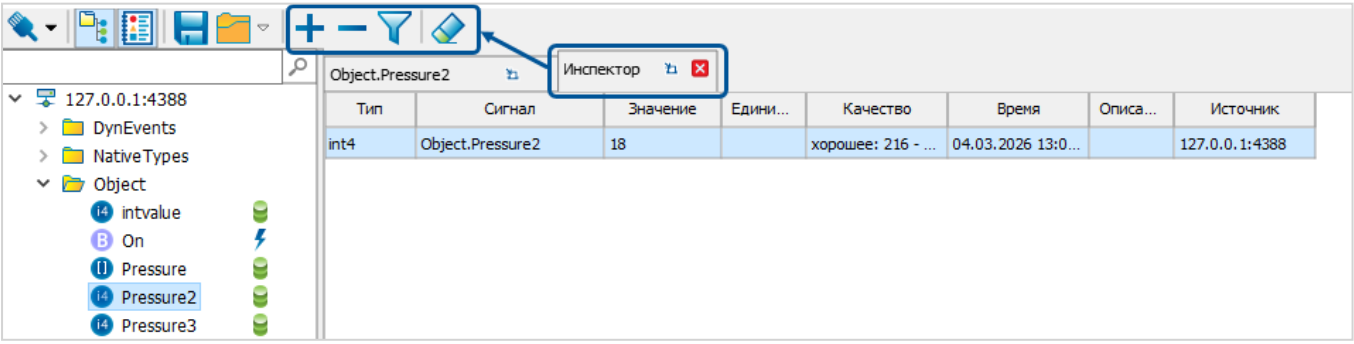


Изменение значений сигналов в инспекторе

Инспектор сигналов предоставляет информацию о сигналах в табличном виде. Открыть **Инспектор** можно двумя способами:

- кликните дважды левой кнопкой мыши на сигнал в дереве сигналов;
- кликните правой кнопкой мыши на сигнал в дереве сигналов и выберите **Добавить в инспектор сигналов** в открывшемся контекстном меню.

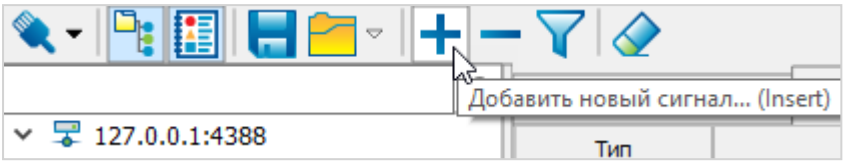
После открытия инспектора на панели инструментов появятся кнопки управления инспектором.



Чтобы добавить сигнал в **Инспектор**, можно:

- перетянуть сигнал из дерева в инспектор сигналов;
- дважды кликнуть по сигналу в дереве;

- кликнуть на сигнал правой кнопкой мыши в дереве сигналов и выбрать **Добавить в инспектор сигналов** в открывшемся контекстном меню;
- нажать **Добавить новый сигнал...** на панели инструментов;



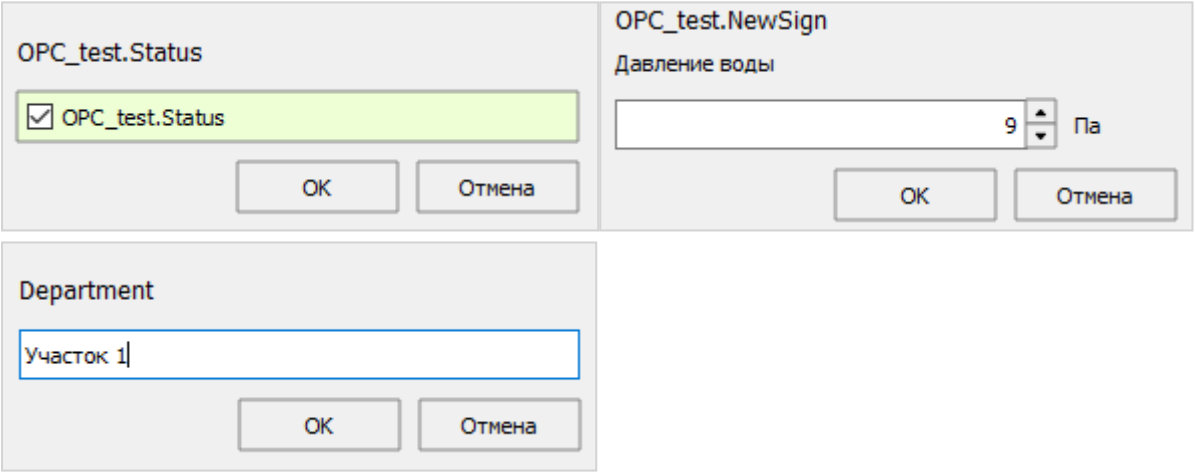
В открывшемся окне введите полный путь сигнала и нажмите **Добавить**.

- нажать клавишу Insert.

Чтобы изменить значение сигнала в инспекторе сигналов, можно:

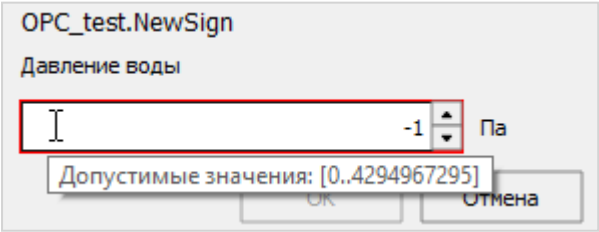
- выбрать строку сигнала в таблице инспектора и нажать клавишу Enter;
- кликнуть по строке сигнала в инспекторе дважды;
- кликнуть правой кнопкой мыши по строке сигнала в инспекторе и выбрать пункт **Изменить значение сигнала...** в контекстном меню.

Откроется окно редактирования значения сигнала. Внешний вид окна зависит от типа данных сигнала.



Введите нужное значение и нажмите **OK**.

- Если значение введено с ошибкой или находится вне диапазона допустимых значений, во всплывающей подсказке появится соответствующее предупреждение.



- Чтобы отменить ввод, нажмите клавишу Esc.

Чтобы скопировать какое-либо значение из инспектора сигналов, можно:

- щелкнуть правой кнопкой мыши по этому значению, и в открывшемся контекстном меню выбрать команду **Копировать**;
- кликнуть на это значение и нажать сочетание клавиш Ctrl + C.

Чтобы выделить все сигналы в инспекторе, можно:

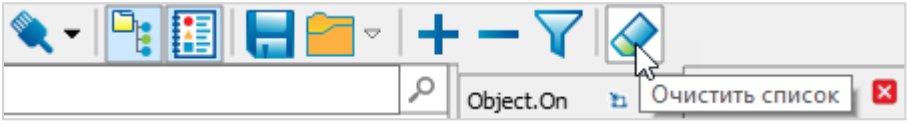
- в контекстном меню таблицы сигналов выбрать **Выделить все сигналы**;
- нажать сочетание клавиш Ctrl + A.

Чтобы оставить в инспекторе сигналов только выделенные сигналы, в контекстном меню таблицы сигналов выберите **Оставить только выделенные сигналы**.

Чтобы удалить сигнал из таблицы инспектора, можно:

- выбрать строку сигнала в таблице инспектора и нажать клавишу Delete;
- кликнуть правой кнопкой мыши по строке сигнала в инспекторе и выбрать пункт **Удалить выделенные сигналы** в контекстном меню;
- выбрать строку сигнала в таблице инспектора и нажать **Удалить выделенные сигналы** на панели инструментов.

Чтобы полностью очистить инспектор сигналов, нажмите **Очистить список** на панели инструментов.



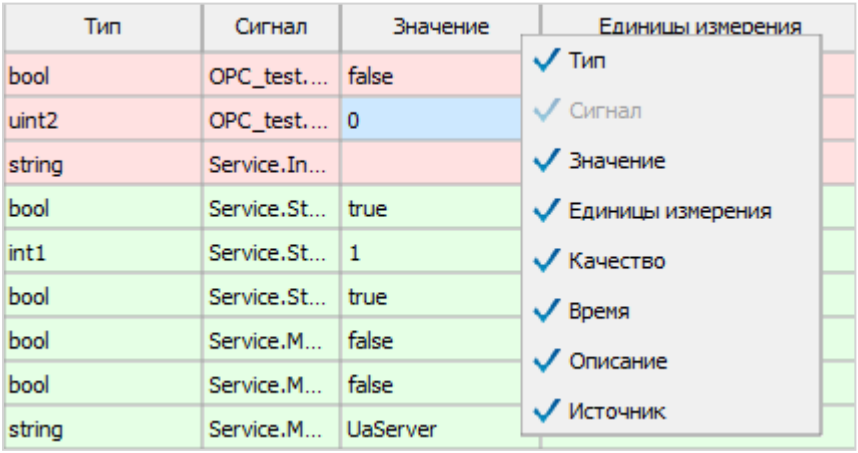
Если в конфигурацию сервера были внесены изменения, которые не отображаются в дереве сигналов, кликните правой кнопкой мыши по узлу дерева, в котором произошли изменения, и выберите команду **Обновить** в контекстном меню.

ПРИМЕЧАНИЕ

Чтобы скопировать полный путь до сигнала или папки в буфер обмена, щелкните по этому узлу правой кнопкой мыши в дереве сигналов, и в открывшемся контекстном меню выберите команду **Копировать полное имя**.

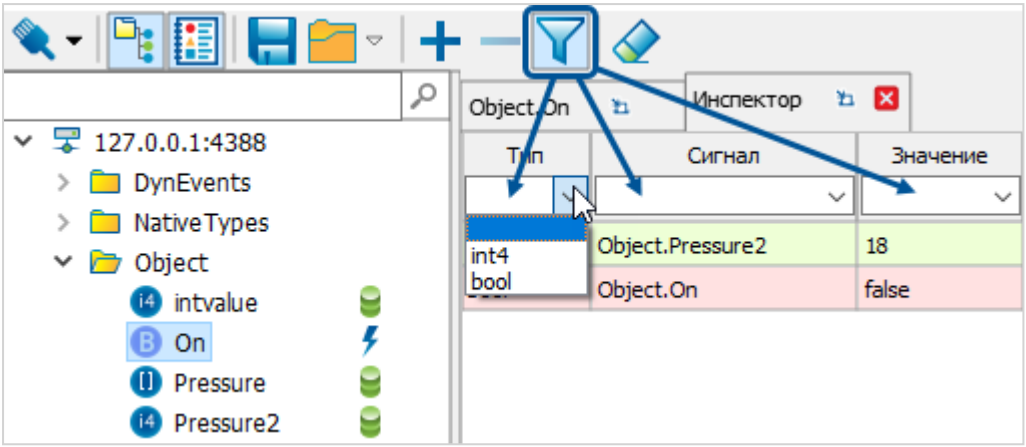
1.6.1. Изменение вида таблицы сигналов в инспекторе

Столбцы таблицы в инспекторе сигналов можно показать или скрыть. Для этого кликните правой кнопкой мыши на заголовке таблицы и в контекстном меню отметьте нужные столбцы.



Данные в столбцах можно фильтровать и сортировать.

- Чтобы использовать фильтр, нажмите **Фильтровать список**.

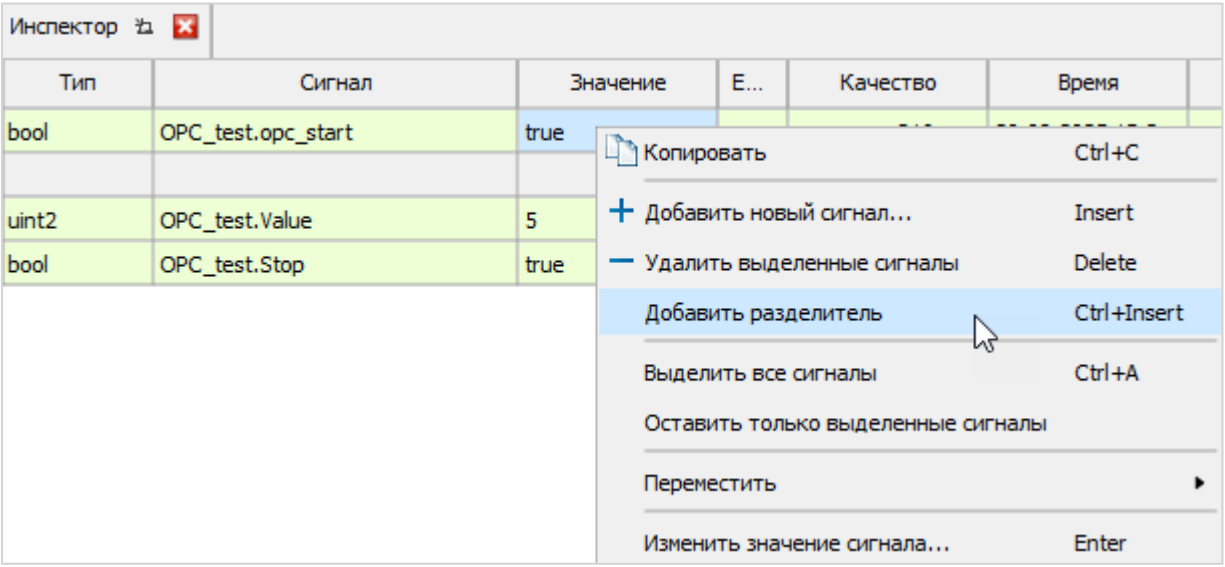


- Чтобы сортировать данные в столбцах, нажмите на заголовок столбца, по которому нужно сортировать данные. При повторном нажатии на заголовок меняются режимы сортировки (по возрастанию - по убыванию - отключение сортировки).

Тип	Сигнал	Значение	Единицы измерения
int2	Level.s_lev...	4	
int1	Level.s_level	10	
uint1	Level.s_lev...	21	

Между строками таблицы можно вставить разделитель - пустую строку, позволяющую визуально отделить сигналы друг от друга. Перед тем, как добавить разделитель, выберите строку, после которой нужно его вставить. Чтобы добавить разделитель, можно:

- кликнуть правой кнопкой мыши на выделенной строке и выбрать **Добавить разделитель** в контекстном меню;
- нажать сочетание клавиш Ctrl + Insert.

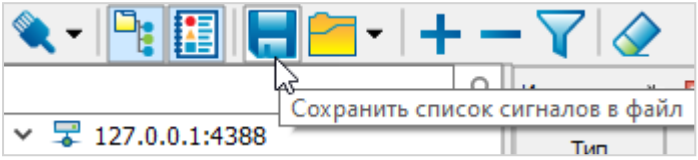


В таблице можно переместить строку вверх или вниз. Для этого кликните на необходимой строке. Чтобы переместить ее, можно:

- в контекстном меню строки выбрать **Переместить**. В открывшемся выпадающем меню выберите необходимое направление перемещения строки;
- нажать сочетание клавиш Ctrl + Вверх или Ctrl + Вниз.

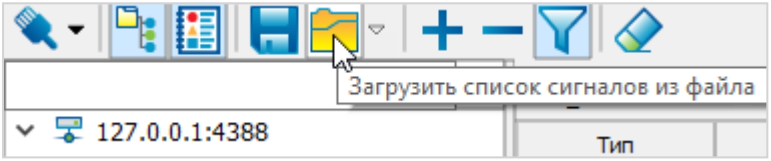
1.6.2. Сохранение списка сигналов в файл и восстановление из файла

Список сигналов и информацию о них можно сохранить в json-формате в файл в с расширением `*.esl`. Помимо информации о сигналах, в сохраняемый файл запишутся адреса и порты серверов, к которым произведено подключение. Чтобы сохранить файл, нажмите **Сохранить список сигналов в файл** на панели инструментов.



Откроется окно, где можно будет ввести имя и выбрать расположение файла.

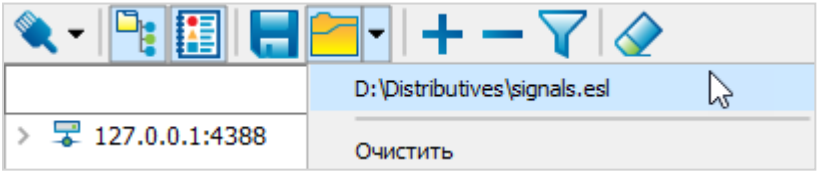
Чтобы восстановить список сигналов из файла, нажмите **Загрузить список сигналов из файла**.



Откроется окно, где можно будет выбрать нужный файл.

> ОБРАТИТЕ ВНИМАНИЕ
> Если текущее подключение отличается от указанного в открываемом файле, произойдет автоматическое подключение к указанным в файле серверам и портам.

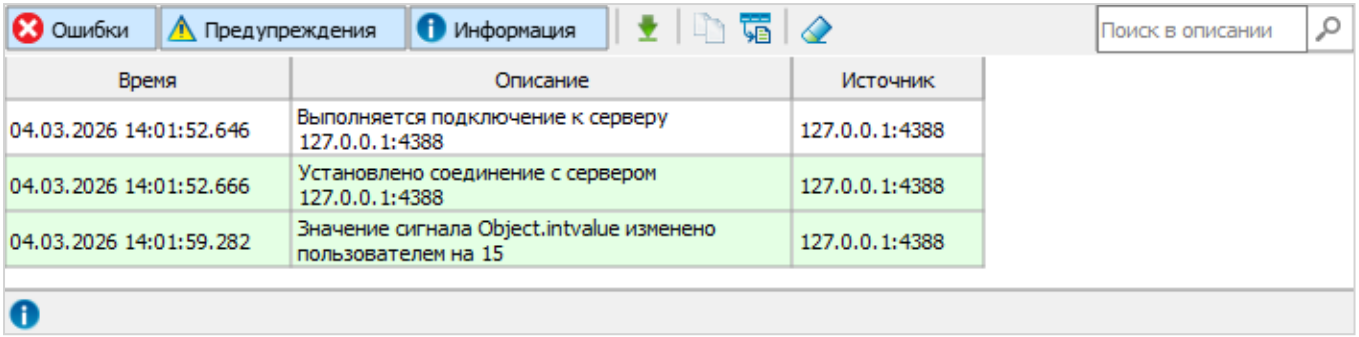
Недавно открытый файл можно открыть повторно, выбрав его из списка недавно открытых файлов. Список открывается по нажатию на стрелку, расположенную рядом с кнопкой **Загрузить список сигналов из файла**.



В списке отображаются до десяти последних файлов. Чтобы очистить список, нажмите **Очистить**.

1.7. Управление журналом сообщений

В журнал записываются сообщения об ошибках, предупреждениях и информационные сообщения.



Журнал можно показать или скрыть, нажав кнопку **Показать журнал** в строке состояния. Кнопка меняет внешний вид в зависимости от наличия сообщений определенного типа в журнале.

- Чтобы в журнале не отображались сообщения определенного типа, отожмите кнопку с названием типа на панели инструментов журнала.
- Чтобы следить за новыми сообщениями, нажмите кнопку **Автоматическое прокручивание списка**.
- Чтобы очистить журнал сообщений, нажмите кнопку **Очистить список**.
- Чтобы найти в журнале сообщения с определенной фразой, введите ее в поле **Поиск в описании** и нажмите кнопку **Поиск**.
- Чтобы изменить порядок сортировки сообщений, нажмите на заголовок столбца таблицы, по которому необходимо сортировать сообщения. При повторном нажатии на заголовок меняются режимы сортировки (по возрастанию - по убыванию - отключение сортировки).

2. Справочное руководство

2.1. Explorer

`Explorer` - тип, предназначенный для размещения на форме Alpha.HMI.

Тип **`Explorer`** не является полноценной заменой приложения Alpha.HMI.Explorer: экземпляр типа отличается тем, что в нем нет верхней панели инструментов и журнала сообщений.

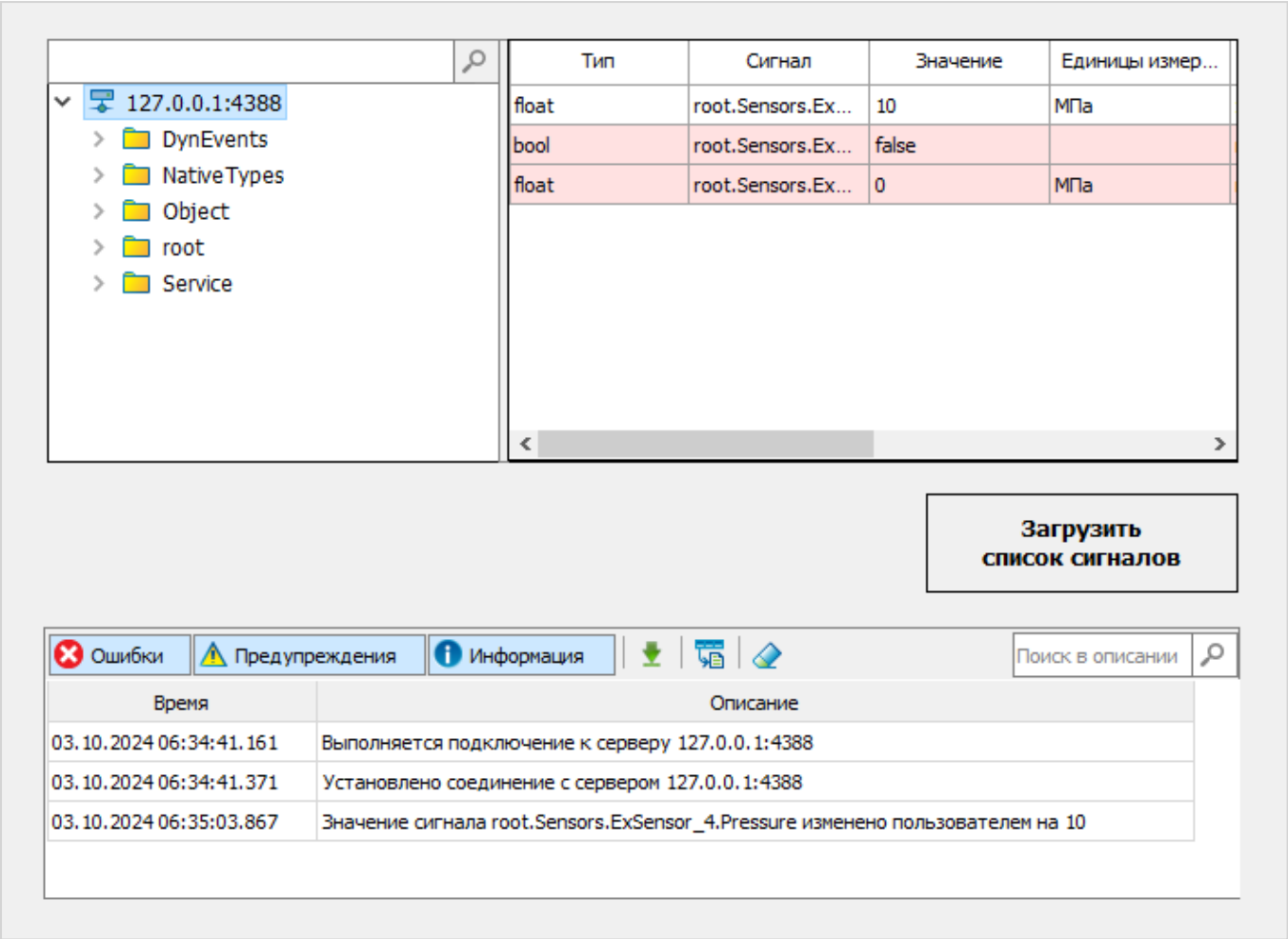
Тип, используемый в проекте Alpha.HMI, в режиме рантайма создает файлы, описанные в таблице ниже.

Файл	Расположение	Назначение
`settings.json`	› `C:\ProgramData\Automiq\HMI.Explorer` для ОС Windows; › `/home/<user>/Automiq/HMI.Explorer` для ОС Linux.	Хранит информацию о папке экспорта списка сигналов в файл. Используется для указания папки, открываемой при сохранении списка сигналов по умолчанию.
`session.json`	› `C:\Users\<user>\Automiq\HMI.Explorer` для ОС Windows; › `/home/<user>/Automiq/HMI.Explorer` для ОС Linux.	Хранит информацию о внешнем виде дерева сигналов и таблицы инспектора сигналов. Файл используется для работы типа, менять его содержимое не рекомендуется.

Пример использования типа в проекте

Ниже описано, как загружать списки сигналов из файлов и получать сообщения об изменении значений сигналов в журнал.

Интерфейс создаваемого в примере приложения может выглядеть так:



Для начала работы разместите на форме экземпляр типа **Explorer**.

Загрузить список сигналов из файла

Чтобы загрузить список сигналов из файла в тип **Explorer** по нажатию кнопки, создайте на форме экземпляр типа **Кнопка**. В обработчике события ButtonPressed кнопки вызовите функцию OpenSignalsList(), в качестве входного параметра укажите путь к файлу со списком сигналов.

ПРИМЕР

```
Explorer_1.OpenSignalsList("Z:\\folder\\signals.esl");
```

Получить сообщение об изменении значения сигнала

Чтобы получать сообщения при изменении значений сигналов, можно использовать разработанные для этого инструменты типа **Explorer** и библиотеки Alpha.HMI.CommonLib. Тип **Explorer** может сослаться на экземпляр типа **Обработчик события**, предоставляемый библиотекой Alpha.HMI.CommonLib, с помощью свойства Обработка события. Тогда этот экземпляр **Обработчик события** предоставит аргументы возникшего в **Explorer** события. Перечень этих аргументов приведен в описании свойства Обработка события. Один из таких аргументов - готовое сообщение об изменении значения сигнала.

Чтобы передать сообщение об изменении сигнала в журнал:

1. Добавьте на форму экземпляры **Обработчик события** и **Журнал сообщений**.
2. Укажите ссылку на созданный **Обработчик событий** в свойстве Обработка события типа **Explorer**.
3. В обработчике события Invoked экземпляра **Обработчик событий**:

   3.1. Получите аргументы возникшего события из объекта **Args** с помощью функции Get(), предварительно разобрав его:

   ```
   var arg = JSON.parse(Args.Get());
   ```

   Обратите внимание, что в окне ввода кода должен быть выбран язык JavaScript.

   3.2. Передайте в **Журнал сообщений** сообщение, полученное в результате разбора объекта, с помощью функции AddMessage():

   ```
   LogView.AddMessage(0,arg.message);
   ```

2.1.1. Свойства

Отображать дерево (TreeVisible)

Меняет видимость дерева сигналов:

- **«true»** - дерево отображается;
- **«false»** - дерево скрыто.

Только одно подключение (init_SingleConnection)

Определяет, к какому количеству серверов или файлов одновременно можно подключиться в текущем экземпляре типа:

- **«true»** - можно подключиться только к одному серверу или файлу;
- **«false»** - можно подключиться к нескольким серверам или файлам.

Тема оформления (init_Theme)

Ссылка на экземпляр типа **Тема оформления**, хранящий настройки оформления приложения. Тип предоставляется библиотекой Alpha.HMI.CommonLib.

Вызов экранной клавиатуры (init_ScreenKeyboard)

Ссылка на экземпляр типа **Параметры вызова электронной клавиатуры**, хранящий настройки вызова экранной клавиатуры. Тип предоставляется библиотекой Alpha.HMI.CommonLib.

Обработка события (init_Log)

Ссылка на экземпляр типа **Обработчик события**, предоставляемого библиотекой Alpha.HMI.CommonLib. Позволяет сослаться на пользовательский экземпляр типа **Обработчик события** для обработки события, произошедшего в **Explorer**.

Тип **Обработчик события** библиотеки Alpha.HMI.CommonLib содержит объект **Args** типа **Буфер аргументов события**. В этом объекте хранятся аргументы произошедшего события. Чтобы использовать аргументы, их

необходимо извлечь с помощью функции Get() объекта **Args**. Результатом работы этой функции является JSON-строка с аргументами произошедшего события.

ПРИМЕР

Результат вызова функции Get():

```
{"severity":"1","message":"Значение сигнала root.Sensors.ExSensor_
4.Pressure изменено пользователем на 6","source":
{"source":"10.0.2.15:4388"}}
```

Аргументы события, произошедшего в **Explorer**:

- severity - важность события. Тип данных - строка.
- message - описание события. Тип данных - строка.
- source - IP-адрес и порт подключения. Тип данных - JSON-строка.

Чтобы обратиться к аргументу по имени, необходимо преобразовать полученную JSON-строку в объект с помощью метода JSON.parse языка JavaScript.

```
JSON.parse(Args.Get());
```

Отображать атрибуты сигналов (ShowAttributes)

Отображает иконки атрибутов сигналов в дереве сигналов:

- **«true»** - иконки отображаются;
- **«false»** - иконки не отображаются.

Вывод описания предпочтительнее имени (DescriptionPreferred)

Отображает описание сигнала в дереве сигналов:

- **«true»** - описание отображается;
- **«false»** - описание не отображается, выводится имя сигнала.

2.2. Explorer_App

Explorer_App - форма, используемая для открытия полноценного приложения Alpha.HMI.Explorer из меню Пуск.

Не имеет собственных свойств и функций.

2.3. Explorer_Form

Форма **Explorer_Form** предназначена для подключения к единственному серверу, указанному в свойствах формы.

Форму можно вызвать в обработчике какого-либо события.

Форма, используемая в проекте Alpha.HMI, в режиме рантайма создает конфигурационный файл `window.json`, хранящий информацию о внешнем виде и настройках формы. Файл расположен в:

- `C:\Users\<user>\Automiq\HMI.Explorer` для ОС Windows;
- `/home/<user>/Automiq/HMI.Explorer` для ОС Linux.

2.3.1. Свойства

init_Theme

Ссылка на экземпляр типа **Тема оформления**, хранящий настройки оформления приложения. Тип предоставляется библиотекой Alpha.HMI.CommonLib.

Восстанавливать настройки окна (init_RestoreWindow)

Указывает, должно ли окно открываться в указанном месте, или в том же, где было открыто в последний раз.

Значения:

- **«true»** – положение окна запоминается. Если переместить окно в определенную область, закрыть его, а затем вновь открыть, то окно откроется в том же месте, где было закрыто.
- **«false»** – положение окна не запоминается. Если переместить окно в определенную область, закрыть его, а затем вновь открыть, то окно откроется в координатах, указанных в свойствах X окна и Y окна.

init_ServerLocation

IP-адрес сервера, к которому необходимо подключиться. Тип значения – string.

init_ServerPort

Порт подключения к указанному серверу. Тип значения – uint2.

init_ServerTitle

Название подключения. Отображается на верхней панели окна. Тип значения – string.

init_Credentials

Пароль от указанного сервера. Тип значения – экземпляр элемента **Учётные данные АР**.

init_ScreenKeyboard

Ссылка на экземпляр типа **Параметры вызова экранной клавиатуры**, хранящий настройки вызова экранной клавиатуры. Тип предоставляется библиотекой Alpha.HMI.CommonLib.

2.4. Explorer_Test

Форма предназначена для ознакомления с примерами использования типа и форм расширения в проекте Alpha.HMI.

Чтобы ознакомиться с этими примерами, откройте проект расширения в режиме редактирования, а затем перейдите к форме.

История изменений

3.2

Улучшения

- В окне просмотра сигнала теперь можно:
  - менять ширину таблицы значений и таблицы пользовательских свойств сигнала.
  - отменять введенное значение сигнала нажатием клавиши Escape.
- Добавлена проверка введенного интервала времени для запроса исторических значений. При вводе некорректных значений поля ввода выделяются красной рамкой.
- Формат иконок приложения изменен на SVG. Ранее использовался формат PNG.

Исправления

- Устранена причина, по которой не выполнялся запрос исторических значений сигнала.
- Устранена причина, по которой снятие флага **Следить за значением** не останавливало процесс отслеживания значений сигнала.
- Длина поля ввода значения в окне просмотра сигнала теперь меняется в зависимости от единиц измерения. Ранее длинные названия единиц измерений обрезались.
- [Alpha.HMI.WebViewer] Добавлены поля ввода интервала времени для запроса исторических значений.

3.2.1

Внутренние изменения. Функциональность продукта не изменилась.

3.2.2

Внесены доработки в изменения версии 3.2.1.

Изменения документации

Редакция 1

В руководстве пользователя:

- Актуализирован раздел 1.6. Просмотр и изменение значений сигналов (стр. 12).
- Актуализированы скриншоты.

Также скриншоты актуализированы в справочном руководстве.

3.1

Улучшения

- В окне просмотра сигнала теперь можно загрузить историю значений, указать временной интервал, а также выбрать интервал из выпадающего списка. При загрузке исторических значений флаг **Следить за значением** автоматически снимается.

- При изменении значения сигнала в окне просмотра сигнала теперь отображается сообщение об этом в журнале событий.

Изменение

В окне просмотра сигнала теперь не нужно вводить вручную значение сигнала типа bool, вместо этого можно установить или снять флаг.

Исправления

- Устранена причина, по которой кнопки панели инструментов для работы с инспектором сигналов не скрывались при повторном открытии сигнала в окне просмотра сигнала.
- Устранено некорректное форматирование сигнала типа double в окне просмотра сигнала.
- Устранена причина, по которой отображалось сообщение об ошибке после отмены ошибочно введенного значения по нажатию кнопки **Вернуть значение** в окне просмотра сигнала.
- Устранена причина, по которой некорректно отображались дата и время в таблице значений в окне просмотра сигнала.
- Устранена причина, по которой невозможно было добавить в инспектор сигнал, если его путь совпадал с уже добавленным сигналом другого источника.

Изменения документации

Редакция 1

В руководстве пользователя:

- Актуализированы разделы 1.3. Запуск самостоятельного приложения (стр. 6), 1.5. Подключение к серверу (стр. 10) и 1.6. Просмотр и изменение значений сигналов (стр. 12).
- Актуализированы скриншоты.

Также актуализировано справочное руководство.

3.0

Улучшение
Добавлена возможность просмотра информации о сигнале. Теперь один клик левой кнопкой мыши по сигналу в дереве открывает окно просмотра информации о сигнале, а два клика - добавляют сигнал в инспектор.
Изменения

- Внешний вид окна редактирования значения сигнала типа bool изменен.
- Кнопки управления инспектором сигналов на панели инструментов теперь скрыты до момента открытия инспектора.

- Изменены расположение и содержимое конфигурационных файлов приложения:
  - Настройки приложения хранятся в файлах `app_settings.json` и `settings.json`, расположенных в папке:
    - `C:\ProgramData\Automiq\HMI.Explorer` для ОС Windows;
    - `/home/<user>/Automiq/HMI.Explorer` для ОС Linux.
  - Настройки сессии приложения хранятся в файле `app_session.json`. Настройки компонентов и сессий компонентов хранятся в файлах `window.json` и `session.json`. Все перечисленные файлы расположены в папке:
    - `C:\Users\<user>\Automiq\HMI.Explorer` для ОС Windows;
    - `/home/<user>/Automiq/HMI.Explorer` для ОС Linux.

Исправление

Устранена причина, по которой перемещение сигналов в инспекторе с помощью клавиш CTRL + "Вверх" или "Вниз" выполнялось некорректно.

2.2

Улучшения

- Значения выделенных в инспекторе ячеек теперь можно скопировать, нажав сочетание клавиш CTRL + C.
- Имя выделенного в дереве сигнала или папки теперь можно скопировать, выбрав команду контекстного меню **Копировать полное имя**.

Исправления

- Устранена причина, по которой невозможно было подключиться к Alpha.Historian 4.0.0 и новее по порту 4600.
- [ОС AstraLinux] Устранена причина, по которой при работе в ОС AstraLinux невозможно было раскрыть дерево сигналов.

2.2.1

Исправление

Устранена причина несовместимости с Alpha.HMI.CommonLib 2.4.0.

Изменения документации

Редакция 1

- Бывший раздел "Запуск приложения" разделен на два раздела: 1.3. Запуск самостоятельного приложения (стр. 6) и 1.4. Встраивание в проект (стр. 8). В разделе 1.3. Запуск самостоятельного приложения (стр. 6) описана возможность изменения настроек приложения в конфигурационном файле.
- Удален раздел "Настройка приложения в конфигурационных файлах", так как часть информации из него устарела, а часть перенесена в раздел 1.3. Запуск самостоятельного приложения (стр. 6).
- В разделе 1.6. Просмотр и изменение значений сигналов (стр. 12) упомянута возможность копирования имен сигналов и папок в буфер обмена.

2.1

Новая возможность
Во внешний модуль **Explorer** добавлена новая форма **Explorer_Form**. Форма предназначена для подключения к единственному серверу, указанному в свойствах формы. Форма отличается от полноценного приложения тем, что верхняя панель инструментов сокращена: здесь нельзя подключиться к другому серверу, сохранить или открыть список сигналов.
Улучшение
Теперь в инспектор сигналов можно добавлять папки целиком, а не только сигналы по отдельности.

2.1.1

Улучшения

- Добавлено поле поиска по дереву сигналов.
- В дереве сигналов теперь справа от названия каждого сигнала отображается столбец с иконками атрибутов. По иконкам можно понять, является ли сигнал источником событий, и ведется ли для него запись истории значений.
- Типу **Explorer** добавлено свойство init_Log - ссылка на экземпляр типа **Обработчик события**, создаваемый для обработки события, произошедшего в **Explorer**. Тип предоставляется библиотекой Alpha.HMI.CommonLib.

Исправления

- Исправлено поведение, при котором нельзя было добавить в таблицу инспектора новый сигнал после превышения максимально допустимого количества сигналов в таблице и последующего удаления из нее нескольких сигналов.
- Устранена причина, по которой сигналу, доступному только для чтения, можно было присвоить новое значение.

2.1.2

Исправление

Устранена причина задержки, возникавшей при добавлении сигнала из дерева в инспектор двойным кликом мыши.

2.1.3

Исправления

- [Alpha.HMI.WebViewer] Устранена причина, по которой исчезали иконки атрибутов сигнала при наведении на него курсором мыши.
- При добавлении сигнала в инспектор тип данных отображался некорректно.
- Исправлены ошибки, возникавшие при отмене поиска по дереву сигналов до его завершения:
  - найденные результаты отображались в корне дерева;
  - при раскрытии узла вместе с ним раскрывались ранее свернутые узлы.

Изменения документации

Редакция 1

В документацию добавлено справочное руководство. В нем описаны назначение, свойства и функции компонентов, поставляемых в составе внешнего модуля **Explorer** и предназначенных для использования в проектах Alpha.HMI.

Редакция 2

В справочном руководстве обновлено описание типа **Explorer**:

- актуализирован список свойств типа;
- приведен пример использования типа в проекте Alpha.HMI.

2.0

Новая возможность
Теперь в окне приложения можно подключиться сразу к нескольким серверам. В рамках одного экземпляра приложения может быть создано не более десяти подключений одновременно. Вместе с тем:

- В контекстном меню элемента сервера добавлена команда отключения от сервера. Кнопка отключения от сервера удалена с панели инструментов.
- В таблице инспектора сигналов добавлен столбец **Источник**.
- Добавлена возможность указания учетных данных при подключении к защищенному серверу.
- Оптимизирована работа фильтра в таблице инспектора сигналов.

Улучшения

- В контекстном меню таблицы инспектора сигналов для некоторых команд теперь отображаются горячие клавиши.
- В диалоговых окнах, предполагающих ввод значений, теперь можно вызвать экранную клавиатуру. Вызов экранной клавиатуры необходимо настроить в конфигурационном файле `app_settings.json`, расположенном в `C:\ProgramData\Automiq\HMI.Explorer` (для ОС Windows) или `/home/<user>/Automiq/HMI.Explorer` (для ОС Linux).
- В окне редактирования значения сигнала всплывающая подсказка с текстом ошибки теперь отображается всегда, без необходимости наведения мыши на поле ввода.

2.0.1

Внутренние изменения. Функциональность решения не изменилась.

2.0.2

Улучшения

- В окне подключения к серверу теперь отображаются адрес и порт последнего подключения.
- Теперь при добавлении сигнала в таблицу инспектора выполняется выделение добавленной строки и прокрутка, если строка добавляемого сигнала оказывается вне видимой части инспектора.
- Сигналы, добавленные в таблицу инспектора, теперь можно перемещать вверх или вниз. Для этого следует использовать команды контекстного меню, либо сочетания клавиш.

Исправление

Устранена причина, по которой сигнал, добавленный в таблицу инспектора с включенным фильтром, не был выделен подсветкой сразу после добавления.

2.0.3

Улучшение

Панель инструментов теперь оформляется в соответствии с выбранной темой.

Исправление

Исправлена ошибка, возникавшая при попытке открыть файл со списком сигналов, содержащим разделитель строк.

2.0.4

Исправление

Исправлена ошибка, из-за которой при работе в Alpha.HMI.WebViewer было невозможно выделить несколько сигналов с помощью клавиш Shift или Ctrl.

2.0.5

Улучшение

Время в инспекторе сигналов теперь отображается с точностью до миллисекунд.

Изменения документации

Редакция 1

- Раздел **Хранение конфигурационных файлов** переименован в и расширен. Здесь описаны настройки вызова экранной клавиатуры в диалоговых окнах приложения.
- В разделе 1.5. Подключение к серверу (стр. 10) описано, как подключиться к нескольким серверам одновременно.
- Во всем документе обновлены скриншоты.

Редакция 2

Внутренние изменения. Содержимое документа не изменилось.

Редакция 3

Внутренние изменения. Содержимое документа не изменилось.

1.2

Важно. Текущая версия предназначена для использования с Alpha.HMI 1.10.
Новые возможности

- Добавлена возможность скрывать/показывать дерево сигналов по нажатию на кнопку на панели инструментов.
- Недавно открытые файлы-списки сигналов теперь можно открыть повторно, выбрав название файла из выпадающего меню рядом с кнопкой **Загрузить список** сигналов из файла. В меню отображается до десяти недавно открытых файлов.

Улучшение

В дереве сигналов появилось контекстное меню с командами:

- Добавить в инспектор сигналов (добавляет выделенный в дереве сигнал в таблицу инспектора).
- Обновить (обновляет дочерние узлы выделенного узла в дереве).

Исправленные ошибки

- Строка фильтра отображалась некорректно при горизонтальной прокрутке таблицы инспектора сигналов.
- Исправлена ошибка, возникающая в журнале времени исполнения при удалении нескольких сигналов из таблицы инспектора одновременно.
- Сигналы и разделители добавлялись в таблицу инспектора некорректно при включенном фильтре.

1.2.1

Исправление

Устранена причина дублирования идентификаторов, происходившего при попытке встроить Alpha.HMI.Explorer в проект Alpha.HMI. Для встраивания используйте папку `Explorer`, расположенную в:

- `C:\Program Files\Automiq\Alpha.HMI.Extentions` - для Windows;
- `/opt/Automiq/Alpha.HMI.Extentions` - для Linux.

1.2.2

Улучшение

Изменилось окно редактирования значения сигнала:

- Теперь внешний вид окна зависит от типа данных сигнала.
- Если у сигнала указаны единицы измерения, то они отображаются рядом с полем ввода значения.

Исправления

- Исправлена ошибка, из-за которой при изменении значения сигнала значение менялось у другого сигнала. Ошибка возникала при включенной сортировке в таблице инспектора сигналов.
- В таблице инспектора числа теперь отображаются с учетом форматирования, зависящего от типа данных сигнала.
- В сообщениях журнала и в окне редактирования значения сигнала числа теперь отображаются с учетом региональных настроек форматирования.

1.2.3

Важно. Текущая версия предназначена для использования с Alpha.HMI 2.0.

Изменения документации

Редакция 1

- Описана возможность скрывать/показывать дерево сигналов по нажатию на кнопку на панели инструментов.
- Описана возможность повторного открытия файлов-списков сигналов из выпадающего меню на панели инструментов.
- Актуализирована информация о хранении конфигурационных файлов.
- Актуализированы скриншоты во всем документе.

Редакция 2

- В раздел 1.3. Запуск самостоятельного приложения (стр. 6) добавлена информация о подключении Alpha.HMI.Explorer в проект Alpha.HMI в виде внешнего модуля.
- Информация о подключении к серверам вынесена из раздела 1.3. Запуск самостоятельного приложения (стр. 6) в самостоятельный раздел - 1.5. Подключение к серверу (стр. 10).

Редакция 3

- В разделе 1.6. Просмотр и изменение значений сигналов (стр. 12) актуализированы скриншоты и описание окна редактирования значений сигналов.
- Во всем документе актуализированы скриншоты.

Редакция 4

В разделе 1.7. Управление журналом сообщений (стр. 17) описано, как сортировать сообщения в журнале.

Редакция 5

Внутренние изменения. Содержимое документа не изменилось.

1.1

Важно. Текущая версия предназначена для использования с Alpha.HMI 1.9.
Улучшения

- Добавлена возможность запуска приложения по ярлыку из меню Пуск.
- Теперь источник при подключении можно выбрать с помощью специального диалога или из выпадающего списка источников.
- Обновлены файловые диалоги сохранения и открытия списка сигналов.

Изменения документации

Редакция 1

Руководство пользователя:

- Обновлено описание запуска приложения (стр. 6).
- Актуализирована информация о хранении конфигурационных файлов.
- Актуализированы скриншоты во всем документе.

Так как текущую версию Alpha.HMI.Explorer рекомендуется использовать только как самостоятельное приложение, Справочник API удален.

Программный комплекс Альфа платформа

Alpha.HMI.IntegrityControl 2.5

Руководство пользователя

Редакция	Соответствует версии ПО
1	2.5.0

© АО «Атомик Софт», 2015-2025. Все права защищены.

Авторские права на данный документ принадлежат АО «Атомик Софт». Копирование, перепечатка и публикация любой части или всего документа не допускается без письменного разрешения правообладателя.

Содержание

1. Руководство пользователя 5
1.1. О продукте 5
1.2. Подготовка к работе 5
1.3. Запуск в виде самостоятельного приложения 6
1.4. Встраивание в проект 8
1.5. Подключение к узлу 10
1.6. Контроль целостности 10
1.6.1. Проверка целостности 11
1.6.2. Создание эталона 13
1.6.3. Экспорт результатов проверки целостности в файл 13
1.7. Конфигурирование контроля целостности 14
1.8. Настройки безопасности для разграничения возможностей пользователей 16

2. Справочное руководство 19
2.1. IntegrityControl 19
2.1.1. Свойства 24
2.1.2. Команды 26
2.2. Разрешения 27
2.2.1. Свойства 27
2.3. Настройки 28
2.3.1. Свойства 29
2.3.2. Функции 29
2.4. IntegrityControl_App 29
2.5. IntegrityControl_KTN 29
2.5.1. Свойства 30
2.6. IntegrityControl_Form 30
2.6.1. Свойства 30
2.7. IntegrityControl_Tests 31

История изменений 32
2.5 32
Изменения документации 32
Редакция 1 32
2.4 32
2.4.1 32
Изменения документации 33
Редакция 1 33
2.3 33
Изменения документации 34
Редакция 1 34
2.2 34
Изменения документации 35
Редакция 1 35
2.1 35
2.1.1 35
2.1.2 35
Изменения документации 36
Редакция 1 36
Редакция 2 36
Редакция 3 36
2.0 36
2.0.1 37
2.0.2 37

2.0.3 37
Изменения документации 38
Редакция 1 38
Редакция 2 38
Редакция 3 38
1.3 38
1.3.1 39
Изменения документации 39
Редакция 1 39
1.2 39
1.2.1 39
1.1 39
1.1.1 40
Изменения документации 40
Редакция 2 40

1. Руководство пользователя

1.1. О продукте

Alpha.HMI.IntegrityControl - приложение, предназначенное для:

- изменения текущих настроек контроля целостности файлов и папок на локальных/удаленных узлах;
- ручного запуска контроля целостности файлов и папок на локальных/удаленных узлах и просмотра результатов проверки.

Приложение используется как дополнение к Alpha.Security и Alpha.HMI.Security, т.к. контроль целостности выполняется с помощью этих компонентов.

Используйте Alpha.HMI.IntegrityControl как самостоятельное приложение, или встраивайте его в проекты автоматизации, разработанные в Alpha.HMI.

1.2. Подготовка к работе

Требования к окружению

Для работы Alpha.HMI.IntegrityControl должны быть установлены:

- Alpha.HMI - среда разработки проектов автоматизации;
- Alpha.Security - подсистема безопасности, которую можно конфигурировать в соответствии с нуждами проекта;
- Alpha.HMI.Security - компонент, обеспечивающее взаимодействие приложения с Alpha.Security;
- Alpha.HMI.Tables - компонент, обеспечивающий отображение компонентов приложения в проекте в режиме исполнения.

Если приложение используется в веб-версии проекта автоматизации, то установите дополнительно:

- Alpha.HMI.WebViewer - для просмотра проектов Alpha.HMI в веб-интерфейсе;
- Alpha.HMI.Security.WebViewer - для работы компонентов Alpha.HMI.Security в веб-интерфейсе;
- Alpha.HMI.Tables.WebViewer - для работы компонентов Alpha.HMI.Tables в веб-интерфейсе.

Если необходимо подключить Alpha.HMI.IntegrityControl к проекту Alpha.HMI, установите также Alpha.HMI.CommonLib.

Установка, удаление или восстановление

ОС Windows

Для установки, удаления или восстановления Alpha.HMI.IntegrityControl запустите установочный файл `alpha.hmi.integritycontrol-<lng>-<version>.<arch>.msi` и следуйте инструкциям мастера.

> ПРИМЕЧАНИЕ
> В названии файла `<lng>` - это язык компонента, `<version>` - номер версии компонента, а `<arch>` - целевая процессорная архитектура.

ОС Linux

ОБРАТИТЕ ВНИМАНИЕ

Команды на установку и удаление нужно запускать с правами суперпользователя.

Для установки Alpha.HMI.IntegrityControl вызовите пакет с командой на установку в зависимости от используемого пакетного менеджера:

- rpm-пакет:

```
sudo rpm -i alpha.hmi.integritycontrol-<lng>-<version>.rpm
```

- deb-пакет:

```
sudo dpkg -i alpha.hmi.integritycontrol-<lng>-<version>.deb
```

ПРИМЕЧАНИЕ

В названии пакета `<lng>` - это язык компонента, а `<version>` - номер версии компонента.

Для удаления Alpha.HMI.IntegrityControl вызовите команду на удаление приложения в зависимости от используемого пакетного менеджера:

- rpm-пакет:

```
sudo rpm -e alpha.hmi.integritycontrol
```

- deb-пакет:

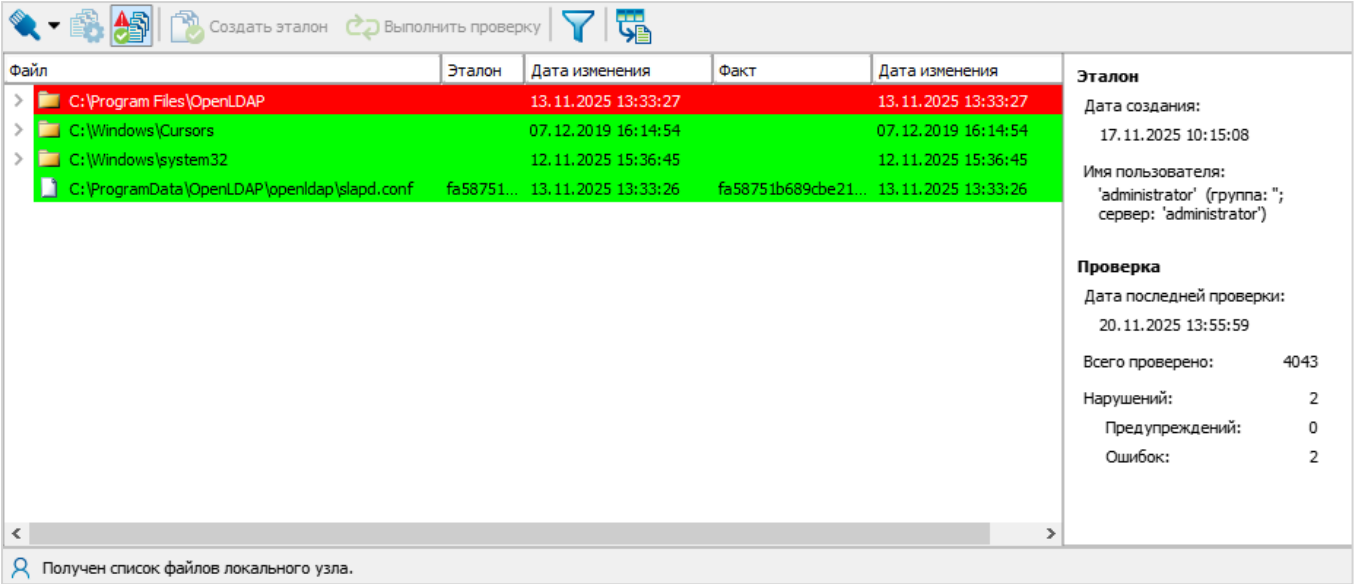
```
sudo dpkg -r alpha.hmi.integritycontrol
```

1.3. Запуск в виде самостоятельного приложения

Чтобы запустить Alpha.HMI.IntegrityControl как самостоятельное приложение, нажмите иконку приложения в меню Пуск.

Откроется окно Alpha.HMI.IntegrityControl, настройки которого можно изменить:

- изменить тему оформления;
- добавить кнопку вызова экранной клавиатуры в диалоговые окна, где требуется ввод значений;
- добавить кнопку входа в подсистему безопасности в строку состояния и сделать кнопки на панели инструментов активными. Подробнее об этом - в разделе 1.8. Настройки безопасности для разграничения возможностей пользователей (стр. 16).



Чтобы изменить настройки окна приложения Alpha.HMI.IntegrityControl, перейдите к файлу `app_settings.json`, расположенному в:

- `C:\ProgramData\Automiq\HMI.IntegrityControl` для ОС Windows;
- `/home/<user>/Automiq/HMI.IntegrityControl` для ОС Linux.

ОБРАТИТЕ ВНИМАНИЕ

Перед редактированием файла закройте приложение.

Изменение темы оформления

Чтобы изменить тему оформления приложения, в конфигурационном файле `app_settings.json` измените значения свойств объекта `Theme`. Свойства объекта соответствуют свойствам типа **Тема оформления**, предоставляемого расширением Alpha.HMI.CommonLib. Подробнее со свойствами можно ознакомиться в документации для Alpha.HMI.CommonLib.

После изменения значений свойств сохраните файл и перезапустите приложение. Его внешний вид будет изменен.

Добавление кнопки вызова экранной клавиатуры

Чтобы добавить в диалоговые окна кнопку вызова экранной клавиатуры:

1. В конфигурационном файле `app_settings.json` измените значения свойств объекта `ScreenKeyboard` следующим образом:
   - включите вызов экранной клавиатуры, изменив значение свойства `Enabled` с **«false»** на **«true»**.
   - укажите полный путь к программе экранной клавиатуры в качестве значения свойства `ProgramPath`. Новое значение можно не указывать, тогда по умолчанию будет использоваться:
     - в Windows - `alpha.hmi.keyboard.exe` (поставляется вместе с дистрибутивом Alpha.HMI).
     - в Astra Linux - `/usr/bin/fly-vkbd`. Для других ОС Linux расположение экранной клавиатуры может быть другим.
   - укажите аргументы запуска программы экранной клавиатуры в качестве значения свойства `Arguments` (зависят от используемой программы).
2. Сохраните файл и перезапустите приложение.

Во всех диалоговых окнах, где требуется ввод значений, появится кнопка вызова указанной экранной клавиатуры.

Настройки сессии приложения

Приложение также сохраняет файл `app_session.json` - он хранит данные о размере и положении окна приложения во время последней сессии приложения.

Файл сохраняется в:

- `C:\Users\<user>\Automiq\HMI.IntegrityControl` для ОС Windows;
- `/home/<user>/Automiq/HMI.IntegrityControl` для ОС Linux.

Файл используется для работы приложения, менять его содержимое не рекомендуется.

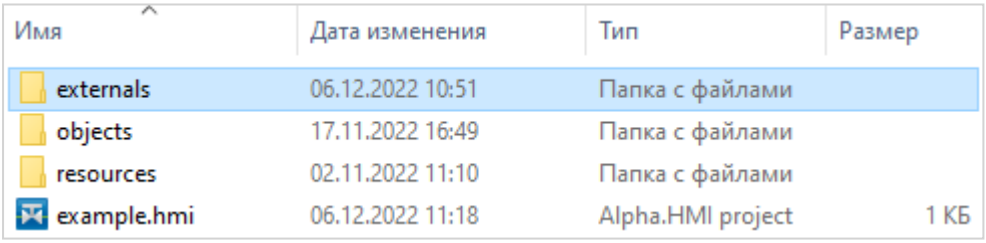
1.4. Встраивание в проект

Чтобы встроить Alpha.HMI.IntegrityControl в проект автоматизации, разработанный в Alpha.HMI, подключите проект расширения в свой проект в виде внешнего модуля.

ОБРАТИТЕ ВНИМАНИЕ

Для встраивания Alpha.HMI.IntegrityControl в проект Alpha.HMI понадобится библиотека Alpha.HMI.CommonLib.

1. Чтобы подключать проекты в виде внешних модулей, создайте в папке своего проекта папку с именем `externals`.

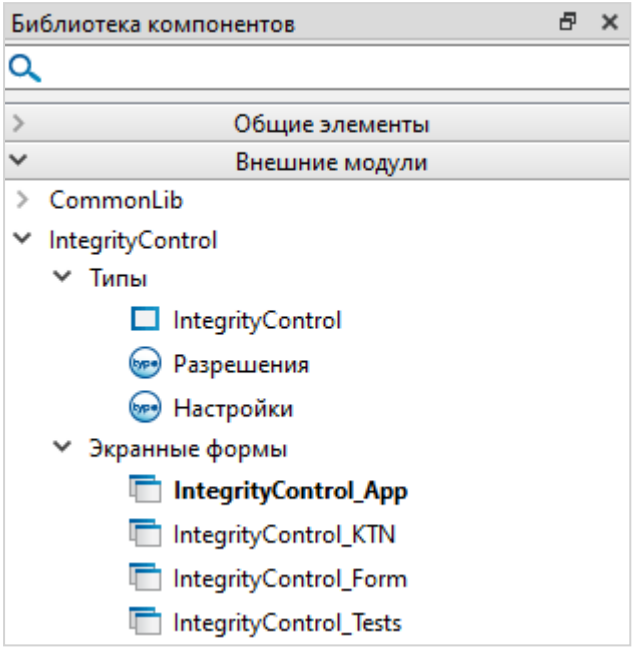


Имя	Дата изменения	Тип	Размер
externals	06.12.2022 10:51	Папка с файлами	
objects	17.11.2022 16:49	Папка с файлами	
resources	02.11.2022 11:10	Папка с файлами	
example.hmi	06.12.2022 11:18	Alpha.HMI project	1 КБ

2. Перейдите к папке с расширениями Alpha.HMI:
   - `C:\Program Files\Automiq\Alpha.HMI.Extensions` (ОС Windows);
   - `/opt/Automiq/Alpha.HMI.Extensions` (ОС Linux).

   Отсюда скопируйте папку `IntegrityControl` и папку библиотеки `CommonLib` в созданную ранее `externals`.

3. Запустите свой проект в Дизайнер Alpha.HMI. В меню **Проект** на панели инструментов выберите **Обновить ссылки на внешние юниты**. В результате в библиотеке компонентов появятся юниты внешних модулей - **`IntegrityControl`** и **`CommonLib`**.



Внешний модуль **`IntegrityControl`** предоставляет типы и экранные формы.

- Чтобы добавить экземпляр типа в проект, просто перетяните его на рабочую область.
- Чтобы использовать экранную форму в проекте, вызовите ее в обработчике какого-либо события или перетяните на рабочую область.

С назначением типов и экранных форм можно ознакомиться в Справочном руководстве.

Обновить Alpha.HMI.IntegrityControl в проекте Alpha.HMI

Если Alpha.HMI.IntegrityControl уже встроен в проект в виде внешнего модуля, то при установке новой версии необходимо обновить файлы модуля в папке `externals` проекта. Для этого:

1. Установите нужную версию Alpha.HMI.IntegrityControl.
2. Замените существующую папку `IntegrityControl` на папку `IntegrityControl`, расположенную в папке установки расширения:
   - `C:\Program Files\Automiq\Alpha.HMI.Extensions` (ОС Windows);
   - `/opt/Automiq/Alpha.HMI.Extensions` (ОС Linux).
3. Если устанавливали новую версию Alpha.HMI.CommonLib, не забудьте аналогично обновить файлы модуля в папке `externals` проекта.
4. Запустите свой проект в Дизайнер Alpha.HMI. В меню **Проект** на панели инструментов выберите **Обновить ссылки на внешние юниты**. В результате внешние модули будут обновлены. Экранные формы и типы, уже добавленные в проект, обновятся автоматически.

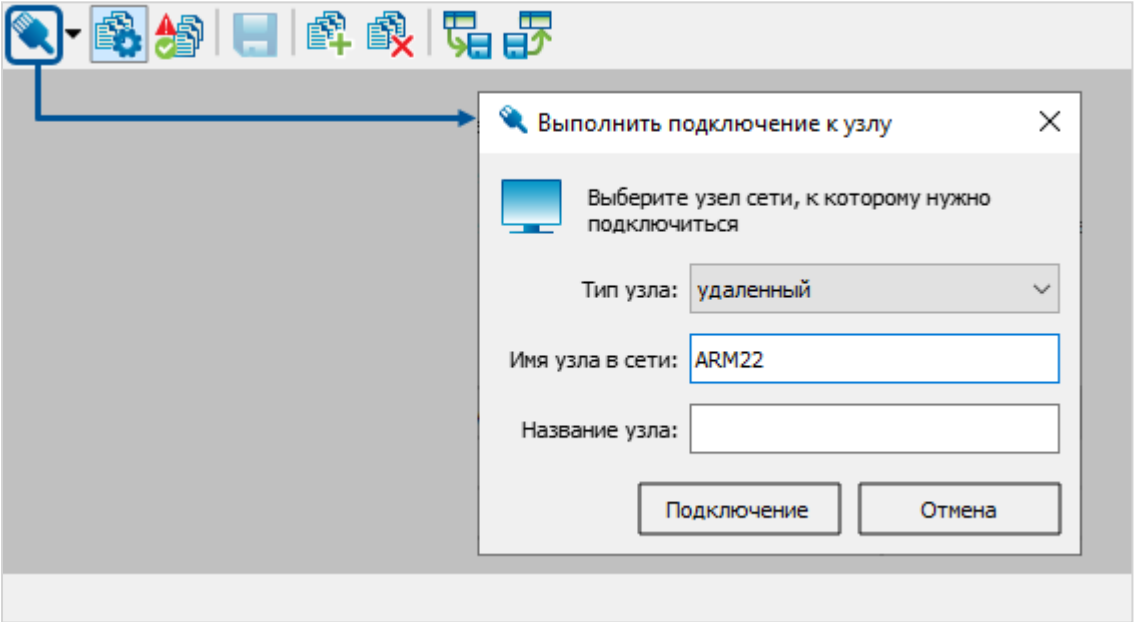
1.5. Подключение к узлу

ОБРАТИТЕ ВНИМАНИЕ

Если вы планируете подключаться к удаленным узлам, перед использованием Alpha.HMI.IntegrityControl обязательно объедините свой компьютер и удаленные узлы в сеть Alpha.Net. Настройте службу **Alpha.Net.Agent** на каждом узле, включая узел с установленным Alpha.HMI.IntegrityControl. Как организовать сеть Alpha.Net, описано в руководстве пользователя на Alpha.Domain.

Приложение по умолчанию подключено к локальному узлу. Вы можете сразу перейти к контролю целости (стр. 10) либо к его конфигурированию (стр. 14).

Чтобы подключиться к удаленному узлу, нажмите **Выполнить подключение к узлу** на панели инструментов. В открывшемся окне укажите тип узла - **«удаленный»** - и имя узла. Имя узла указывается в конфигурационном файле службы **Alpha.Net.Agent**. Название узла вводить не обязательно - оно используется для удобства выбора подключения из списка подключений в дальнейшем.

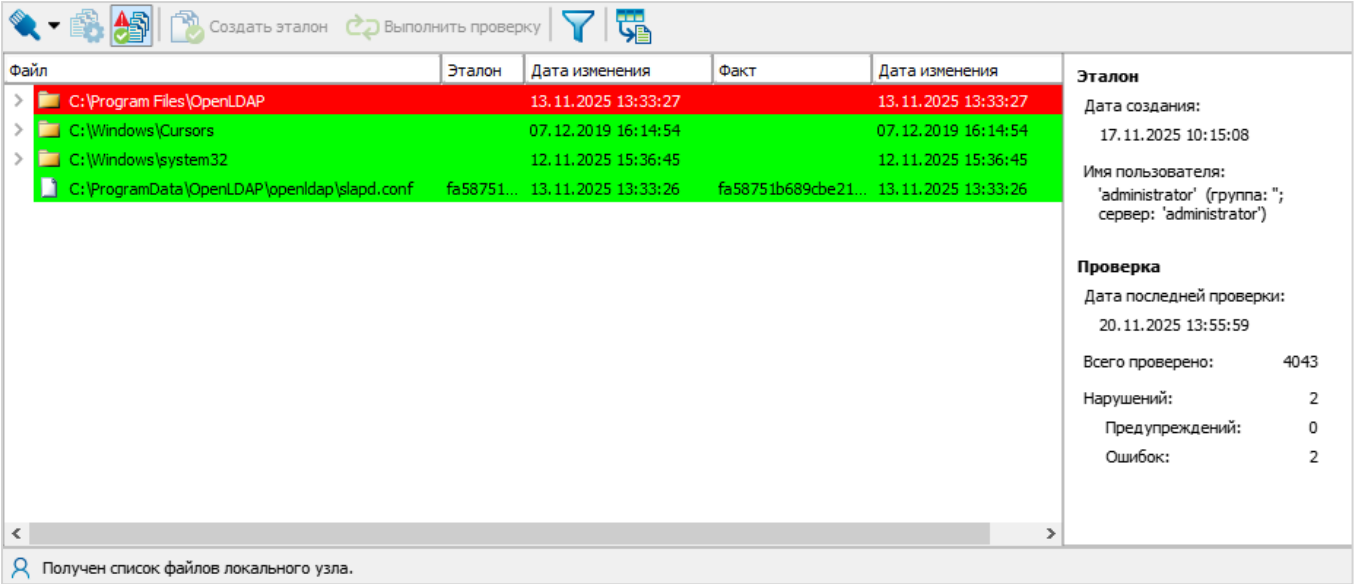


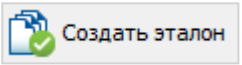
Если подключение выполнено успешно, можно перейти к контролю целости (стр. 10) либо к его конфигурированию (стр. 14).

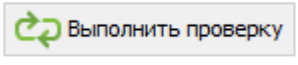
1.6. Контроль целостности

Нажмите **Контроль целостности** на панели инструментов, чтобы перейти к просмотру текущего состояния контролируемых файлов, запустить проверку целостности или создать эталон состояния файлов. Откроется окно контроля целостности, где:

- в левой части окна отображен список контролируемых файлов и папок, цвет строк которого соответствует состоянию целостности файла/папки;
- в правой части окна отображена информация об эталоне и последней проверке целостности.



Кнопки **Конфигурирование контроля целостности** , 

и 

активны для пользователей с соответствующими правами. Подробнее о том, как настроить права пользователей, написано в разделе 1.8. Настройки безопасности для разграничения возможностей пользователей (стр. 16).

ПРИМЕЧАНИЕ

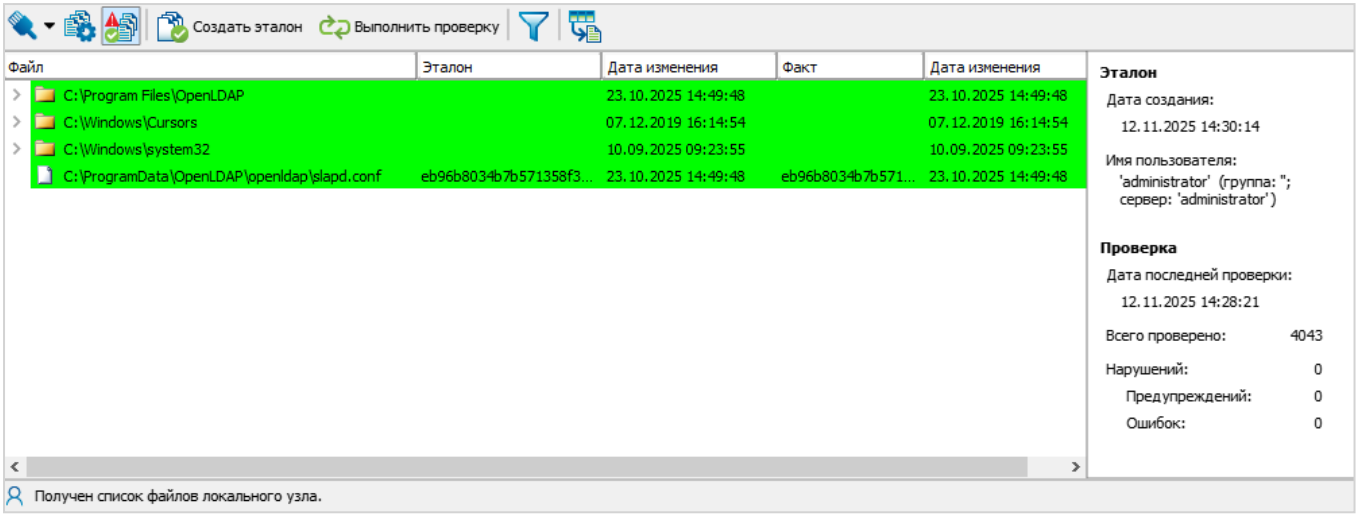
Если вы установили Alpha.Security впервые, контроль целостности будет выполняться для некоторых системных файлов и папок, приведенных на скриншоте выше.

Если вы обновили Alpha.Security до версии 1.7.24 или новее, то список контролируемых файлов будет соответствовать настройкам контроля целостности, выполненных вами до обновления Alpha.Security.

Как изменить набор файлов и папок, целостность которых должна контролироваться, или изменить другие настройки контроля целостности, описано в разделе 1.7. Конфигурирование контроля целостности (стр. 14).

1.6.1. Проверка целостности

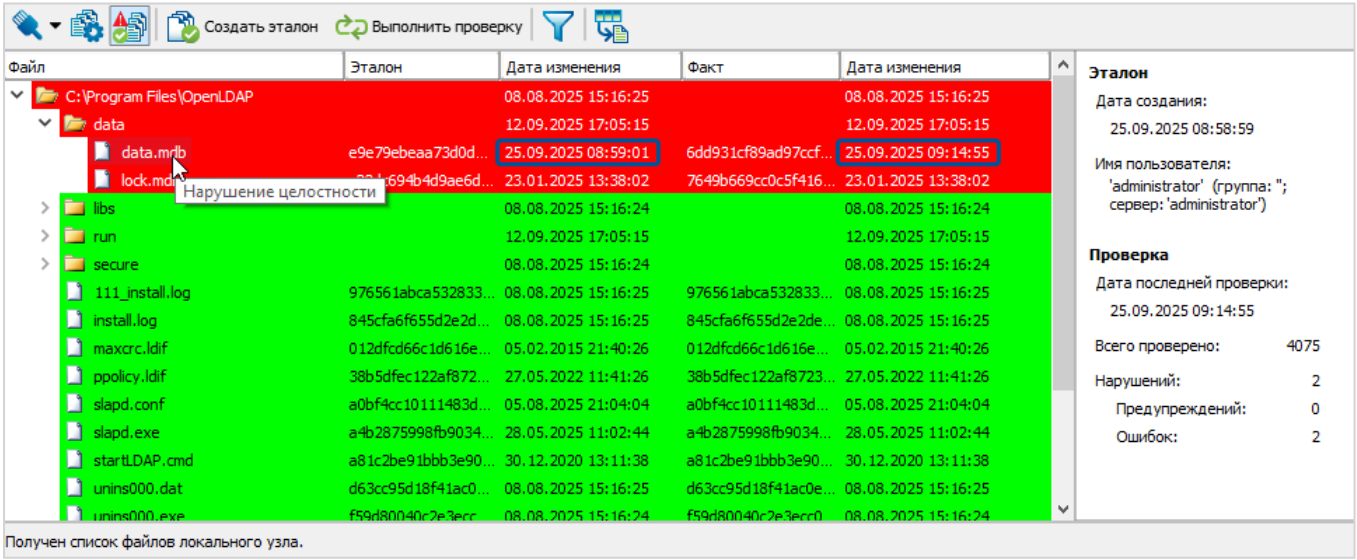
При проверке целостности проверяется соответствие фактического состояния файлов эталонному состоянию. Для каждого файла вы можете увидеть эталонную контрольную сумму и эталонную дату изменения, а также фактическую контрольную сумму и фактическую дату изменения. Файлы и папки, для которых обнаружено нарушение целостности, по умолчанию выделяются красным цветом. Цвет строки зависит от серьезности нарушения. Серьезность нарушений определяется в настройках контроля целостности.



ПРИМЕЧАНИЕ

Кнопка Выполнить проверку активна, если приложение использует подсистему безопасности Alpha.Security и пользователю назначено право на выполнение проверки целостности файлов. Подробнее о том, как настроить права пользователей, написано в разделе 1.8. Настройки безопасности для разграничения возможностей пользователей (стр. 16).

Проверка может выполняться автоматически с заданным периодом, если это указано в настройках контроля целостности, либо может быть запущена вручную. Чтобы вручную запустить проверку целостности, нажмите кнопку Выполнить проверку на панели инструментов. В результате проверки по каждому файлу вы сможете отследить, был ли изменен файл, и когда он был изменен. Чтобы посмотреть подробную информацию о нарушении, наведите курсор на строку с нарушением.

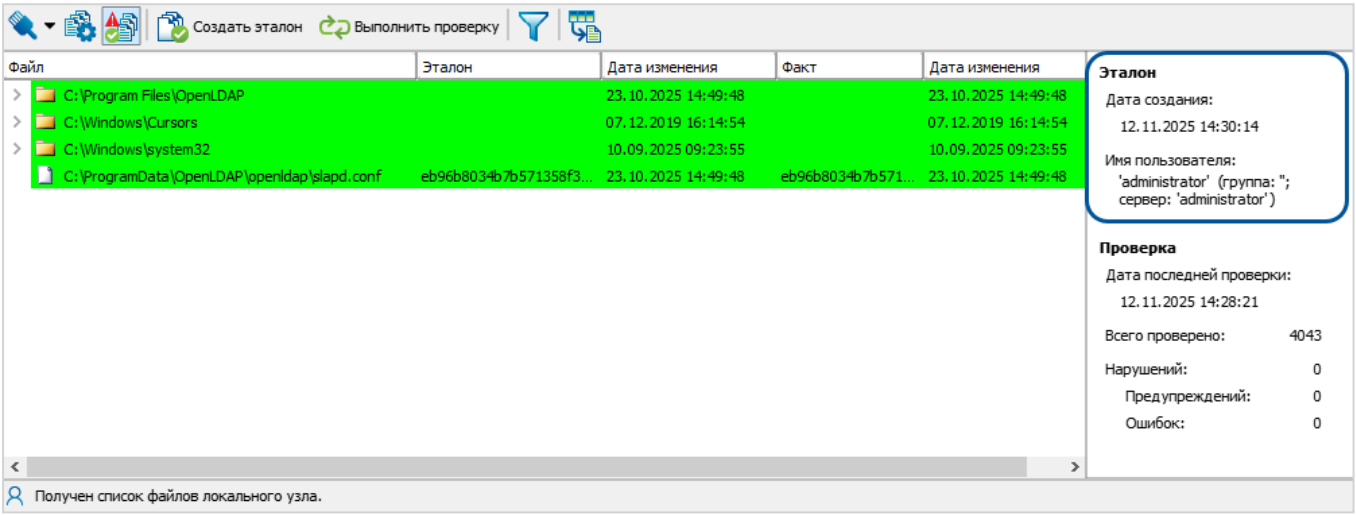


Чтобы отобразить только файлы с нарушением целостности, нажмите кнопку **Показать только файлы с нарушением целостности** на панели инструментов. Чтобы снова отобразить все файлы, нажмите кнопку еще раз.

1.6.2. Создание эталона

Под эталоном понимается срез текущего состояния (контрольная сумма) по каждому файлу, с которым будет сравниваться состояние файлов при следующей проверке.

Чтобы создать новый эталон, нажмите кнопку Создать эталон на панели инструментов. В результате информация об эталоне обновится в правой части окна приложения.



ПРИМЕЧАНИЕ

Кнопка Создать эталон активна, если приложение использует подсистему безопасности Alpha.Security и пользователю назначено право на создание эталона. Подробнее о том, как настроить права пользователей, написано в разделе 1.8. Настройки безопасности для разграничения возможностей пользователей (стр. 16).

1.6.3. Экспорт результатов проверки целостности в файл

Вы можете сохранить результаты проверки целостности в виде таблицы в файл формата `*.csv`, `*.xlsx` или `*.pdf`.

Номер	Каталог	Файл и расширение	Эталонная контрольная сумма	Текущая контрольная сумма	Эталонная дата и время	Текущая дата и время	Нарушение
1	C:\Program Files\OpenLDAP						
1.1	C:\Program Files\OpenLDAP\data						
1.1.1		C:\Program Files\OpenLDAP\data\data.mdb	50f229ced1b0afc26d8f8f7cddbd8d7c	50f229ced1b0afc26d8f8f7cddbd8d7c	15.08.2024 10:57:14	15.08.2024 10:57:14	
1.1.2		C:\Program Files\OpenLDAP\data\lock.mdb	7a88fc3b9180b959794134f6b9730e72	993f2d45534a459ccdea362d26798408	09.07.2024 07:44:54	09.07.2024 07:44:54	Нарушение целостности

По каждому файлу в таблицу сохраняется:

- имя файла и расширение;
- эталонная контрольная сумма;
- текущая контрольная сумма;
- эталонная дата и время;
- текущая дата и время;
- информация о нарушении целостности.

Чтобы сохранить результаты проверки в файл, нажмите кнопку **Экспорт** на панели инструментов. В открывшемся окне выберите место сохранения и формат файла. По умолчанию в открывшемся окне выбрана папка:

- `C:\ProgramData\Automiq\HMI.IntegrityControl\Export` - для ОС Windows;
- `/home/<user>/Automiq/HMI.IntegrityControl/Export` - для ОС Linux.

Папку, открываемую по умолчанию, можно изменить. Для этого перейдите к файлу `settings.json`, расположенному в:

- `C:\ProgramData\Automiq\HMI.IntegrityControl` - для ОС Windows;
- `/home/<user>/Automiq/HMI.IntegrityControl` - для ОС Linux.

Редактируйте файл, указав полный путь к новой папке экспорта в качестве значения свойства `ExportFolder` объекта `Folders`. Сохраните файл и перезапустите приложение. При экспорте результатов проверок в файлы по умолчанию будет открываться указанная папка.

1.7. Конфигурирование контроля целостности

Информация о том, целостность каких файлов и папок контролируется, с какой периодичностью и т.п., называется конфигурацией контроля целостности, и хранится в виде зашифрованных файлов в папке:

- `C:\Program Files\Automiq\Alpha.Security\IC` для ОС Windows;
- `/opt/Automiq/Alpha.Security/IC` для ОС Linux.

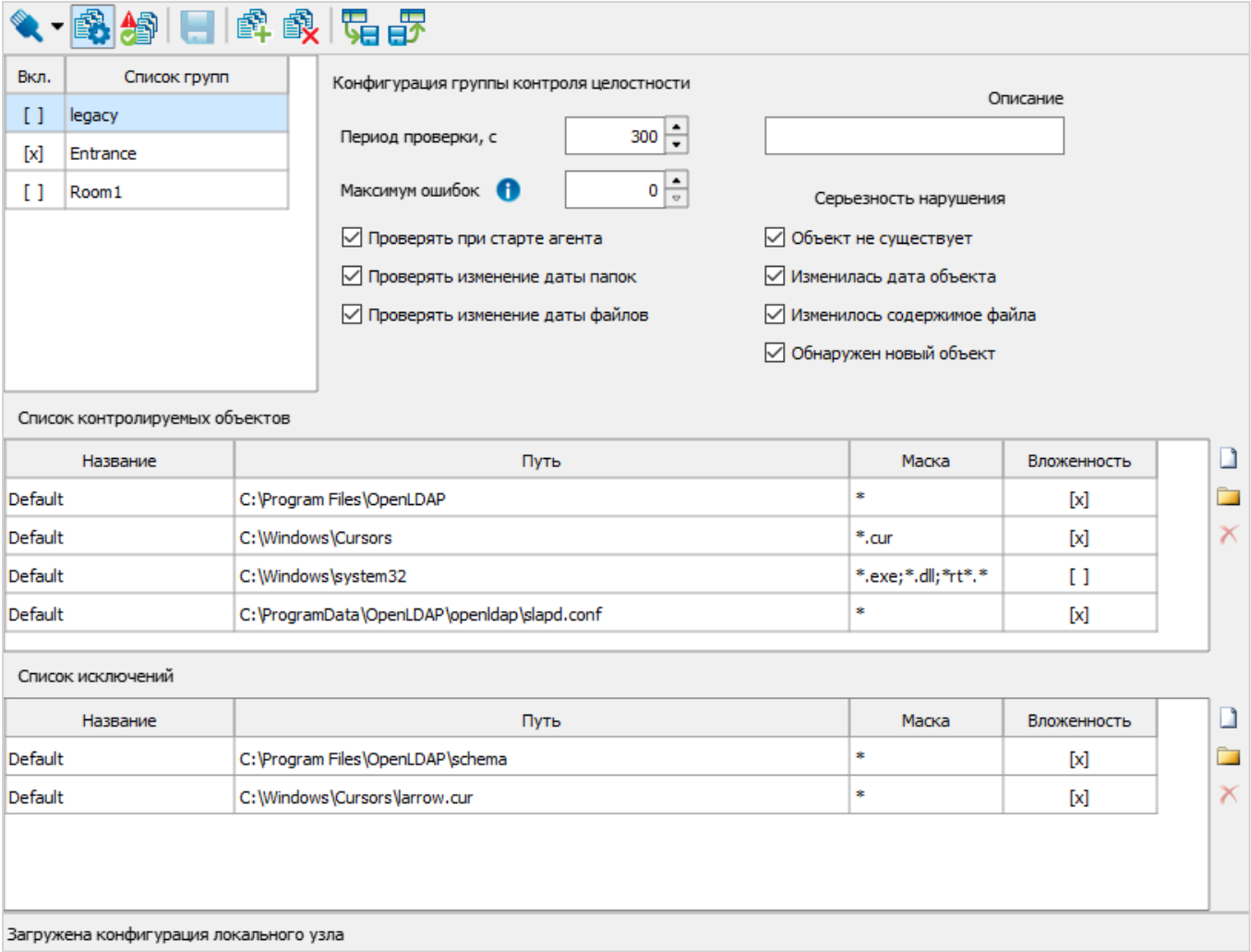
Каждый файл представляет собой определенный набор настроек контроля целостности и называется группой контроля целостности.

ОБРАТИТЕ ВНИМАНИЕ

Агент безопасности может хранить несколько групп одновременно, однако активна может быть только одна из них.

Чтобы ознакомиться со списком групп, просмотреть и изменить настройки контроля целостности групп, нажмите **Конфигурирование контроля целостности** на панели инструментов. Откроется окно конфигурирования контроля целостности, где:

- слева вверху отображен список групп контроля целостности;
- справа вверху отображены текущие настройки контроля целостности для группы, выбранной в списке групп;
- ниже отображен список контролируемых файлов и папок для группы, выбранной в списке групп.



ПРИМЕЧАНИЕ

Кнопка **Конфигурирование контроля целостности** активна, если приложение использует подсистему безопасности Alpha.Security и пользователю назначено право на просмотр конфигурации. Подробнее о том, как настроить права пользователей, написано в разделе 1.8. Настройки безопасности для разграничения возможностей пользователей (стр. 16).

Группа **legacy** - группа, содержащая либо настройки целостности по умолчанию (если вы установили Alpha.Security впервые), либо настройки целостности, выполненные до обновления Alpha.Security до версии 1.7.24 или новее. Вы можете либо изменить настройки для группы **legacy**, либо добавить новую группу контроля целостности.

- Чтобы изменить настройки группы **legacy**, выберите ее в списке групп.
- Чтобы добавить новую группу контроля целостности, нажмите **Добавить группу** на панели инструментов. В списке групп появится группа **newgroup_1**. Чтобы переименовать группу, дважды кликните левой кнопкой мыши по добавленной группе и введите новое имя. Обратите внимание, что новая группа по умолчанию активна. По завершении конфигурирования контроля целостности активной может остаться только одна группа.

Чтобы изменить настройки контроля целостности текущей группы, укажите новые значения полей ввода и флагов в разделе **Конфигурация группы контроля целостности**. Здесь можно изменить:

- периодичность автоматической проверки целостности контролируемых файлов и папок;
- настройки автоматических проверок целостности и т.п.

Чтобы редактировать список контролируемых файлов и папок, используйте кнопки , и разделов **Список контролируемых объектов** и **Список исключений**.

По завершении редактирования нажмите **Сохранить изменения** на панели инструментов.

Сохранение и загрузка конфигурации из файла

Конфигурацию контроля целостности можно сохранить в файл формата `*.json`, чтобы затем загрузить на другом устройстве или восстановить в случае повреждения текущей конфигурации. Для этого на панели инструментов нажмите **Сохранить конфигурацию в файл** . В открывшемся окне укажите расположение файла, а затем нажмите **Сохранить**.

Чтобы загрузить конфигурацию из файла формата `*.json`, нажмите **Загрузить конфигурацию из файла** . В открывшемся окне укажите расположение файла, а затем нажмите **Открыть**.

ПРИМЕЧАНИЕ

Кнопки сохранения и загрузки конфигурации отображаются только для пользователей с правом на редактирование конфигурации.

1.8. Настройки безопасности для разграничения возможностей пользователей

Можно разграничить возможности пользователей:

- в окне приложения Alpha.HMI.IntegrityControl;
- в окне собственного приложения, разработанного на основе элементов расширений Alpha.HMI.IntegrityControl и Alpha.HMI.CommonLib, и предназначенного для выполнения контроля целостности файлов.

Прежде чем перейти к настройкам приложений, добавьте в текущую конфигурацию Alpha.Security права, ограничивающие возможности пользователей.

Добавление прав в Alpha.Security

Ограничения для пользователей можно установить, используя подсистему безопасности Alpha.Security. Необходимые для этого права описаны в таблице в конце этого раздела. Их можно создать самостоятельно, либо - если вы используете Alpha.HMI.SecurityConfigurator - загрузить из шаблона приложения, поставляемого вместе с дистрибутивом конфигуратора. Для этого в окне просмотра приложений в конфигураторе следует нажать кнопку импорта из файла. В открывшемся окне указать путь к папке с шаблонами приложений:

- `C:\ProgramData\Automiq\HMI.SecurityConfigurator\AppTemplates` для ОС Windows;
- `/usr/local/share/Automiq/HMI.SecurityConfigurator/AppTemplates` для ОС Linux.

Загрузите шаблон **IntegrityControl**. Приложение появится в текущей конфигурации подсистемы безопасности. Назначьте их нужным пользователям.

Еще для разграничения возможностей пользователей используется системное право **Просмотр конфигурации**:

- если текущему пользователю назначено право **Просмотр конфигурации**, то кнопка **Конфигурирование контроля целостности** на панели инструментов самостоятельного приложения будет активна для этого пользователя;
- если текущему пользователя не назначено такое право, кнопка будет неактивна.

Настройка безопасности в окне самостоятельного приложения Alpha.HMI.IntegrityControl

Чтобы включить использование подсистемы безопасности приложением:

1. В конфигурационном файле `app_settings.json` измените значение свойства `UseSecurity` объекта `Permissions` с **«false»** на **«true»**. Файл расположен в:
   - `C:\ProgramData\Automiq\HMI.IntegrityControl` для ОС Windows;
   - `/home/<user>/Automiq/HMI.IntegrityControl` для ОС Linux.
2. Сохраните файл и перезапустите приложение.

В левом нижнем углу окна приложения появится кнопка входа, открывающая диалог авторизации пользователя, а возможности пользователей в окне приложения будут зависеть от их прав.

Настройка безопасности в окне собственного приложения

Предположим, вы разрабатываете приложение, в котором можно создать эталон и запустить проверку целостности. В проекте такого приложения можно использовать либо тип 2.1. IntegrityControl (стр. 19), либо форму 2.6. IntegrityControl_Form (стр. 30), либо форму 2.5. IntegrityControl_KTN (стр. 29).

Пусть необходимо ограничить доступ пользователей к созданию эталонов, проверке файлов и файловой системе. Для этого необходимо добавить в проект экземпляр типа 2.2. Разрешения (стр. 27), поставляемого в составе модуля Alpha.HMI.IntegrityControl. У этого экземпляра есть свойства **Проверка файлов**, **Создание эталонных значений**, **Доступность файловой системы**, значения которых можно изменять. Значения свойств могут быть установлены вами, а могут вычисляться на основе значений прав текущего пользователя в подсистеме безопасности Alpha.Security.

Чтобы получать значения прав текущего пользователя, необходимо выполнить следующие шаги.

1. Укажите для созданного экземпляра типа 2.2. Разрешения (стр. 27) в качестве значения свойства **Использовать сервер безопасности для определения прав** значение **«true»**, а в качестве значения свойства **Имя приложения с правами безопасности** - название приложения, созданного вами в текущей конфигурации Alpha.Security.
2. Для типа или формы, используемой в проекте для создания эталонов и запуска проверок целостности, в качестве значения свойства **Разрешения** укажите ссылку на созданный вами экземпляр типа 2.2. Разрешения (стр. 27).

Теперь для определения возможности создания эталонов, запуска проверок и смены папки экспорта для текущего пользователя будут использоваться значения прав, назначенных ему в подсистеме безопасности.

Права из шаблона приложения IntegrityControl

Право	Назначение
Проверка файлов (CheckFiles)	Значение права регулирует возможность проверки целостности файлов для конкретного пользователя (или группы). При входе пользователя в подсистему безопасности свойство **Проверка файлов** экземпляра типа 2.2. Разрешения (стр. 27) принимает значение, с которым это право назначено текущему пользователю.
Создание эталонных значений (CreateEtalon)	Значение права регулирует возможность создания эталонных значений для конкретного пользователя (или группы). При входе пользователя в подсистему безопасности свойство **Создание эталонных значений** экземпляра типа 2.2. Разрешения (стр. 27) принимает значение, с которым это право назначено текущему пользователю.
Доступность файловой системы (FileSystemAccess)	Значение права регулирует доступность файловой системы для конкретного пользователя (или группы). При входе пользователя в подсистему безопасности свойство **Доступность файловой системы** экземпляра типа 2.2. Разрешения (стр. 27) принимает значение, с которым это право назначено текущему пользователю.

2. Справочное руководство

2.1. IntegrityControl

`IntegrityControl` - тип, предназначенный для размещения на форме в проекте Alpha.HMI.

Тип не является полноценной заменой приложения Alpha.HMI.IntegrityControl: экземпляр типа отличается от приложения тем, что в нем отсутствует верхняя панель инструментов.

Тип, используемый в проекте Alpha.HMI, в режиме рантайма создает файлы, описанные в таблице ниже.

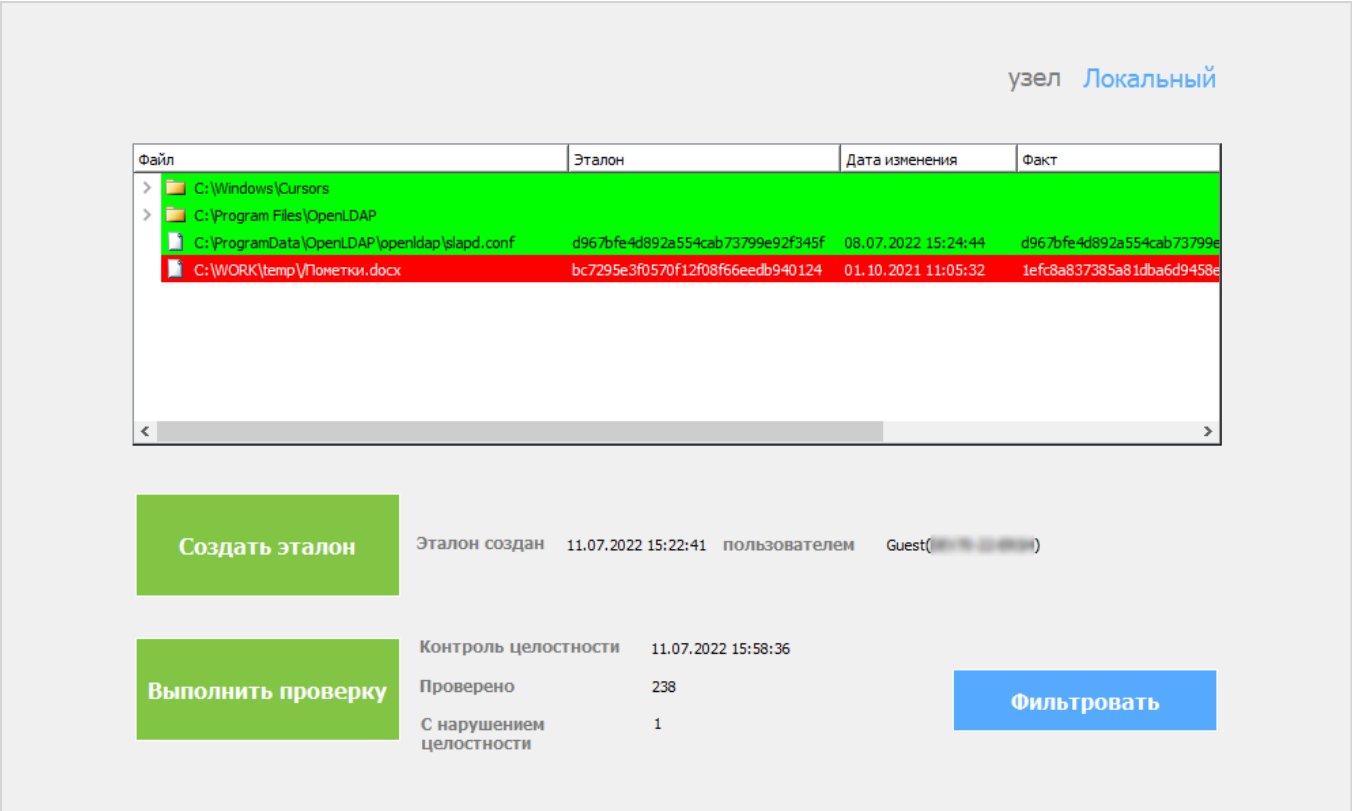
Файл	Расположение	Назначение
`settings.json`	› `C:\ProgramData\Automiq\HMI.IntegrityControl` для ОС Windows; › `/home/<user>/Automiq/HMI.IntegrityControl` для ОС Linux.	Хранит информацию о папке экспорта файлов. Используется для указания папки, открываемой при экспорте файлов по умолчанию.
`session.json`	› `C:\Users\<user>\Automiq\HMI.IntegrityControl` для ОС Windows; › `/home/<user>/Automiq/HMI.IntegrityControl` для ОС Linux.	Хранит информацию об окне контроля целостности: размере колонок таблицы, установленном фильтре и о типе последнего сохраненного файла, в который был выполнен экспорт информации. Файл используется для работы типа, менять его содержимое не рекомендуется.

Пример использования типа в проекте

В примере показано, как:

1. Вывести на форму дерево контролируемых файлов на узле Alpha.Net.
2. Добавить кнопку создания эталона и вывести на форму дату/время создания эталона и пользователя, создавшего эталон.
3. Добавить кнопку выполнения проверки контроля целостности и вывести на форму дату/время выполнения проверки, количество проверяемых файлов и количество файлов с нарушением целостности.
4. Добавить кнопку фильтрации.
5. Вывести на форму имя узла.

В результате подготовлена форма с пользовательским вариантом интерфейса Alpha.HMI.IntegrityControl.

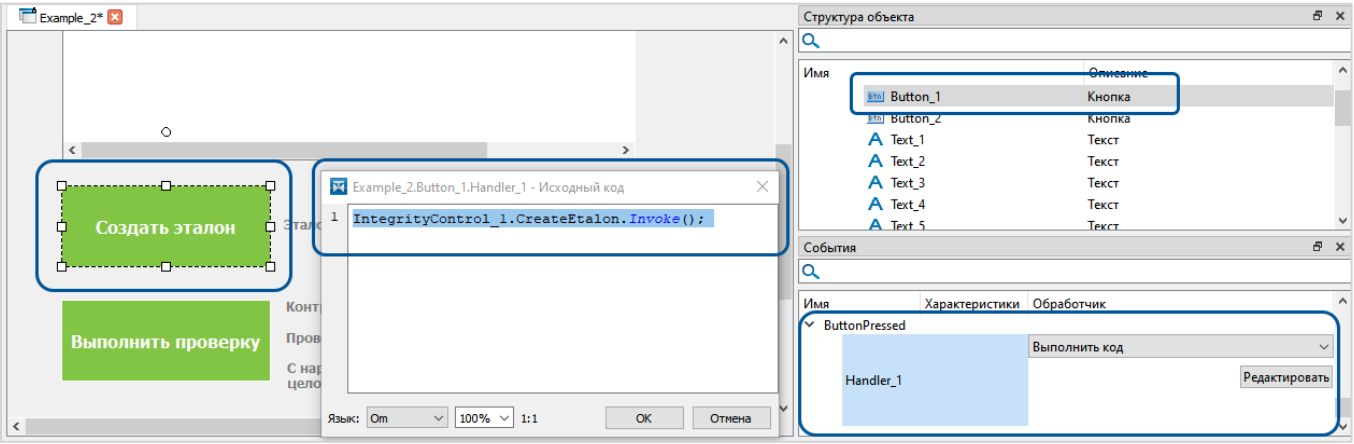


Чтобы начать работу, добавьте на форму экземпляр типа **`IntegrityControl`**. Укажите имя узла, на котором будет выполняться контроль целостности, в свойстве Имя узла сети добавленного экземпляра *`IntegrityControl_1`*.
Если контроль целостности выполняется на вашем локальном узле, при запуске формы в режим исполнения вы увидите дерево контролируемых файлов.

Добавить кнопку создания эталона

Чтобы создать кнопку, по нажатию на которую будет создаваться эталон, используйте команду CreateEtalon экземпляра *`IntegrityControl_1`*. Добавьте на форму кнопку **Создать эталон** и в обработчике события ButtonPressed укажите:

```
IntegrityControl_1.CreateEtalon.Invoke();
```

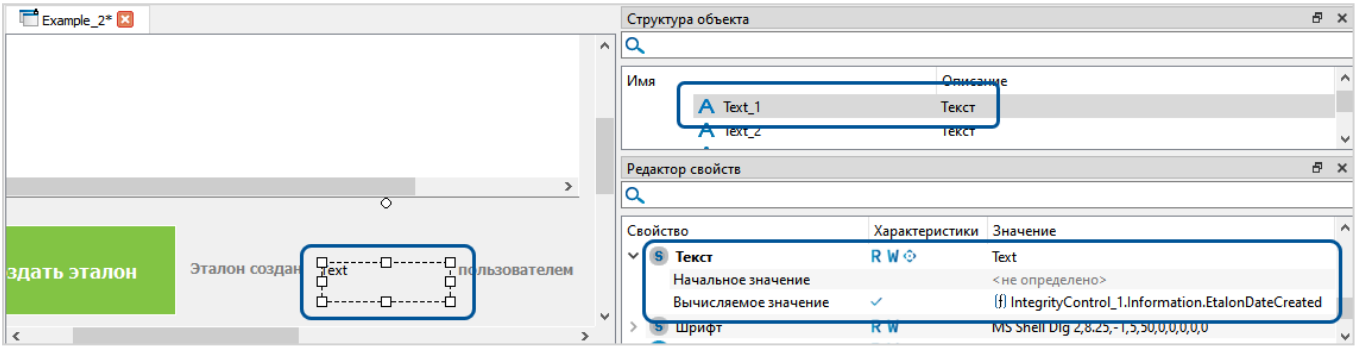


Вывести статистику создания эталона

Для вывода статистических данных создания эталона (дата/время и имя пользователя) подготовьте два текстовых поля. Чтобы вывести дату/время и имя пользователя, нужен объект Information (стр. 25) и его поля EtalonDateCreated и EtalonNameCreator.

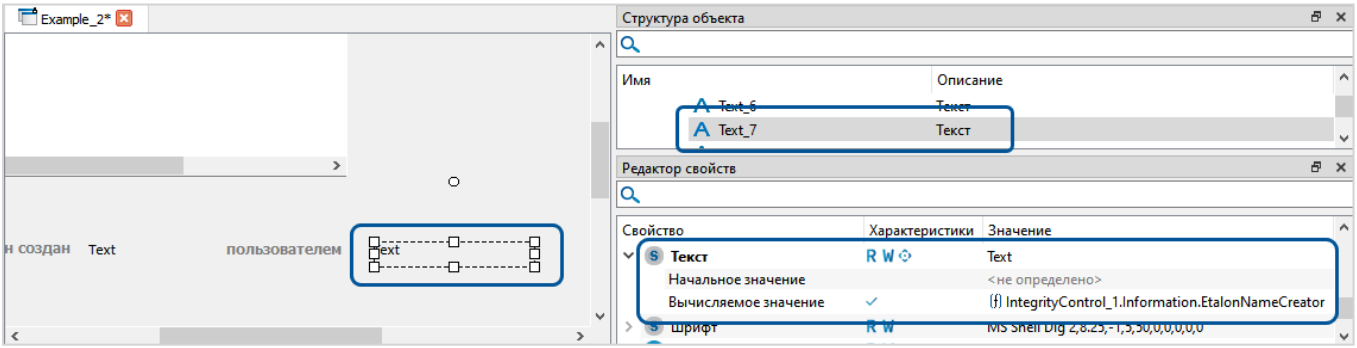
Пусть дата и время создания эталона нужно вывести в поле *Text_1*. Для этого перейдите к свойству **Текст** поля и укажите в качестве вычисляемого значения формулу:

```
IntegrityControl_1.Information.EtalonDateCreated
```



Пусть имя пользователя, создавшего эталон, нужно вывести в поле *Text_7*. Для этого перейдите к свойству **Текст** поля и укажите в качестве вычисляемого значения формулу:

```
IntegrityControl_1.Information.EtalonNameCreator
```

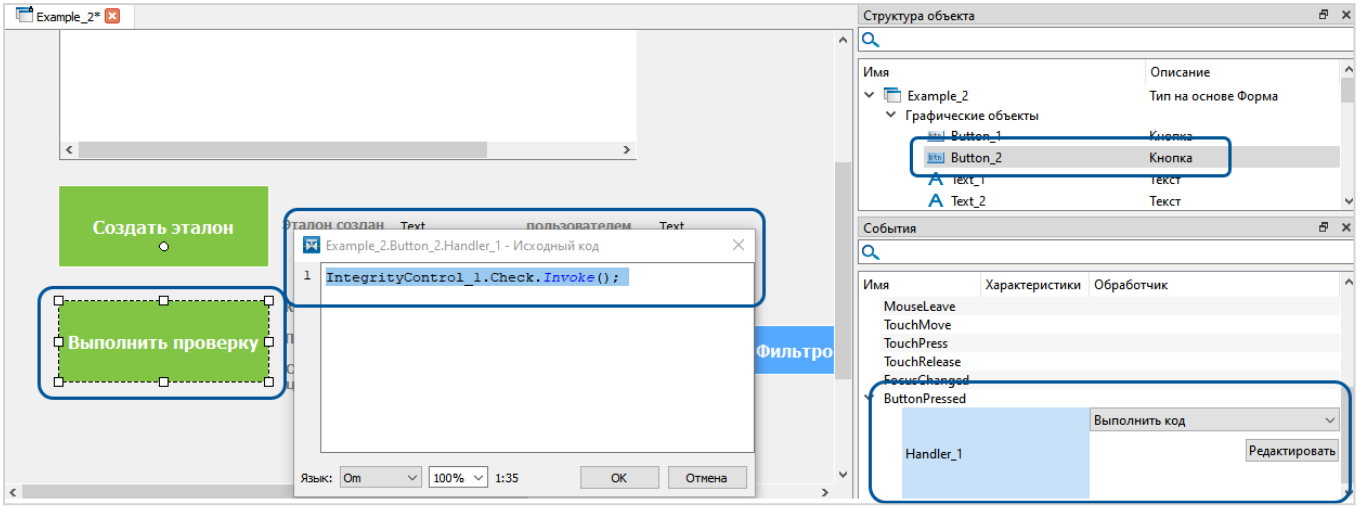


Добавить кнопку выполнения проверки целостности

Можно создать кнопку, нажатие на которую будет запускать контроль целостности. Для этого используется команда Check экземпляра *IntegrityControl_1*.

Создайте кнопку и в обработчике события **ButtonPressed** этой кнопки укажите:

```
IntegrityControl_1.Check.Invoke();
```



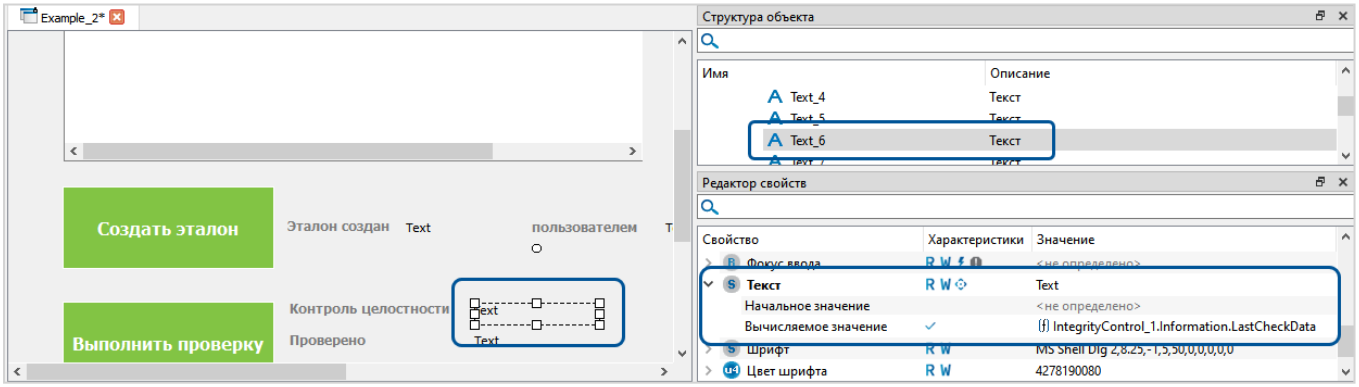
Вывести статистику выполнения проверки

Для вывода количества проверенных файлов, даты/времени последней проверки и количества файлов с нарушением целостности, подготовьте три текстовых поля.

Чтобы вывести дату и время последней проверки нужен объект Information (стр. 25) и его поле LastCheckData.

Пусть дата и время последней проверки нужно вывести в поле *Text_6*. Для этого перейдите к свойству **Текст** поля и укажите в качестве вычисляемого значения формулу:

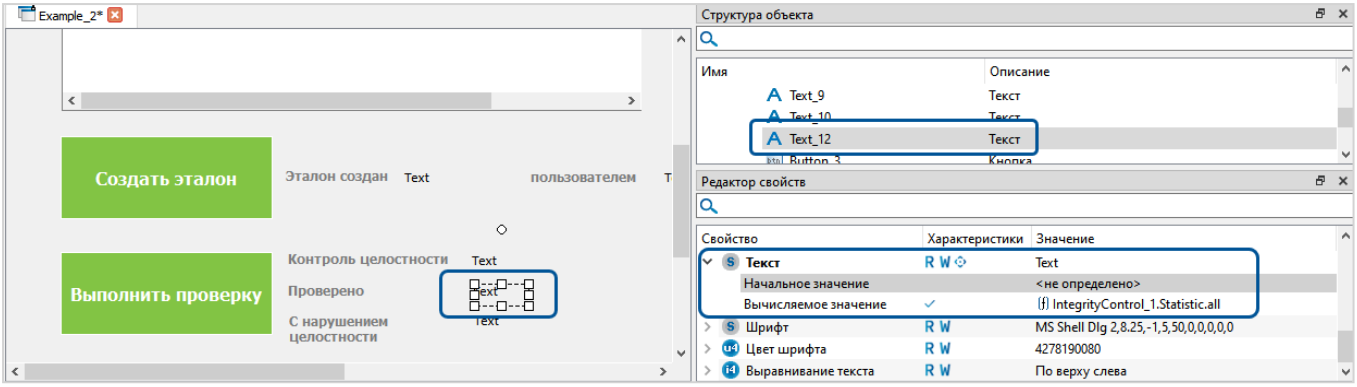
```
IntegrityControl_1.Information.LastCheckData
```



Чтобы вывести общее число проверенных файлов и число файлов с нарушениями нужен объект Statistic (стр. 26) и его поля all и violations.

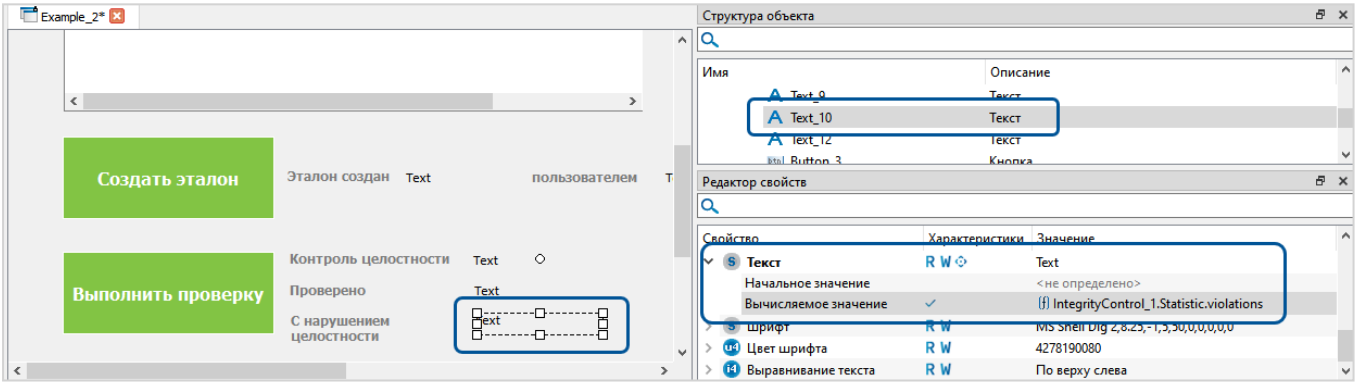
Пусть общее число файлов нужно вывести в поле *Text_12*. Для этого перейдите к свойству **Текст** поля и укажите в качестве вычисляемого значения формулу:

```
IntegrityControl_1.Statistic.all
```



Пусть число файлов с нарушением целостности нужно вывести в поле *Text_10*. Для этого перейдите к свойству **Текст** поля и укажите в качестве вычисляемого значения формулу:

```
IntegrityControl_1.Statistic.violations
```

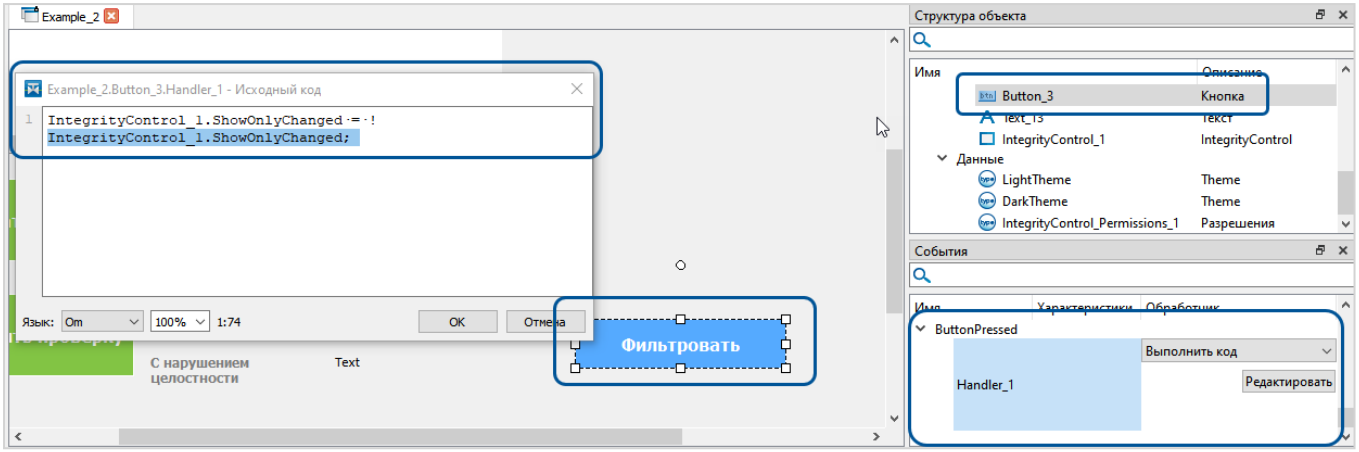


Добавить кнопку фильтрации

Чтобы создать кнопку, по нажатию на которую элементы дерева будут фильтроваться (т.е. будут отображаться только файлы с нарушением целостности), используйте свойство **Показывать только файлы с нарушением целостности** (стр. 24) экземпляра *IntegrityControl_1*.

Создайте кнопку, которая будет включать и отключать фильтрацию. В обработчике события **ButtonPressed** созданной кнопки пропишите:

```
IntegrityControl_1.ShowOnlyChanged = !IntegrityControl_1.ShowOnlyChanged;
```



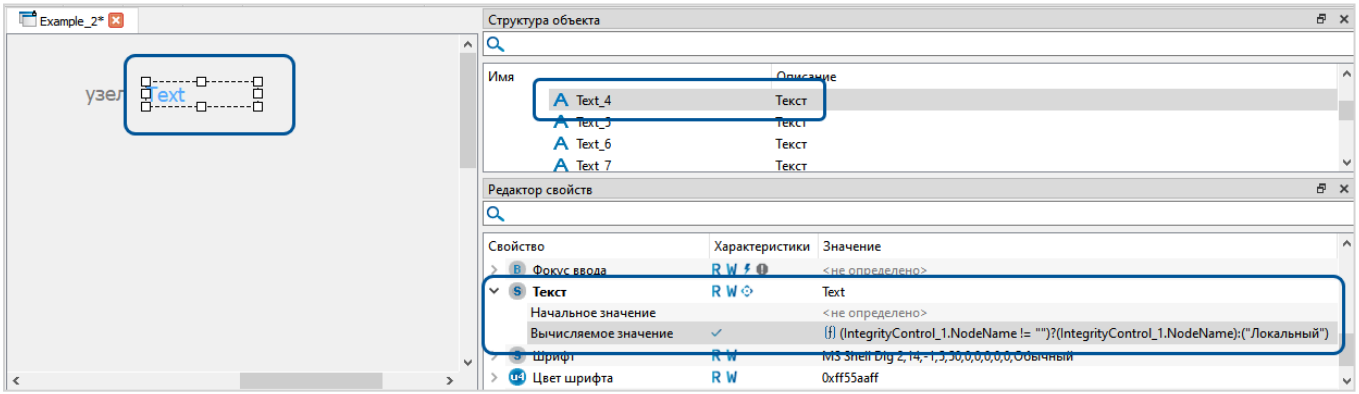
Вывести имя узла

Для вывода на форму имени узла подготовьте текстовое поле. Настройте вывод, используя свойство **Имя узла сети** (стр. 25) экземпляра *IntegrityControl_1*.

Пусть имя узла нужно вывести в поле *Text_4*. Для этого перейдите к свойству **Текст** поля и укажите в качестве вычисляемого значения формулу:

```
(IntegrityControl_1.NodeName != "")?(IntegrityControl_1.NodeName):("Локальный")
```

Данная тернарная операция вернет имя узла, к которому выполнено подключение, либо значение "Локальный".



2.1.1. Свойства

Показывать только файлы с нарушением целостности (ShowOnlyChanged)

Фильтрует список контролируемых файлов и папок еще до отображения их в Alpha.HMI.IntegrityControl.

Тип - уведомляющий bool.

Значения:

- **«true»** - в дереве контролируемых файлов отображаются только элементы с нарушением целостности, изменённой датой и т.д.;

- «`false`» - в дереве контролируемых файлов отображаются все элементы.

Тема оформления (init_Theme)

Ссылка на экземпляр типа **Тема оформления**, хранящий настройки оформления. Тип предоставляется библиотекой Alpha.HMI.CommonLib.

Настройки (init_Settings)

Ссылка на экземпляр типа 2.3. Настройки (стр. 28), позволяющий выбирать папку для экспорта результатов контроля целостности файлов.

Разрешения (init_Permissions)

Ссылка на экземпляр типа 2.2. Разрешения (стр. 27), хранящий настройки доступа пользователей к проверке файлов, созданию эталонов, а также к файловой системе в диалогах открытия и сохранения файлов.

Вызов экранной клавиатуры (init_ScreenKeyboard)

Ссылка на экземпляр типа **Параметры вызова экранной клавиатуры**, хранящий настройки вызова экранной клавиатуры. Тип предоставляется библиотекой Alpha.HMI.CommonLib.

Имя узла сети (NodeName)

Имя узла Alpha.Net, к которому подключается экземпляр типа.

Тип - уведомляющий string.

Имя узла указано в файле конфигурации Alpha.Net. Можно не указывать значение, если узел локальный.

Status

Описание текущего статуса проверки целостности. Только для чтения в режиме рантайма.

Тип - уведомляющий string.

Error

Текст ошибки при выполнении контроля целостности. Только для чтения в режиме рантайма.

Тип - уведомляющий string.

Поля для вывода статистики

Тип `IntegrityControl` предоставляет также поля, с помощью которых можно получить статистику создания эталона, выполнения проверки, а также узнать количество проверенных и поврежденных файлов. Такие поля недоступны в редакторе свойств, но к ним можно обратиться в любом обработчике через объекты Information и Statistic:

- Information

Группа текстовых полей с информацией о выполнении проверки и данных об эталонном значении.

EtalonDateCreated	string	Дата и время создания эталонного файла.
EtalonNameCreator	string	Имя пользователя, создавшего эталон.
LastCheckData	string	Дата и время выполнения последней проверки целостности файлов.
EtalonDiffConfig	bool	Эталонное значение отличается от конфигурации.

› Statistic

Группа полей для статистики в зависимости от статуса файла.

all	int4	Количество всех проверенных файлов.
violations	int4	Количество файлов с нарушением целостности (новые, удаленные и измененные файлы).
warnings	int4	Количество предупреждений.
errors	int4	Количество ошибок.

ПРИМЕР

Вывести в текстовые поля дату/время создания эталона и имя пользователя, создавшего эталон.

```
TextDate = MyIntegrityControl.Information.EtalonDateCreated;TextName =
MyIntegrityControl.Information.EtalonNameCreator;
```

2.1.2. Команды

Check

Выполняет проверку контролируемых файлов на узле, указанном в свойстве **Имя узла сети** (стр. 25).

ПРИМЕР

Запустить контроль целостности на выбранном узле.

```
MyIntegrityControl.Check.Invoke();
```

CreateEtalon

Создает эталон на узле, указанном в свойстве **Имя узла сети** (стр. 25).

ПРИМЕР

Создать эталон на выбранном узле.

```
MyIntegrityControl.CreateEtalon.Invoke();
```

ExportToFile

Сохраняет список контролируемых файлов.

ПРИМЕР

Сохранить список контролируемых файлов.

```
MyIntegrityControl.ExportToFile.Invoke();
```

2.2. Разрешения

Разрешения - тип, позволяющий управлять доступом пользователей к проверке файлов и созданию эталонов, а также к файловой системе в диалогах открытия и сохранения файлов. Если ограничить доступ к файловой системе, пользователь сможет открывать только папки, указанные в экземпляре типа 2.3. Настройки (стр. 28).

Для определения прав доступа текущего пользователя к проверке файлов и созданию эталонов, а также к файловой системе, можно использовать конфигурацию подсистемы безопасности Alpha.Security. Подробнее - в описании свойств типа.

2.2.1. Свойства

Использовать сервер безопасности для определения прав (UseSecurity)

Позволяет включить использование подсистемы безопасности Alpha.Security для определения прав доступа текущего пользователя к проверке файлов и созданию эталонов, а также к файловой системе.

Тип значения - уведомляющий bool.

Значения:

- **«true»** - использовать сервер безопасности для определения прав;
- **«false»** - не использовать сервер безопасности для определения прав.

При включении следует заполнить значение свойства **Имя приложения с правами безопасности (SecurityApplication)**.

Имя приложения с правами безопасности (SecurityApplication)

Хранит имя приложения Alpha.Security, в котором содержатся права **CheckFiles (Проверка файлов)**, **CreateEtalon (Создание эталонных значений)** и **FileSystemAccess (Доступность файловой системы)**,

регулирующие доступность проверки файлов, создания эталонов, а также файловой системы для конкретного пользователя или группы пользователей.

Тип значения - string.

Чтобы использовать сервер безопасности для определения прав текущего пользователя, установите значение **«true»** для свойства Использовать сервер безопасности для определения прав (UseSecurity).

Значение по умолчанию: **«IntegrityControl»**.

Проверка файлов (CheckFiles)

Разрешает выполнять контроль целостности файлов.

Тип значения - уведомляющий bool.

Значения:

- **«true»** - выполнять контроль целостности;
- **«false»** - не выполнять контроль целостности.

Создание эталонных значений (CreateEtalon)

Разрешает создавать эталонные значения.

Тип значения - уведомляющий bool.

Значения:

- **«true»** - создавать эталонные значения;
- **«false»** - не создавать эталонные значения.

Доступность файловой системы (FileSystemAccess)

Позволяет включить или отключить доступ пользователя к файловой системе в диалогах открытия и сохранения файлов.

Тип значения - уведомляющий bool.

Значения:

- **«true»** - включить доступ пользователя к файловой системе;
- **«false»** - отключить доступ пользователя к файловой системе.

Если доступ пользователя к файловой системе отключен, то:

- в файловых диалогах недоступен просмотр дерева папок и создание новых папок;
- экспорт возможен только в папку экспорта, указанную в свойстве ExportFolder типа **Настройки**.

2.3. Настройки

Настройки - вспомогательный тип, применяемый для выбора папки для экспорта результатов контроля целостности файлов. Тип также позволяет сохранять и загружать из JSON-строки путь папки для экспорта.

2.3.1. Свойства

Папка для экспорта (ExportFolder)

Хранит полный путь к папке, куда пользователь может выполнять экспорт файлов. Тип значения - string.

По умолчанию экспорт файлов осуществляется в папку:

- `C:\ProgramData\Automiq\HMI.IntegrityControl\Export` для ОС Windows;
- `/home/<user>/Automiq/HMI.IntegrityControl/Export` для ОС Linux.

2.3.2. Функции

ToJSON

```
string ToJSON()
```

Возвращает JSON-строку, содержащую путь к папке для экспорта. Функция не имеет входных параметров.

FromJSON

```
void FromJSON(string JSON)
```

Получает значения параметров из JSON-строки. Используется для загрузки пути к папке для экспорта в экземпляр компонента. Входной параметр - JSON-строка.

ПРИМЕР

Вызов: MyIntegrityControl_Settings.FromJSON("{\"ExportFolder\":\"C:/Users/test\"}");

В результате значение свойства **Папка для экспорта** экземпляра компонента изменится на C:/Users/test.

2.4. IntegrityControl_App

`IntegrityControl_App` - форма, используемая для открытия полноценного приложения Alpha.HMI.IntegrityControl из меню `Пуск`.

Не имеет собственных свойств и функций.

2.5. IntegrityControl_KTN

`IntegrityControl_KTN` - форма, предназначенная для проверки целостности программного обеспечения верхнего уровня.

На форму выводится результат проверки файлов на целостность и информация о созданном эталоне. В статусной строке отображается информация о количестве проверенных объектов и обнаруженных нарушениях.

Кнопки **Проверка** и **Создать эталон** доступны для пользователей с соответствующими правами.
Форму можно вызвать в обработчике какого-либо события.

2.5.1. Свойства

Настройки (init_Settings)

Ссылка на экземпляр типа 2.3. Настройки (стр. 28), позволяющий указывать папку для экспорта в файл, а также сохранять и загружать из JSON-строки путь папки для экспорта.

Разрешения (init_Permissions)

Ссылка на экземпляр типа 2.2. Разрешения (стр. 27), хранящий настройки доступа пользователей к проверке файлов и созданию эталонов, а также к файловой системе в диалогах открытия и сохранения файлов.

Вызов экранной клавиатуры (init_ScreenKeyboard)

Ссылка на экземпляр типа **Параметры вызова экранной клавиатуры**, хранящий настройки вызова экранной клавиатуры. Тип предоставляется библиотекой Alpha.HMI.CommonLib.

2.6. IntegrityControl_Form

IntegrityControl_Form - форма, используемая для проверки целостности узла, указанного в свойстве **Имя узла домена**. По умолчанию происходит подключение к локальному узлу.

От приложения Alpha.HMI.IntegrityControl отличается тем, что отсутствует кнопка выполнения подключения к узлу, а также кнопки **Конфигурирование контроля целостности** и **Контроль целостности**. Кнопки **Создать эталон** и **Выполнить проверку** доступны для пользователей с соответствующими правами.

Форму можно вызвать в обработчике какого-либо события.

2.6.1. Свойства

Тема оформления (init_Theme)

Ссылка на экземпляр типа **Тема оформления**, хранящий настройки оформления. Тип предоставляется библиотекой Alpha.HMI.CommonLib.

Настройки (init_Settings)

Ссылка на экземпляр типа 2.3. Настройки (стр. 28), позволяющий указывать папку для экспорта в файл, а также сохранять и загружать из JSON-строки путь папки для экспорта.

Разрешения (init_Permissions)

Ссылка на экземпляр типа 2.2. Разрешения (стр. 27), хранящий настройки доступа пользователей к проверке файлов и созданию эталонов, а также к файловой системе в диалогах открытия и сохранения файлов.

Вызов экранной клавиатуры (init_ScreenKeyboard)

Ссылка на экземпляр типа **Параметры вызова экранной клавиатуры**, хранящий настройки вызова экранной клавиатуры. Тип предоставляется библиотекой Alpha.HMI.CommonLib.

Имя узла домена (init_NodeName)

Имя узла домена, к которому выполняется подключение. Значение имени узла указано в файле конфигурации Alpha.Net.Agent. Тип значения - string.

Восстанавливать положение окна (init_RestoreWindow)

Указывает, должно ли окно открываться в указанном месте, или в том же, где было открыто в последний раз.

Тип значения - bool.

Значения:

- **«true»** - положение окна запоминается. Если переместить окно в определенную область, закрыть его, а затем вновь открыть, то окно откроется в том же месте, где было закрыто. Настройки положения окна сохранятся в файл `window.json` в папке:
  - `C:\Users\<user>\Automiq\HMI.IntegrityControl` для ОС Windows;
  - `/home/<user>/Automiq/HMI.IntegrityControl` для ОС Linux.

  Файл используется для работы формы, менять его содержимое не рекомендуется.
- **«false»** - положение окна не запоминается. Если переместить окно в определенную область, закрыть его, а затем вновь открыть, то окно откроется в координатах, указанных в свойствах X окна и Y окна.

2.7. IntegrityControl_Tests

IntegrityControl_Tests - форма, предназначенная для ознакомления с примерами использования типа и форм расширения в проекте Alpha.HMI.

Чтобы ознакомиться с этими примерами, откройте проект расширения в режиме редактирования, а затем перейдите к форме.

Тип, используемый в проекте Alpha.HMI, в режиме рантайма создает файл `tests.json` в папке:

- `C:\Users\<user>\Automiq\HMI.IntegrityControl` для ОС Windows;
- `/home/<user>/Automiq/HMI.IntegrityControl` для ОС Linux.

Файл используется для работы формы, менять его содержимое не рекомендуется. Чтобы ознакомиться с тем, как создавать подобные файлы для восстановления внешнего вида и положения окна формы, откройте проект расширения в режиме редактирования и ознакомьтесь с обработчиками событий формы на вкладке **События**.

История изменений

2.5

Новые возможности

- В окне конфигурирования контроля целостности теперь можно сохранить конфигурацию в файл `*.json` или загрузить конфигурацию из такого файла. Для этого на панель инструментов добавлены соответствующие кнопки.
- Рядом с кнопкой подключения к узлу добавлена кнопка, открывающая список недавних подключений - для удобства повторного подключения.

Исправления

- Исправлена ошибка получения значений прав пользователей от подсистемы безопасности. Из-за этой ошибки пользователи, имеющие права просмотр дерева файлов, создание эталона и запуск проверок, фактически не имели таких возможностей.
- Устранена причина, по которой при редактировании настроек контроля целостности было невозможно добавить папку в список контролируемых объектов.
- Исправлено название кнопки фильтрации на панели инструментов окна контроля целостности.
- Исправлены опечатки в информационных сообщениях.

Изменения документации

Редакция 1

В руководстве пользователя:

- Актуализировано описание настроек безопасности:
  - в разделах 1.6. Контроль целостности (стр. 10), 1.6.1. Проверка целостности (стр. 11) и 1.6.2. Создание эталона (стр. 13) описано, как меняется доступность кнопок панели инструментов в зависимости от прав текущего пользователя;
  - в разделе 1.8. Настройки безопасности для разграничения возможностей пользователей (стр. 16) актуализирована информация об использовании настроек безопасности в самостоятельном приложении Alpha.HMI.IntegrityControl и в разрабатываемых с помощью решения проектах приложений.
- В раздел 1.7. Конфигурирование контроля целостности (стр. 14) добавлен подраздел, описывающий возможность сохранения и загрузки конфигурации из файла.
- Актуализированы скриншоты.

Также актуализировано справочное руководство.

2.4

2.4.1

Новая возможность

Добавлена возможность изменения настроек контроля целостности на локальном и удаленных узлах. Для этого на панель инструментов приложения добавлены кнопки:

- **Конфигурирование контроля целостности**, нажав которую можно перейти в режим изменения настроек контроля целостности на узле, к которому выполнено подключение;
- **Контроль целостности**, нажав которую можно перейти в режим просмотра текущего состояния контроля целостности и ручного запуска проверок.

Улучшение

В таблице просмотра результатов проверок целостности раскрытые строки больше не сворачиваются после ручного запуска проверки целостности.

Изменения документации

Редакция 1

В руководстве пользователя:

- Актуализированы разделы 1.3. Запуск в виде самостоятельного приложения (стр. 6), 1.5. Подключение к узлу (стр. 10), 1.6.1. Проверка целостности (стр. 11), 1.6.2. Создание эталона (стр. 13) и 1.6.3. Экспорт результатов проверки целостности в файл (стр. 13).
- Создан новый раздел 1.6. Контроль целостности (стр. 10), в который перенесены разделы 1.6.1. Проверка целостности (стр. 11), 1.6.2. Создание эталона (стр. 13) и 1.6.3. Экспорт результатов проверки целостности в файл (стр. 13).
- Создан новый раздел 1.7. Конфигурирование контроля целостности (стр. 14), описывающий, как с помощью приложения изменить настройки контроля целостности.
- Удален более неактуальный раздел "Дополнительная настройка окружения" и раздел "Фильтрация списка файлов" - информация из него перенесена в 1.6.1. Проверка целостности (стр. 11).

Также актуализировано справочное руководство.

2.3

Улучшения

- Теперь если конфигурация эталона отличается от текущей конфигурации контроля целостности, в правой части окна приложения отображается уведомление об этом.
- По завершении автоматической проверки контроля целостности список контролируемых файлов теперь обновляется автоматически.
- Прогресс сохранения результатов проверки целостности в файл теперь отображается в строке состояния.

Изменение

Изменены расположение и содержимое конфигурационных файлов приложения и компонентов расширения:

- Настройки приложения хранятся в файлах `app_settings.json` и `settings.json`, расположенных в папке:
  - `C:\ProgramData\Automiq\HMI.IntegrityControl` для ОС Windows;
  - `/home/<user>/Automiq/HMI.IntegrityControl` для ОС Linux.

- Настройки сессии приложения хранятся в файле `app_session.json`. Настройки компонентов и сессий компонентов хранятся в файлах `layout.json`, `session.json`, `tests.json` и `window.json`. Все перечисленные файлы расположены в папке:
  - `C:\Users\<user>\Automiq\HMI.IntegrityControl` для ОС Windows;
  - `/home/<user>/Automiq/HMI.IntegrityControl` для ОС Linux.

Исправление

Исправлена ошибка фильтрации списка файлов в экземпляре типа **`IntegrityControl`**, у которого для свойства **Показывать только файлы с нарушением целостности (ShowOnlyChanged)** установлено значение **«`true`»**.

Изменения документации

Редакция 1

Изменена структура руководства пользователя:

- Бывший раздел "Запуск приложения" разделен на разделы 1.3. Запуск в виде самостоятельного приложения (стр. 6) и 1.4. Встраивание в проект (стр. 8).
- Удален раздел "Возможности приложения", его подразделы, описывающие взаимодействие с приложением, стали самостоятельными разделами верхнего уровня оглавления.

Также в руководстве пользователя:

- Бывший раздел "Установка и удаление" переименован в 1.2. Подготовка к работе (стр. 5) и расширен: в него добавлена информация о требованиях к окружению.
- Обновлена информация о настройках контроля целостности.
- Бывший раздел "Экспорт в файл" переименован в 1.6.3. Экспорт результатов проверки целостности в файл (стр. 13) и расширен: в нем описана возможность изменения папки экспорта, открываемой при сохранении файла по умолчанию.
- Бывший раздел "Ограничение возможностей пользователей при выполнении контроля целостности файлов" переименован в 1.8. Настройки безопасности для разграничения возможностей пользователей (стр. 16) и расширен: настройка разграничения возможностей пользователей описана как для приложения, разработанного в Alpha.HMI с использованием компонентов, предоставляемых расширением Alpha.HMI.IntegrityControl, так и для самого приложения Alpha.HMI.IntegrityControl.

2.2

Улучшение

Из результатов проверки целостности исключен столбец **Дополнительная информация**. Описания нарушений целостности теперь отображаются в виде всплывающих подсказок.

Исправленные ошибки

- Экспорт результатов проверки целостности в файл теперь выполняется корректно.
- Строка папки, содержащей файл с нарушением целостности, теперь окрашивается в тот же цвет, что и файл с нарушением.

Изменения документации

Редакция 1

Документ разделен на руководство пользователя и справочное руководство.

В справочном руководстве:

- Описаны все типы и формы, предоставляемые внешним модулем Alpha.HMI.IntegrityControl, приведены их свойства и функции.
- Приведен пример использования типа **IntegrityControl** в проекте.

В руководстве пользователя:

- Раздел "Встраивание в проект" со всеми его подразделами удален. Часть информации из раздела теперь находится в новом разделе 1.4. Встраивание в проект (стр. 8), часть - в справочном руководстве.
- Добавлен раздел, описывающий 1.8. Настройки безопасности для разграничения возможностей пользователей (стр. 16).
- Обновлены скриншоты.

2.1

Новые возможности

- Папка для экспорта теперь указывается через поле ExportFolder отдельного типа **IntegrityControl_Settings**. Такой способ задания папки экспорта заменил существующий ранее - через свойство ExportFolder компонента **IntegrityControl**. Если же для указания папки экспорта не использовать тип **IntegrityControl_Settings**, данные будут экспортироваться в папку:
  - ОС Windows: `C:\ProgramData\Automiq\HMI.IntegrityControl\Export`
  - ОС Linux: `/home/<user>/Automiq/HMI.IntegrityControl/Export`
- Реализована возможность ограничения доступа пользователя к файловой системе. Для ограничения доступа укажите значение «**false**» в свойстве FileSystemAccess у объекта разрешений Permissions. При ограниченном доступе для пользователя:
  - В файловых диалогах будет недоступен просмотр дерева папок и создание новых папок.
  - Экспорт будет возможен только в папку экспорта, указанную в свойстве ExportFolder типа **IntegrityControl_Settings**.

2.1.1

Улучшение

Панель инструментов теперь оформляется в соответствии с выбранной темой.

2.1.2

Улучшение

Ускорена загрузка результатов контроля целостности файлов.

Изменения документации

Редакция 1

Актуализировано описание API.

Редакция 2

Внутренние изменения. Содержимое документа не изменилось.

Редакция 3

Внутренние изменения. Содержимое документа не изменилось.

2.0

Новая возможность

Для ограничения прав пользователей вместо свойств Разрешить выполнять проверку (AllowCheck) и Разрешить создание эталонных значений (AllowCreateEtalon) типа **IntegrityControl** теперь используйте новый тип **Разрешения (IntegrityControl_Permissions)**. Новый тип содержит свойства Проверка файлов (CheckFiles) и Создание эталонных значений (CreateEtalon).

> ОБРАТИТЕ ВНИМАНИЕ
> При переходе на новую версию Alpha.HMI.IntegrityControl для использования нового типа **Разрешения** вам потребуется доработать проект.

В экземпляре типа **IntegrityControl** в параметре инициализации Разрешения сошлитесь на нужный экземпляр типа **Разрешения**. Если вы используете **IntegrityControl_Form**, можете создать экземпляр разрешений прямо в обработчике открытия формы.

Если вы желаете использовать систему безопасности для ограничения прав пользователя:

1. Создайте в подсистеме безопасности приложение для Alpha.HMI.IntegrityControl с правами:
   - CheckFiles (Проверка файлов);
   - CreateEtalon (Создание эталонных значений).
2. Для экземпляра типа **Разрешения**:
   - Включите использование подсистемы безопасности (в свойстве Использовать сервер безопасности для определения прав (UseSecurity)).
   - Укажите имя приложения, которое вы используете для Alpha.HMI.IntegrityControl в подсистеме безопасности (в свойстве Имя приложения с правами безопасности (SecurityApplication).

Исправленная ошибка

Исправлена ошибка, из-за которой сообщения о состоянии файла не соответствовали сообщениям аудита.

2.0.1

Внутренние изменения. Функциональность компонента не менялась.

2.0.2

- Добавлена возможность вызова экранной клавиатуры в диалоговых окнах:
  - экспорта в файл;
  - авторизации.

Для отображения кнопки вызова клавиатуры нужно включить эту функцию в параметре init_ScreenKeyboard (по умолчанию кнопка появляется только в диалоге авторизации).

- Для Alpha.HMI.IntegrityControl, работающего в виде отдельного приложения, реализована возможность загрузки настроек из файла `app_settings.json`.

Загрузить можно:

- Параметры работы с виртуальной клавиатурой. Чтобы включить клавиатуру, используйте параметр Enabled элемента **`ScreenKeyboard`**.
- Разрешение использования сервера безопасности для ограничения прав пользователя. Чтобы разрешить использовать сервер безопасности, используйте параметр UseSecurity в элементе **`Permissions`**.

Файл хранится по пути:

- ОС Windows: `C:\ProgramData\Automiq\HMI.IntegrityControl`
- ОС Linux: `/home/<user>/Automiq/HMI.IntegrityControl`

Изменение только для формы IntegrityControl_KTN

Ошибки и предупреждения, возникшие после проверки целостности, теперь считаются нарушениями. В статусной строке сначала выводится общее количество нарушений, а затем точное количество ошибок и предупреждений.

Улучшение

Теперь текстовая подсказка к ошибке всегда принудительно отображается рядом с полем ввода, содержащим ошибку.

2.0.3

Исправленная ошибка

Исправлена ошибка, из-за которой при удалении файла родительская папка окрашивалась в желтый цвет, вместо красного.

Изменение

В Alpha.Security 1.7.6 добавлена возможность классифицировать серьёзность нарушений каждого типа. На основе этого в Alpha.HMI.IntegrityControl изменились поля в группе Statistic (используемой для вывода информации через ссылку pUpdateOutputViewData).

Что изменилось:

- Поле valid теперь называется violations и отражает количество нарушений вместо количества файлов без нарушений целостности.
- Поле warrning теперь называется warnings и отражает количество предупреждений вместо количества файлов с измененной датой, но корректной контрольной суммой.
- Поле alarm теперь называется errors и отражает количество ошибок вместо количества файлов с нарушением целостности (новые, удаленные и измененные файлы).

Изменения документации

Редакция 1

- Добавлено описание (стр. 13) возможности выгрузки информации о контролируемых файлах в таблицу.
- В описание настройки окружения добавлены названия конфигурационных файлов Alpha.Security.
- Описана возможность (стр. 10) выбора узла для подключения.
- Описано как запускать приложение в ОС Linux (стр. 6).
- Ошибочно указанная служба Alpha.Net заменена на Alpha.Security (стр. 11).

Редакция 2

- Обновлены скриншоты в разделе с описанием встраивания Alpha.HMI.IntegrityControl в проект (стр. 8).
- В нескольких разделах имя типа изменено с **`IC_Tree`** на **`IntegrityControl`**.

Редакция 3

Внутренние изменения. Содержимое документа не изменилось.

1.3

Новые возможности

- Переработано API компонента, используемого для встраивания в проекты пользователя. Ранее этот компонент назывался **`IC_Tree`**, теперь называется **`IntegrityControl`**. API компонента стало проще и понятней для использования.

  Состав нового API:

  - Свойства:
    - NodeName для задания имени узла сети.
    - AllowCheck, разрешающее выполнять проверку.
    - AllowCreateEtalon, разрешающее создавать эталонные значения.
    - ShowOnlyChanged для включения фильтрации элементов дерева и отображения только файлов с нарушением целостности.
    - Status, содержащее описание текущей операции.
    - Error, содержащее описание ошибки, которая произошла при выполнении последней операции.
  - Команды:
    - Check для проверки контролируемых файлов на узле, указанном в свойстве NodeName.
    - CreateEtalon для создания эталона на узле, указанном в свойстве NodeName.
  - Добавлены объекты **`Information`** и **`Statistic`**, с помощью свойств которых можно получить дополнительную информацию о целостности системы.

- Добавлена форма **`IntegrityControl_Form`** для открытия с определенными параметрами. Такая форма может использоваться, например, для контроля определенного узла.
  - Свойства формы:
    - AllowCheck, разрешающее выполнять проверку.
    - AllowCreateEtalon, разрешающее создавать эталонные значения.
  - Параметры инициализации:
    - init_NodeName для задания имени узла сети.
    - init_RestoreWindow для разрешения восстановления положения окна.
- В строке статуса теперь выводится текст ошибок подключения к узлу.
- Реализовано сохранение ширины столбцов в дереве файлов.

Исправленные ошибки

Исправлена проблема, из-за которой не работали свойства AllowCheck (Разрешить выполнять проверку) и AllowCreateEtalon (Разрешить создание эталонных значений).

1.3.1

Новая возможность

Добавлена возможность экспорта данных из дерева в таблицу формата `*.csv` и `*.xlsx`.

Исправленная ошибка

Папка, не содержащая контролируемые файлы, ошибочно окрашивалась в желтый цвет.

Изменения документации

Редакция 1

Внутренние изменения. Содержимое документа не изменилось.

1.2

1.2.1

Внутренние изменения. Функциональность компонента не менялась.

1.1

Новые возможности

Реализован запуск Alpha.HMI.IntegrityControl в виде отдельного приложения по ярлыку из Пуск.

1.1.1

Новые возможности

В окне подключения к узлу теперь можно выбрать тип узла: локальный или удаленный.

Изменения документации

Редакция 2

Внутренние изменения. Содержимое документа не изменилось.

Атомик Софт

Программный комплекс Альфа платформа

Alpha.HMI.Security 2.0

Руководство пользователя

Редакция	Соответствует версии ПО
11	2.0.15

© АО «Атомик Софт», 2015-2026. Все права защищены.

Авторские права на данный документ принадлежат АО «Атомик Софт». Копирование, перепечатка и публикация любой части или всего документа не допускается без письменного разрешения правообладателя.

Содержание

1. Руководство пользователя 5
1.1. О продукте 5
1.2. Подготовка к работе 6
1.2.1. Требования к окружению 6
1.2.2. Установка, удаление или восстановление 6
1.2.2.1. ОС Windows 6
1.2.2.2. ОС Linux 7
1.3. Управление доступом в проектах 8
1.4. Собственная форма входа 17
1.5. Собственный конфигуратор подсистемы безопасности 22
1.6. Контроль целостности файлов и папок 35
1.7. Конфигурирование агента безопасности 40
1.8. Логирование сообщений безопасности 46

2. Справочное руководство 50
2.1. Контекст безопасности (SecurityContext) 50
2.1.1. Свойства 50
2.1.2. Функции 56
2.1.3. События 63
2.2. Список пользователей (UserList) 65
2.2.1. Свойства 65
2.2.2. Функции 66
2.2.3. События 68
2.3. Настройка безопасности: Контроль целостности (SecurityIntegrityControl) 68
2.3.1. Свойства 79
2.3.2. Функции 81
2.3.3. События 86
2.4. Строковый элемент безопасности (StringTokenProxy) 92
2.4.1. Свойства 92
2.4.2. Функции 93
2.4.3. События 94
2.5. Булевский элемент безопасности (BoolTokenProxy) 94
2.5.1. Свойства 94
2.5.2. События 96
2.6. Настройка безопасности: Менеджер (SecurityManager) 96
2.6.1. Свойства 96
2.6.2. Функции 97
2.6.3. События 103
2.7. Настройка безопасности: Приложение (SecurityManagerApplication) 110
2.7.1. Свойства 111
2.7.2. Функции 112
2.7.3. События 119
2.8. Настройка безопасности: Пользователь (SecurityManagerUser) 120
2.8.1. Свойства 120
2.8.2. Функции 122
2.8.3. События 130
2.9. Настройка безопасности: Группа (SecurityManagerGroup) 131
2.9.1. Свойства 131
2.9.2. Функции 133
2.9.3. События 139
2.10. Мастер конфигурирования Security (Configurator) 140
2.10.1. Свойства 141
2.10.2. Функции 145

2.10.3. События ... 157
2.11. Информация лицензирования: Получение (LicenseInfo) ... 158
2.11.1. Свойства ... 170
2.11.2. Функции ... 171
2.11.3. События ... 172

История изменений ... 174
2.0 ... 174
2.0.1 ... 174
2.0.2 ... 174
2.0.3 ... 174
2.0.4 ... 175
2.0.5 ... 175
2.0.7 ... 175
2.0.8 ... 175
2.0.9 ... 176
2.0.10 ... 176
2.0.12 ... 177
2.0.13 ... 178
2.0.15 ... 178
Изменения документации ... 178
Редакция 1 ... 178
Редакция 2 ... 179
Редакция 3 ... 179
Редакция 4 ... 179
Редакция 5 ... 179
Редакция 6 ... 179
Редакция 7 ... 179
Редакция 8 ... 180
Редакция 9 ... 180
Редакция 10 ... 180
Редакция 11 ... 180
1.2 ... 180
1.2.3 ... 180
Изменения документации ... 181
Редакция 1 ... 181
Редакция 2 ... 181
Редакция 3 ... 181
1.1 ... 182
1.1.1 ... 182
Изменения документации ... 182
Редакция 2 ... 182
1.0 ... 182
Изменения документации ... 182
Редакция 2 ... 182

1. Руководство пользователя

1.1. О продукте

Alpha.HMI.Security - это расширение среды разработки и исполнения Alpha.HMI.

Расширение представляет собой библиотеку компонентов, позволяющих взаимодействовать с подсистемой безопасности Alpha.Security из проектов Alpha.HMI.

Под взаимодействием подразумевается:

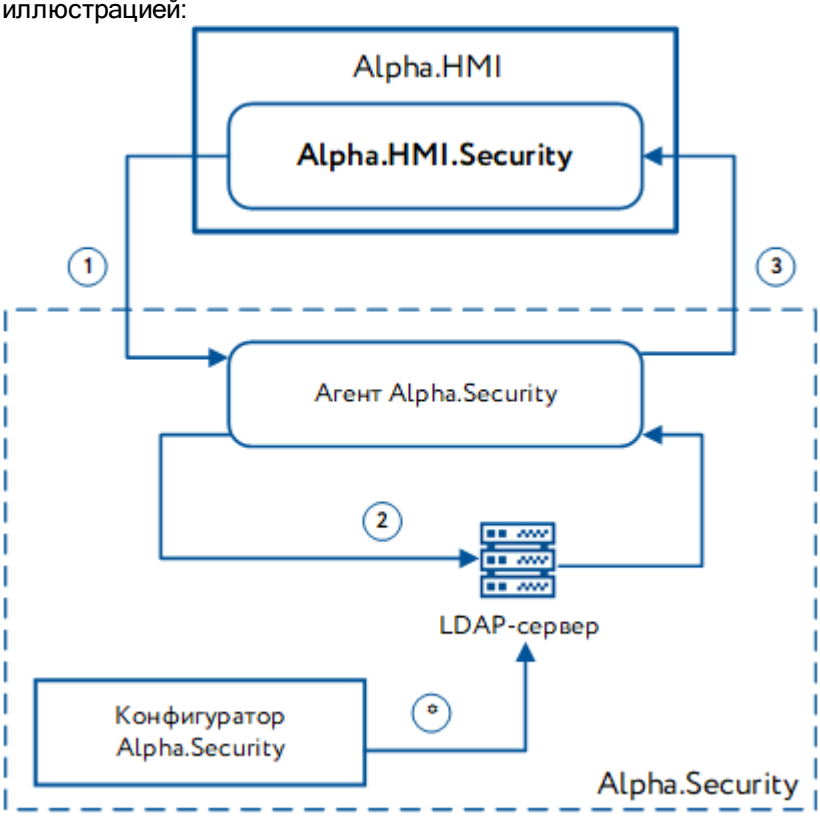
- регистрация пользователя в подсистеме безопасности (вход) с использованием учетных данных;
- просмотр текущей конфигурации подсистемы безопасности Alpha.Security;
- изменение текущей конфигурации подсистемы безопасности Alpha.Security;
- получение информации о статусе операций пользователя;
- выход из подсистемы безопасности.

ПРИМЕЧАНИЕ

Компоненты расширения Alpha.HMI.Security обращаются к подсистеме безопасности Alpha.Security с помощью API: свойств, функций и событий. Подробнее - в Справочном руководстве (стр. 50).

Место Alpha.HMI.Security во взаимодействии Alpha.HMI и Alpha.Security

Взаимодействие Alpha.HMI и Alpha.Security с помощью Alpha.HMI.Security можно описать следующей иллюстрацией:



1. Компонент Alpha.HMI.Security отправляет запрос на просмотр или изменение конфигурации подсистемы безопасности в Агент Alpha.Security;
2. Агент Alpha.Security находит нужную информацию на LDAP-сервере или записывает новую конфигурацию на LDAP-сервер;
3. Агент Alpha.Security предоставляет результат операции в проект Alpha.HMI с помощью компонентов Alpha.HMI.Security.

ПРИМЕЧАНИЕ

Информация на LDAP-сервере также может быть изменена с помощью Конфигуратор Alpha.Security.

1.2. Подготовка к работе

1.2.1. Требования к окружению

Для работы расширения Alpha.HMI.Security должны быть установлены:

- среда разработки и исполнения Alpha.HMI;
- подсистема безопасности Alpha.Security;
- компонент Alpha.Domain - для обеспечения связи между компонентами расширения Alpha.HMI.Security и подсистемой безопасности Alpha.Security.

Если работаете с веб-версией проекта автоматизации, установите дополнительно Alpha.HMI.WebViewer - для использования проектов Alpha.HMI в веб-интерфейсе.

1.2.2. Установка, удаление или восстановление

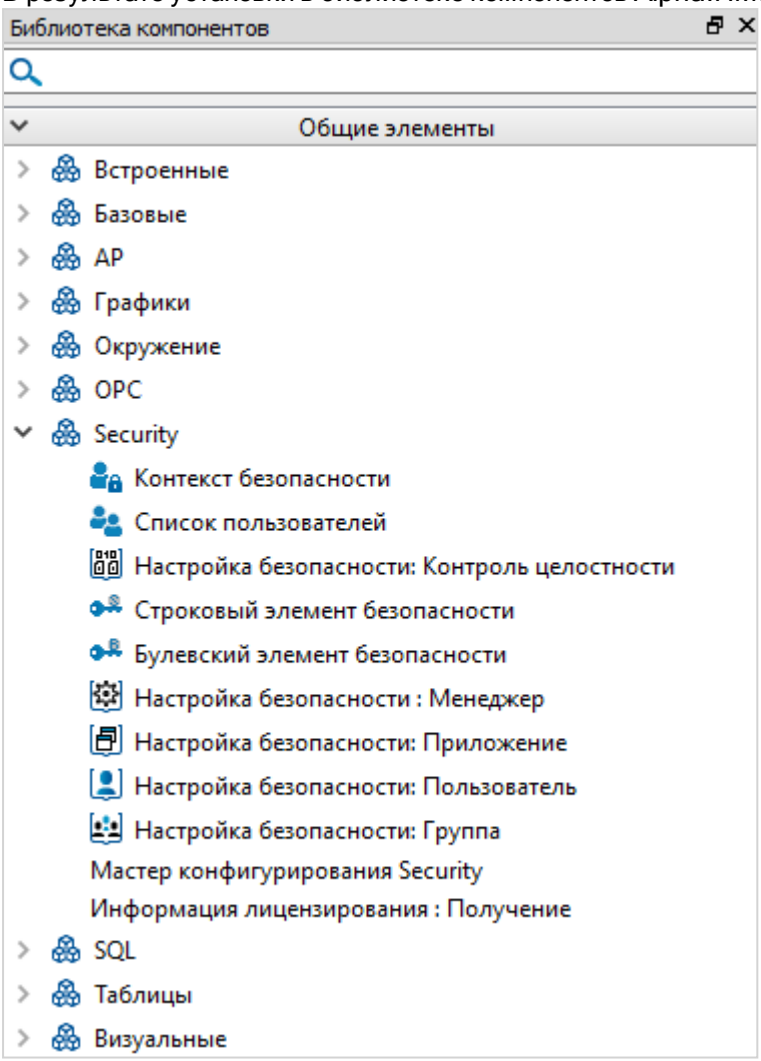
1.2.2.1. ОС Windows

Для установки, удаления или восстановления Alpha.HMI.Security запустите установочный файл `alpha.hmi.security-<lng>-<arch>-<version>.msi` и следуйте инструкциям мастера. Если устанавливаете веб-версию, запустите файл `alpha.hmi.security.webviewer-<lng>-<arch>-<version>.msi`.

ПРИМЕЧАНИЕ

В названии файла `<lng>` - это язык расширения, `<arch>` - целевая процессорная архитектура, а `<version>` - номер версии расширения.

В результате установки в библиотеке компонентов Alpha.HMI должен появиться юнит **Security**.



1.2.2.2. ОС Linux

ОБРАТИТЕ ВНИМАНИЕ

Команды на установку и удаление нужно запускать с правами суперпользователя.

Для установки Alpha.HMI.Security вызовите пакет с командой на установку в зависимости от используемого пакетного менеджера:

- rpm-пакет:

```
sudo rpm -i alpha.hmi.security-<lng>-<version>.rpm
```

или, если используете веб-версию:

```
sudo rpm -i alpha.hmi.security.webviewer-<lng>-<version>.rpm
```

- deb-пакет:

```
sudo dpkg -i alpha.hmi.security-<lng>-<version>.deb
```

или, если используете веб-версию:

```
sudo dpkg -i alpha.hmi.security.webviewer-<lng>-<version>.deb
```

ПРИМЕЧАНИЕ

В названии пакета `<lng>` - это язык компонента, а `<version>` - номер версии расширения.

В результате установки в библиотеке компонентов Alpha.HMI должен появиться юнит **`Security`**.
Для удаления Alpha.HMI.Security вызовите команду на удаление приложения в зависимости от используемого пакетного менеджера:

- rpm-пакет:

```
sudo rpm -e alpha.hmi.security-desktop
```

или, если используете веб-версию:

```
sudo rpm -e alpha.hmi.security-webviewer
```

- deb-пакет:

```
sudo dpkg -r alpha.hmi.security-desktop
```

или, если используете веб-версию:

```
sudo dpkg -r alpha.hmi.security-webviewer
```

1.3. Управление доступом в проектах

ОБРАТИТЕ ВНИМАНИЕ

Прежде чем изучить этот и следующие разделы рекомендуется ознакомиться с:

- основами разработки проектов в Дизайнер Alpha.HMI;
- назначением и принципом работы подсистемы безопасности Alpha.Security;
- принципами конфигурирования подсистемы безопасности Alpha.Security.

В разделе продемонстрировано применение компонентов Alpha.HMI.Security для разграничения доступа пользователей в проектах Alpha.HMI. Для этого описан проект-пример, имитирующий АРМ системы управления насосом и резервуаром. Доступ к управлению имитируемой системой регулируется подсистемой безопасности Alpha.Security и зависит от прав текущего пользователя.

Для работы с проектом-примером должны быть установлены:

- Дизайнер Alpha.HMI - для просмотра, запуска и модификации проекта;
- Alpha.Security - для создания учетных записей пользователей и прав;
- Alpha.Domain - для обеспечения взаимодействия компонентов подсистемы безопасности

Alpha.Security;

- Alpha.HMI.Security - для взаимодействия Alpha.HMI и Alpha.Security.

Также для работы необходимы:

- файлы проекта-примера `AccessControl.hmi`;
- файл конфигурации Alpha.Security `Configuration.Example.bak`.

Скачайте архив с указанными файлами. Откройте проект-пример в Дизайнер Alpha.HMI. Восстановите резервную копию из файла конфигурации Alpha.Security.

> **ВАЖНО**
> Если у вас имеется собственная конфигурация Alpha.Security, сохраните ее резервную копию, прежде чем приступить к работе с проектом-примером.

Подключитесь к каталогу OpenLDAP, в который вы восстановили резервную копию проекта-примера, с помощью SecurityConfigurator, поставляемого в составе дистрибутива Alpha.Security. Администратор - **«manager»**, пароль - **«secret»**. Убедитесь, что в конфигурации каталога появились:

- учетные записи пользователей **«Иванов»** и **«Петров»**, состоящих в группах **«Диспетчеры»** и **«Операторы»** соответственно;
- приложение **«Управление состоянием оборудования»**, содержащее:
  - два логических права: **«Управление насосом»** и **«Управление резервуаром»**;
  - одно строковое право **«Доступ к оборудованию»**.

Для того, чтобы агент безопасности предоставлял информацию в проект Alpha.HMI из нужного каталога OpenLDAP, укажите название этого каталога в конфигурационном файле агента безопасности - `alpha.security.agent.xml`, расположенном в `C:\Program Files\Automiq\Alpha.Security` (для ОС Windows) или `/opt/Automiq/Alpha.Security` (для ОС Linux).

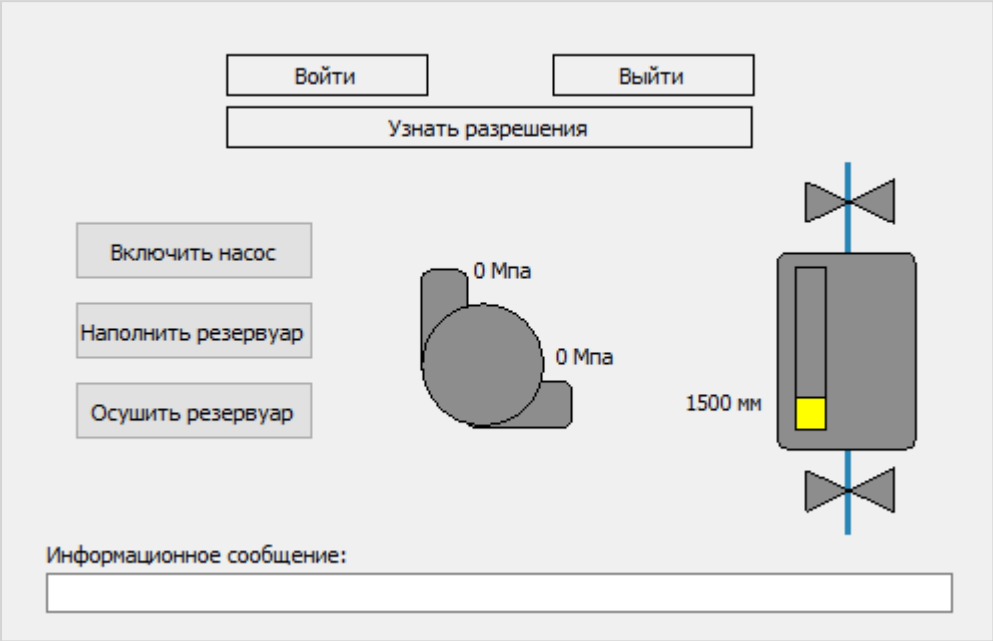
Назначение проекта-примера

Проект-пример имитирует АРМ, которое позволяет:

- запустить проект под пользователем по умолчанию и воспользоваться общедоступными функциями АРМ без ввода учетных данных;
- войти с личными учетными данными (логин/пароль);
- воспользоваться функциями, доступными только одному из пользователей:
  - пользователю, состоящему в группе **«Диспетчеры»**, доступно управление резервуаром, но недоступно управление состоянием насоса;
  - пользователю, состоящему в группе **«Операторы»**, доступно управление состоянием насоса, но недоступно управление резервуаром;
- выйти, не потеряв возможности воспользоваться общедоступными функциями.

Состав проекта-примера

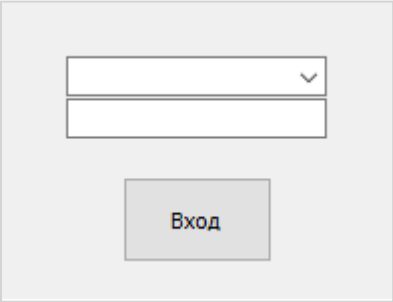
Главная экранная форма проекта - *MainForm*:



На главную форму добавлены:

- *SecurityContext* - элемент типа **Контекст безопасности** (стр. 50), обеспечивающий взаимодействие с Агент Alpha.Security;
- *BoolTokenProxy_ControlTank* - элемент типа **Булевский элемент безопасности** (стр. 94), привязанный к праву, регулирующему доступ к управлению резервуаром;
- *BoolTokenProxy_ControlPump* - элемент типа **Булевский элемент безопасности**, привязанный к праву, регулирующему доступ к управлению насосом;
- *StringTokenProxy* - элемент типа **Строковый элемент безопасности** (стр. 92), привязанный к праву, содержащему текстовые разрешения и запреты обоих пользователей.

Вторая форма проекта - форма входа *AuthorizationForm*:



На форму входа *AuthorizationForm* добавлены:

- *UserList* - элемент типа **Список пользователей** (стр. 65), позволяющий работать со списком учетных записей из подсистемы безопасности Alpha.Security;
- *SecurityContext* - элемент типа **Контекст безопасности**, обеспечивающий взаимодействие с Alpha.Security.

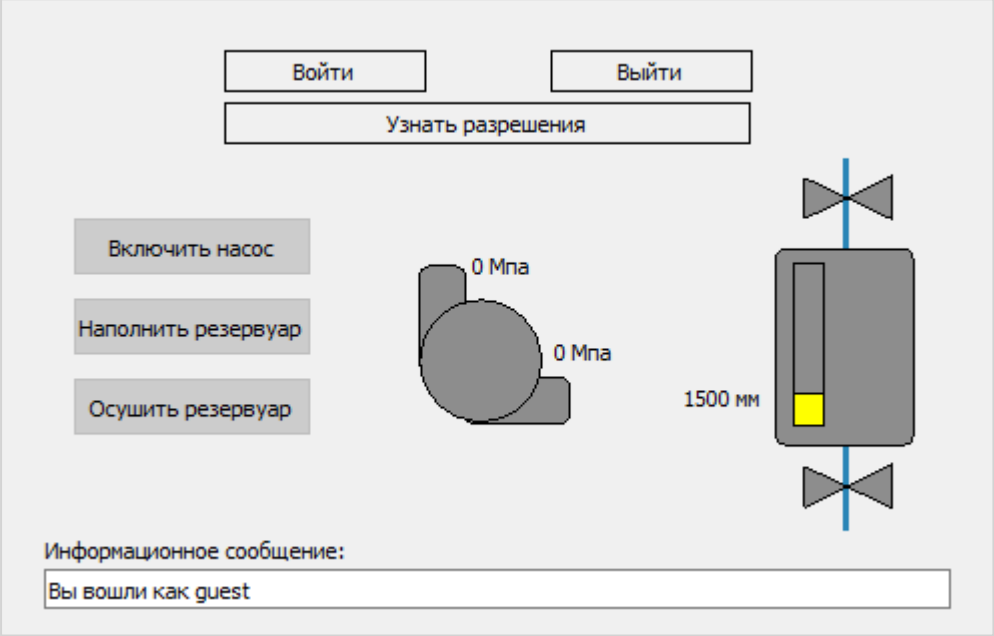
ПРИМЕЧАНИЕ

Здесь и в других проектах-примерах на каждой форме размещен свой элемент типа **Контекст безопасности**. Такое решение не является обязательным. Можно применить любое подходящее для проекта решение, например:

- использовать ссылки на элемент типа **Контекст безопасности**, расположенный на одной из форм проекта;
- использовать один глобальный элемент типа **Контекст безопасности**.

Использование проекта с правами пользователя по умолчанию

Запустите проект-пример в рантайм. При запуске откроется главное окно:



При запуске главного окна элемент *SecurityContext* обратился к Агент Alpha.Security для получения информации о текущем пользователе и его правах. В систему никто не вошел, поэтому Агент Alpha.Security предоставил информацию о пользователе, чья учетная запись используется по умолчанию - **«guest»**.

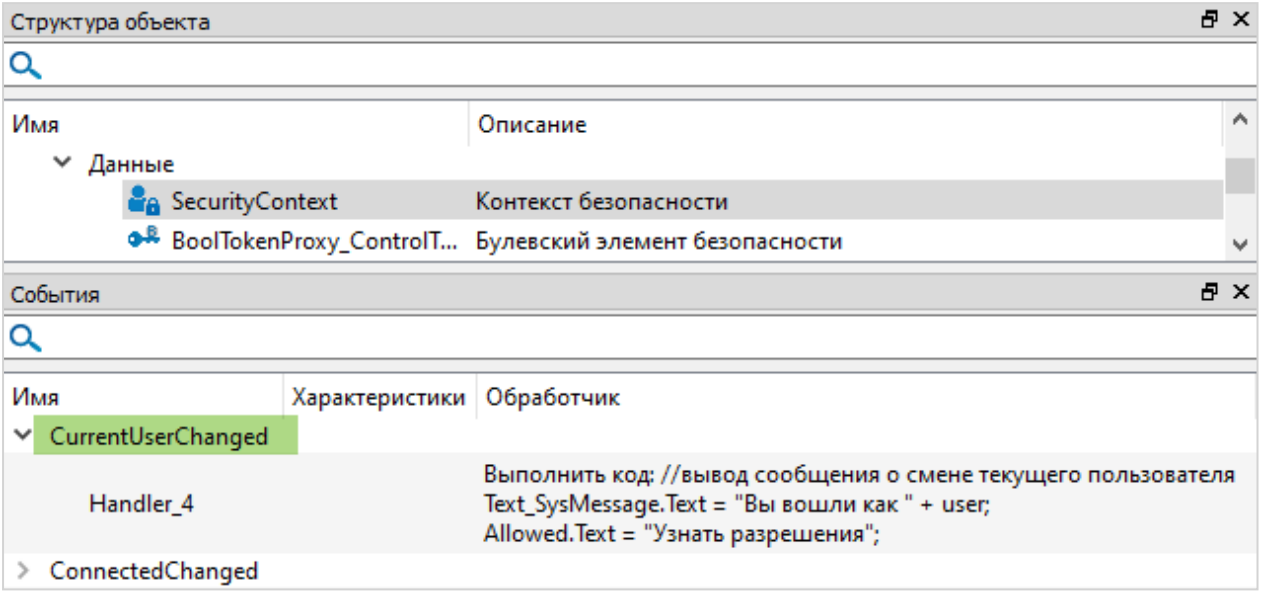
ПРИМЕЧАНИЕ

В качестве пользователя по умолчанию можно указать любого из имеющихся в подсистеме безопасности Alpha.Security.

Пользователь **«guest»** не имеет прав на управление системой, поэтому кнопки **Включить насос**, **Наполнить резервуар** и **Осушить резервуар** недоступны. О том, что текущим пользователем является **«guest»**, написано в информационном сообщении в нижней части главного окна.

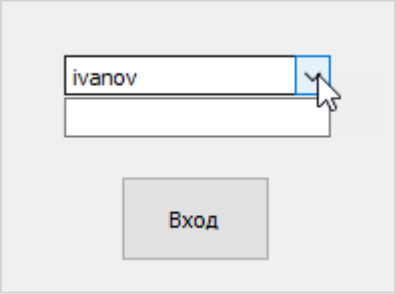
ПРИМЕЧАНИЕ

Тексты сообщений о состоянии подключения помещены в обработчики событий элемента *SecurityContext*. Ниже приведен код обработчика события **CurrentUserChanged** (стр. 63), активирующегося при смене текущего пользователя:



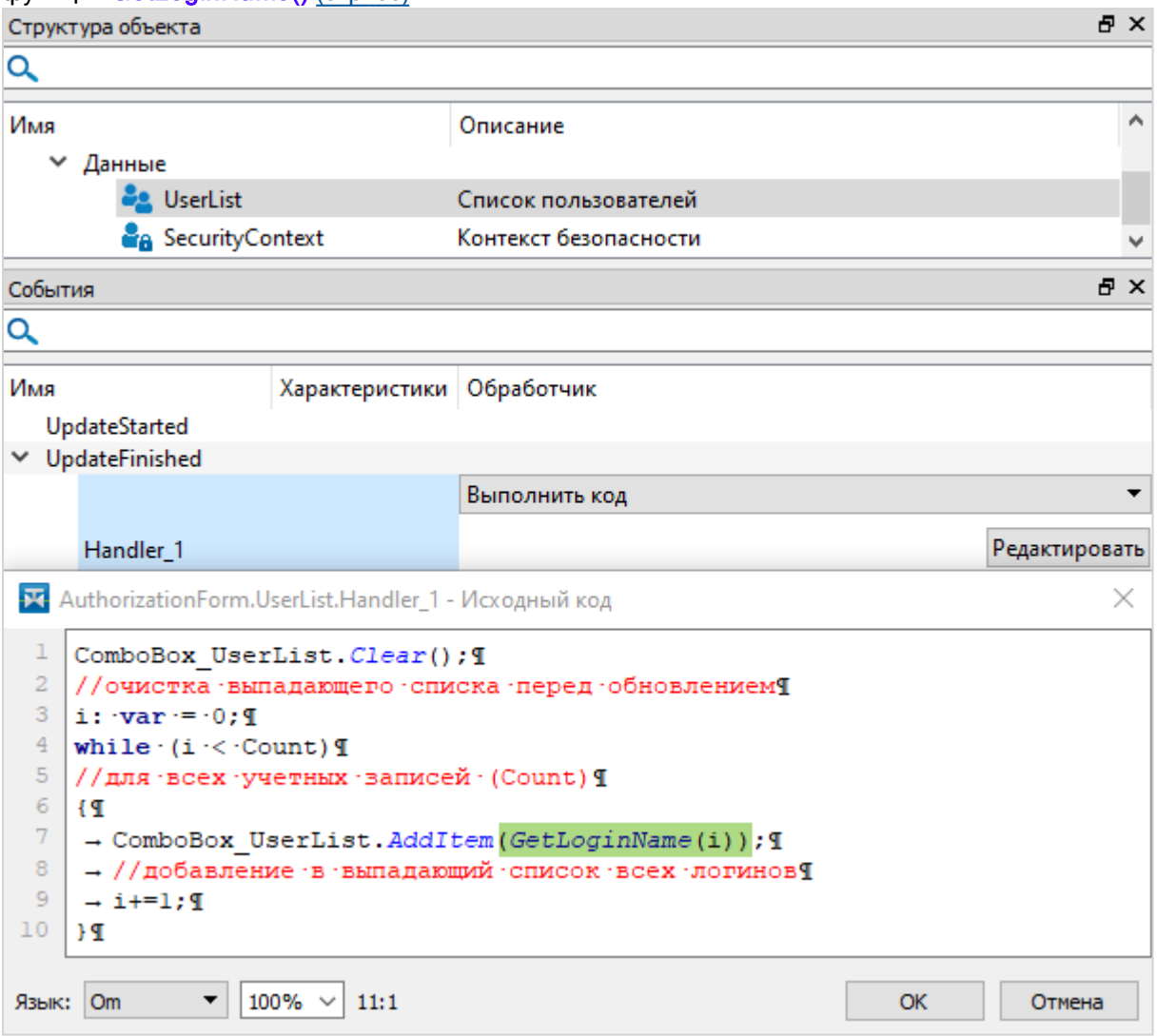
Вход с личными учетными данными

Для того, чтобы войти с личными учетными данными, нажмите кнопку **Войти**. Откроется окно входа:



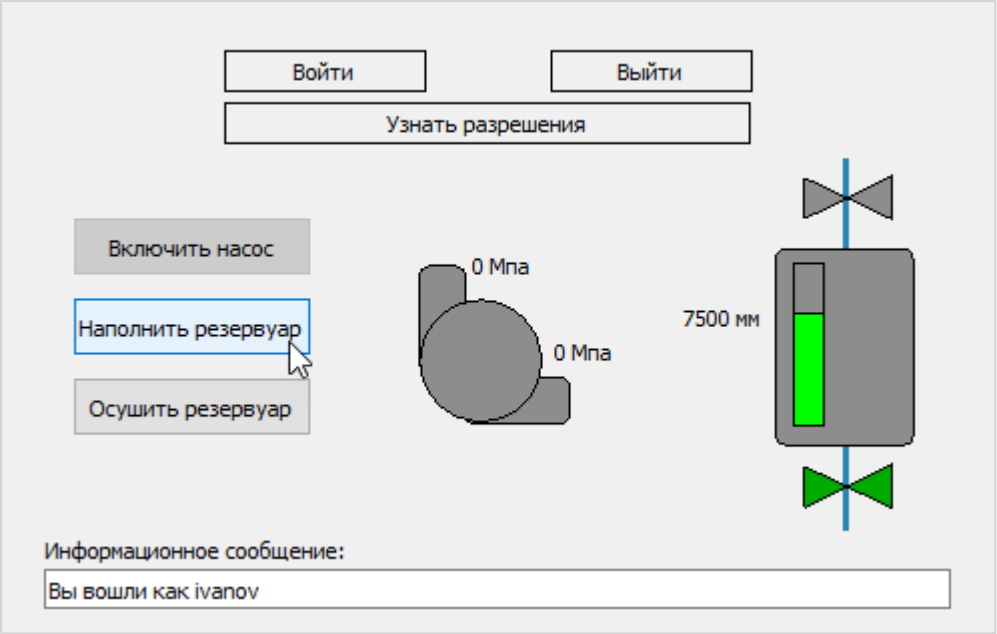
Логин можно ввести либо выбрать из выпадающего списка.

За предоставление списка учетных записей отвечает элемент *UserList*. При запуске окна входа с помощью функции **BeginUpdate()** (стр. 67) был запрошен список учетных записей из подсистемы безопасности Alpha.Security (см. код обработчика события **Opened** формы входа). Когда обновление завершилось, активировалось событие **UpdateFinished** (стр. 68) элемента *UserList*. Ознакомьтесь с кодом обработчика этого события, написанном на языке Alpha.Om. Для получения логинов учетных записей используется функция **GetLoginName()** (стр. 66):



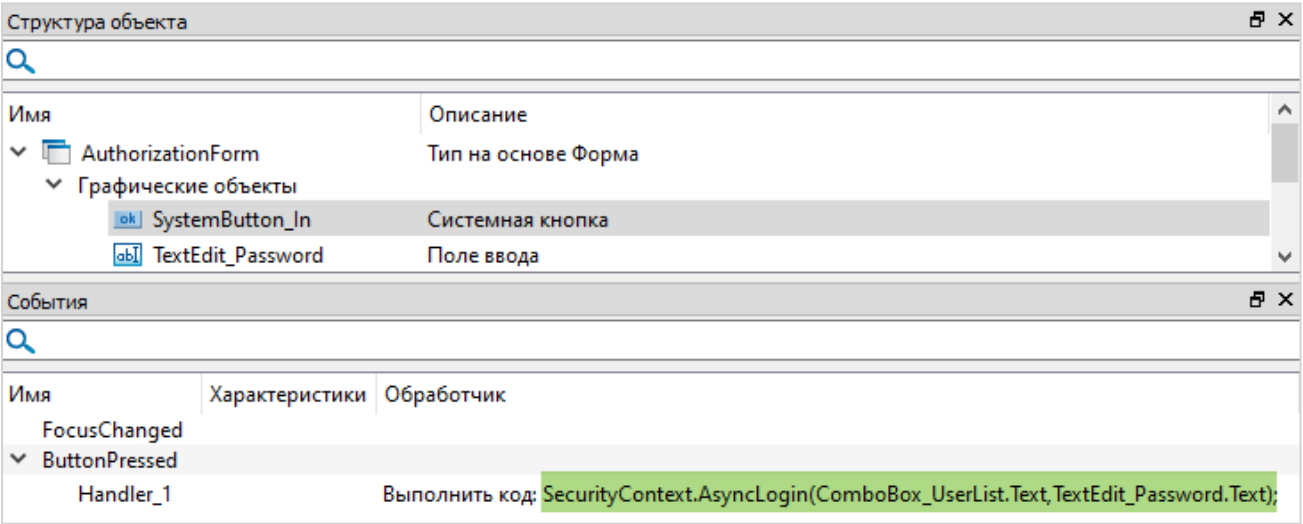
Использование проекта с личной учетной записью

Выберите логин **«ivanov»** из выпадающего списка и введите пароль **«ivanov1»**. Нажмите кнопку **Вход**. Окно входа закроется, а на главном окне станут доступны кнопки **Наполнить резервуар** и **Осушить резервуар**:



При нажатии кнопки **Вход**:

1. Была вызвана функция AsyncLogin() (стр. 56) элемента *SecurityContext*, входными параметрами для которой являются логин и пароль:

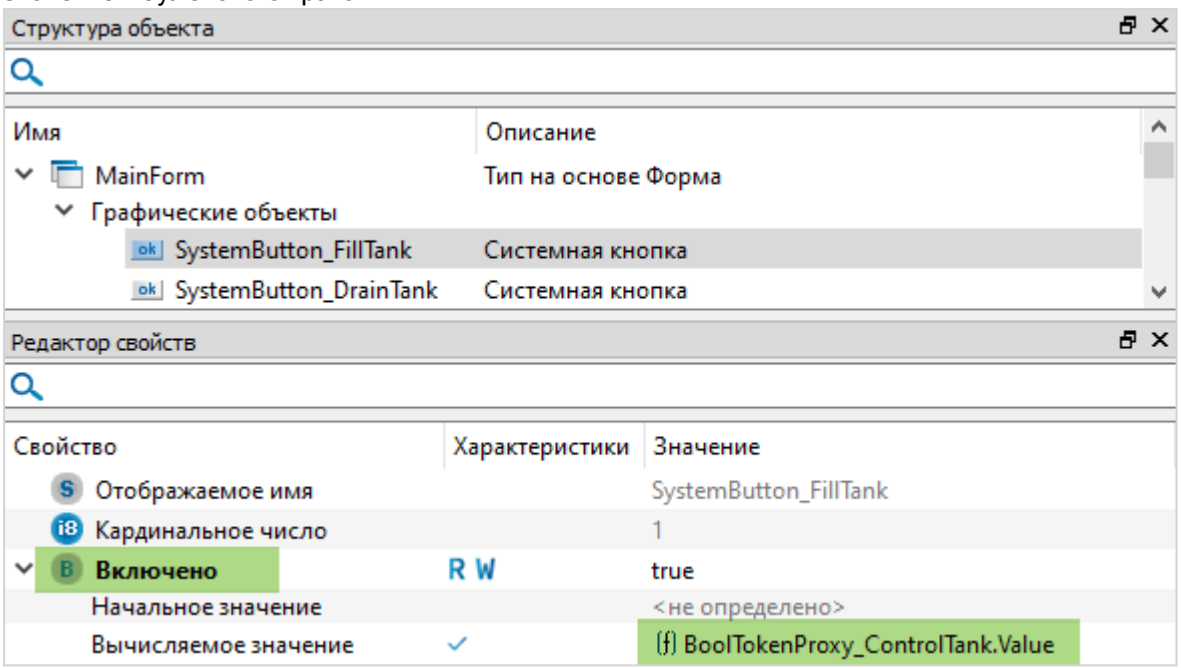


2. Агент Alpha.Security принял введенные учетные данные и зарегистрировал **«ivanov»** как текущего пользователя.

3. Агент Alpha.Security обновил текущие значения прав в соответствии с конфигурацией Alpha.Security для текущего пользователя.

4. Элементы *BoolTokenProxy_ControlTank*, *BoolTokenProxy_ControlPump* и *StringTokenProxy* получили информацию о новых значениях прав.

5. Кнопки управления резервуаром стали доступны, так как их свойства активности связаны со значением булевского права:



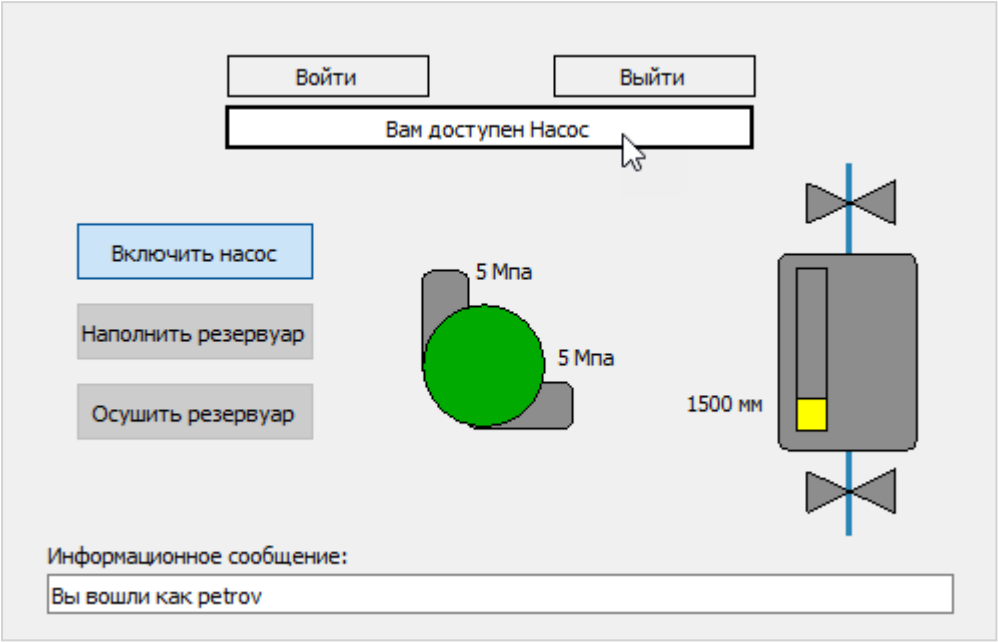
Нажмите кнопку **Узнать разрешения**. Название кнопки изменится. Теперь на ней будет написано, какое оборудование доступно пользователю. Ознакомьтесь с обработчиком события MouseClick этой кнопки. Элемент *StringTokenProxy* предоставляет значение разрешения текущего пользователя с помощью функции **GetAllowed()** (стр. 93).

Нажмите кнопку **Выйти**. Будет вызвана функция **LogOut()** (стр. 58) элемента *SecurityContext*. В информационном сообщении вы увидите, что текущим пользователем снова стал **«guest»**, а кнопки управления резервуаром недоступны.

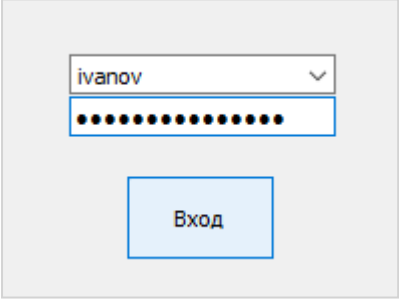
Откройте окно входа и введите другие учетные данные: логин **«petrov»** и пароль **«petrov1»**.

1. Вновь будет вызвана функция **AsyncLogin()** элемента *SecurityContext*, и на стороне Агент Alpha.Security **«petrov»** будет зарегистрирован как текущий пользователь.
2. Выполнится условие активности, связанное с правом на управление насосом, и кнопка **Включить насос** станет доступна.

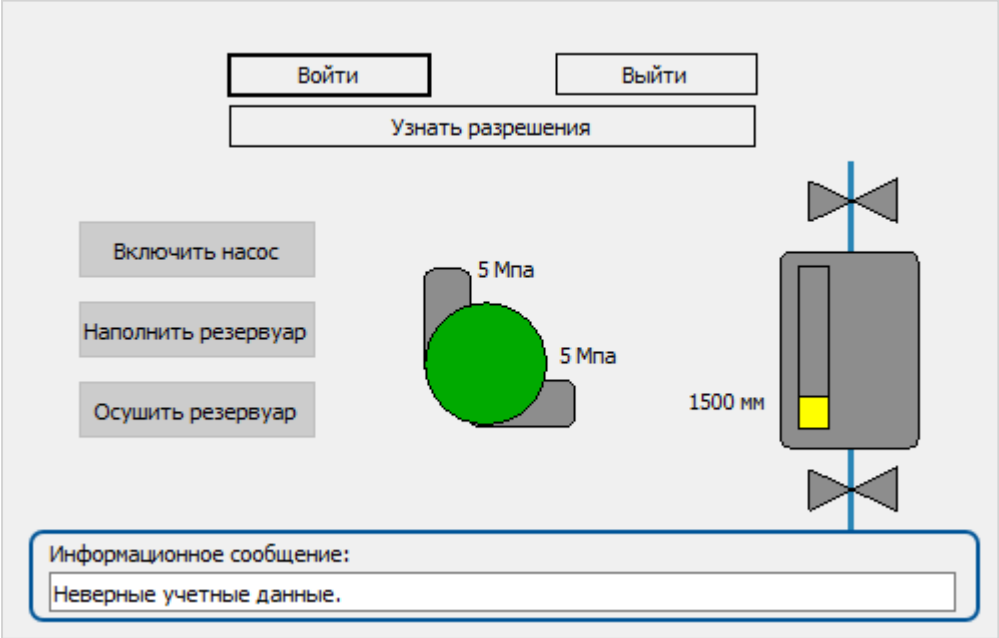
3. При нажатии на кнопку **Узнать разрешения** появится текстовое описание разрешения текущего пользователя:



Попробуйте при попытке входа ввести случайную комбинацию символов в качестве логина и/или пароля:



Так как введены несуществующие учетные данные, Агент Alpha.Security отклонит попытку входа. Активируется событие **LoginRejected** (стр. 64) элемента *SecurityContext*, и в поле вывода информационного сообщения появится соответствующее предупреждение:



1.4. Собственная форма входа

Когда пользователь вводит личные учетные данные в проекте Alpha.HMI, подсистема безопасности "запоминает" его как текущего пользователя, и может предоставлять информацию о нем сторонним приложениям. Для удобства регистрации в подсистеме безопасности можно использовать собственную форму входа.

Для создания собственной формы входа необходимы компоненты расширения Alpha.HMI.Security. В разделе описано, как с их помощью:

- использовать ограничение длительности сессий пользователей;
- использовать ограничение срока действия паролей;
- менять пароли в учетных записях.

ПРИМЕЧАНИЕ

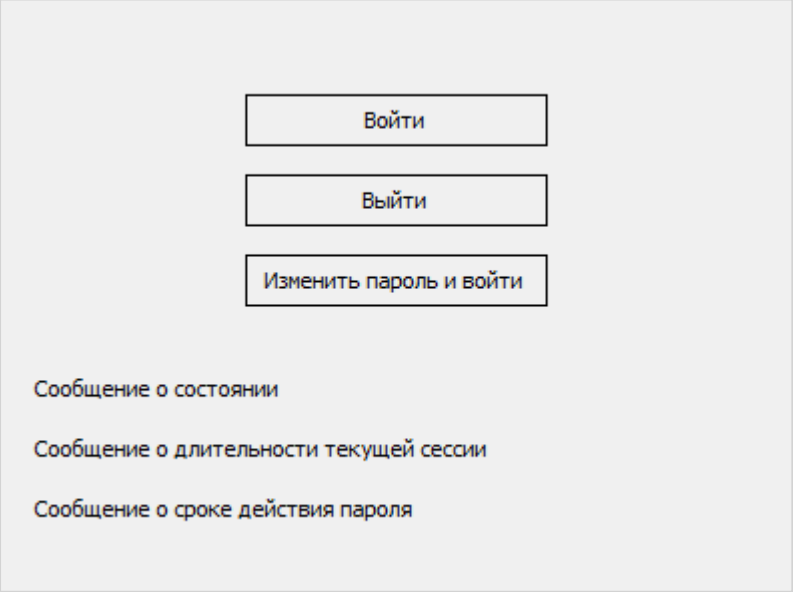
Раздел описывает расширенный сценарий применения компонентов Alpha.HMI.Security. В большинстве случаев в проектах Alpha.HMI достаточно обеспечить вход и выход пользователя так, как описано в разделе 1.3. Управление доступом в проектах (стр. 8).

Для наглядности описан проект-пример формы входа.

1. Скачайте архив с проектом-примером `AuthorizationForm.hmi` и откройте в Дизайнер Alpha.HMI.
2. Запустите Конфигуратор Alpha.Security для просмотра текущей конфигурации подсистемы безопасности.

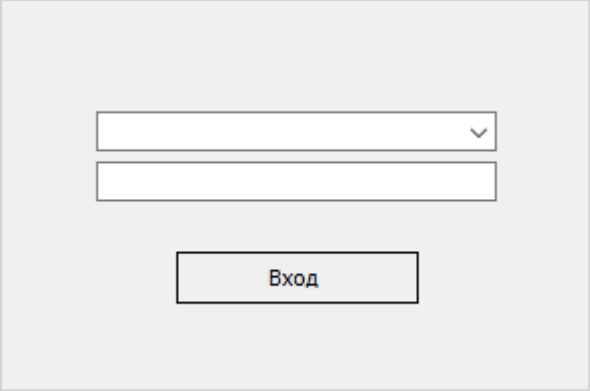
Состав проекта-примера

Основная форма проекта-примера - *`AuthorizationForm`*:



Форма *`AuthorizationForm`* предназначена для вывода информационных сообщений. На ней расположены элементы управления, поля вывода системных сообщений и два элемента типа **Таймер**. Кроме того, на форме находится *`SecurityContext`* - элемент типа **Контекст безопасности** (стр. 50), обеспечивающий взаимодействие с Alpha.Security.

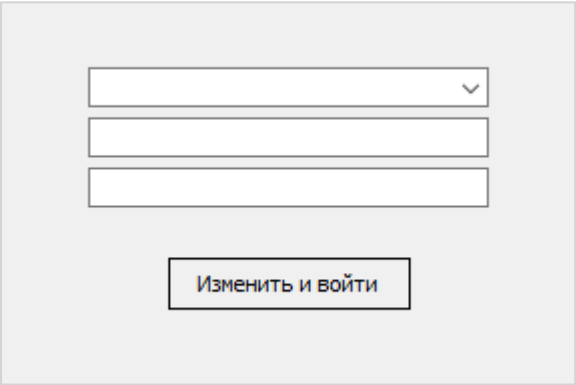
Форма входа - форма с именем *LoginForm*:



Форма *LoginForm* предназначена для ввода учетных данных. На форму добавлены:

- *SecurityContext* - элемент типа **Контекст безопасности**, обеспечивающий взаимодействие с подсистемой безопасности;
- *UserList* - элемент типа **Список пользователей** (стр. 65), позволяющий работать со списком учетных записей из подсистемы безопасности.

Форма изменения пароля - форма с именем *ChangePasswordAndLoginForm*:



Форма предназначена для изменения пароля и регистрации в подсистеме безопасности с новыми учетными данными. На форму добавлены:

- *SecurityContext* - элемент типа **Контекст безопасности**, обеспечивающий взаимодействие с подсистемой безопасности;
- *UserList* - элемент типа **Список пользователей**, позволяющий работать со списком учетных записей из подсистемы безопасности.

Уведомление об ограничении длительности сессии

Для того, чтобы ограничить длительность сессии пользователя, необходимо настроить соответствующее право **Максимальное время сессии** (**SessionDurationLimit**) в Конфигуратор Alpha.Security. Обратите внимание на вкладку учетной записи пользователя **«Иванов»**: для него это право уже добавлено, длительность сессии - 2 минуты. По истечении двух минут после входа сессия завершается, и текущим пользователем становится пользователь по умолчанию.

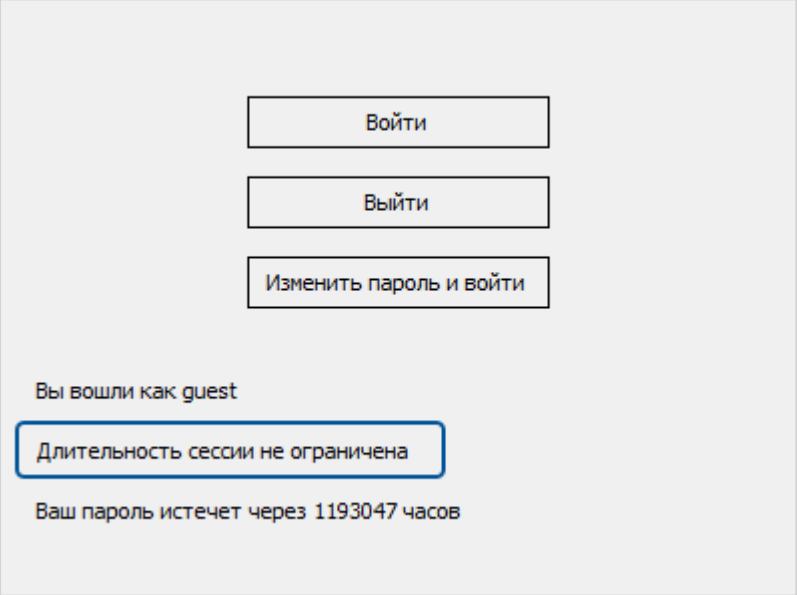
С правом **Максимальное время сессии** (**SessionDurationLimit**) связаны свойства элемента *SecurityContext*:

- SessionDurationLimit (стр. 53) - хранит значение, записанное в право **Максимальное время сессии** (**SessionDurationLimit**);
- SessionExpiresIn (стр. 53) - хранит значение времени, оставшегося до завершения сессии текущего пользователя.

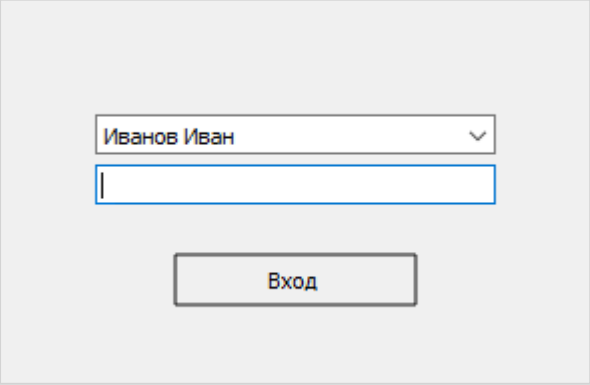
Откройте форму входа *LoginForm*. Ознакомьтесь с кодом обработчика события OnTimer элемента *Timer_Session*, написанном на языке Alpha.Om. Свойство SessionExpiresIn применяется для вывода сообщения об оставшемся до завершения сессии времени:

```
if (SecurityContext.SessionExpiresIn == 0)
//если время до истечения сессии пользователя равно 0, сессия не ограничена
{
Text_SysMsgSessionDuration.Text = "Длительность сессии не ограничена";
}
else
{
//иначе выводится значение времени, оставшегося до истечении сессии,
переведенное в минуты
Text_SysMsgSessionDuration.Text="До конца сессии осталось "
+String.ToString(SecurityContext.SessionExpiresIn/60+1)+" минут(-ы)";
}
```

Запустите проект в рантайм. Откроется окно выбора действия - основное окно проекта. Для пользователя по умолчанию **«guest»** длительность сессии не ограничена, о чем будет написано в информационном сообщении:

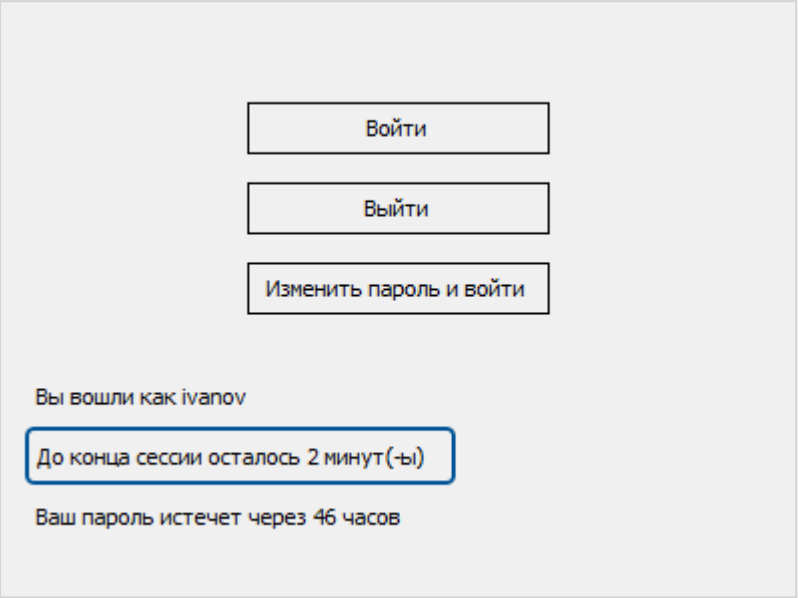


Нажмите кнопку **Войти**. Откроется окно входа:



Из выпадающего списка выберите имя **«Иванов»** и введите пароль **«ivanov1»**. Окно входа закроется, а в информационном сообщении основного окна появится уведомление о том, что:

- текущим пользователем стал пользователь с логином **«ivanov»**;
- до завершения сессии текущего пользователя осталось 2 минуты:



ПРИМЕЧАНИЕ

О том, как добавить в выпадающий список логины учетных записей, подробнее написано в разделе 1.3. Управление доступом в проектах (стр. 8).

При вводе учетных данных видеть имя в выпадающем списке привычнее, чем логин. Поэтому в данном примере выпадающий список заполняется не логинами, а отображаемыми именами. Для этого вместо функции **GetLoginName()** (стр. 66) используется функция **GetDisplayName()** (стр. 67) элемента *UserList*. Функция **GetDisplayName()** возвращает отображаемое имя учетной записи.

Уведомление об истечении срока действия пароля

Для того, чтобы установить срок действия пароля учетной записи, необходимо настроить право **Срок действия пароля** (**PasswordAge**) в Конфигуратор Alpha.Security. Для пользователя с именем **«Иванов»** максимальный срок действия пароля - 2 дня. Через два дня после изменения пароля попытки входа отклоняются подсистемой безопасности.

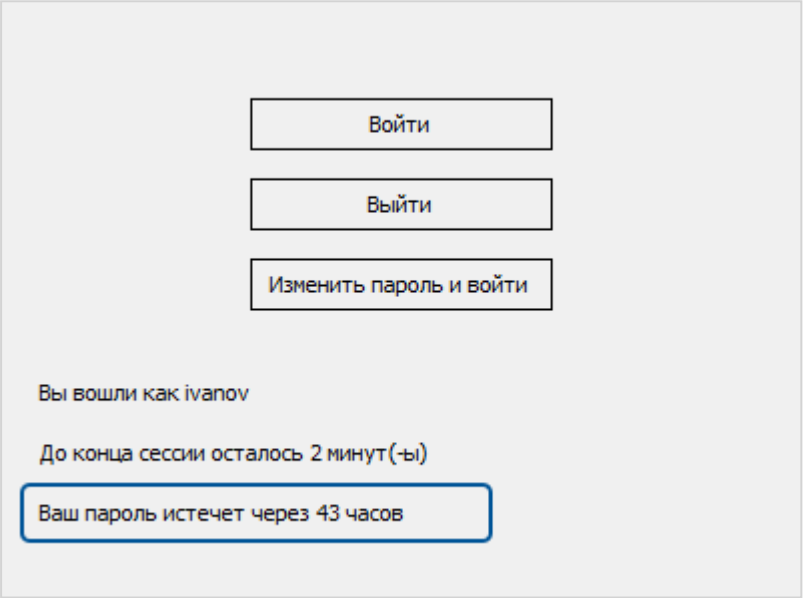
С правом **Срок действия пароля** (**PasswordAge**) связано свойство **PasswordExpiresIn** (стр. 54) элемента *SecurityContext*. Это свойство хранит оставшееся время действия пароля.
Откройте форму входа *LoginForm*. Ознакомьтесь с кодом обработчика события **OnTimer** элемента *Timer_Password*, написанном на языке Alpha.Om. Свойство **PasswordExpiresIn** применяется для вывода сообщения об истечении срока действия пароля:

```
if (SecurityContext.PasswordExpiresIn == 0)
//если время до истечения срока действия пароля равно 0, срок действия пароля не ограничен
{
Text_SysMsgPswdExpiration.Text = "Срок действия пароля не ограничен";
}
else
```

```
{
//иначе выводится значение времени, оставшегося до конца срока действия пароля,
переведенное в часы
Text_SysMsgPswdExpiration="Ваш пароль истечет через "
+String.ToString(SecurityContext.PasswordExpiresIn/3600+1)+" часов";
}
```

Запустите проект в рантайм. Войдите с учетными данными **«Иванов»**. В информационном сообщении основного окна появится уведомление о том, что:

- текущим пользователем стал пользователь с логином **«ivanov»**;
- срок действия пароля текущего пользователя истечет в указанное время:



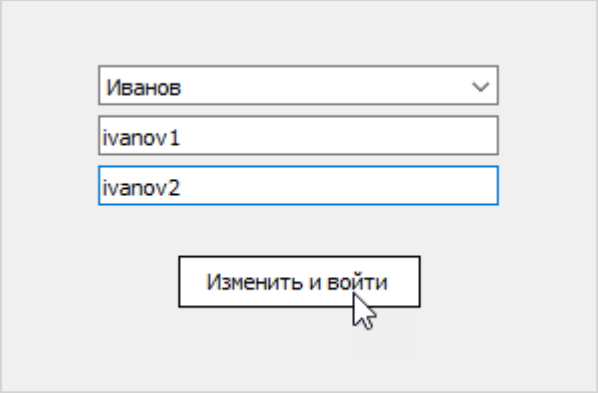
Изменение пароля учетной записи

Когда необходимо реализовать возможность изменения пароля, можно использовать функцию AsyncLoginWithPasswordChange() (стр. 57) элемента *SecurityContext*.
Откройте форму изменения пароля *ChangePasswordAndLoginForm*. Ознакомьтесь с обработчиком события MouseClick кнопки **Изменить и войти**:

```
SecurityContext.AsyncLoginWithPasswordChange(UserName[ComboBox_
UserList.SelectedIndex],
TextEdit_OldPswd,TextEdit_NewPswd);
//выполнение функции AsyncLoginWithPasswordChange, входными параметрами для
которой являются
логин, старый пароль и новый пароль
Close();
```

Входными параметрами функции AsyncLoginWithPasswordChange() являются логин, текущий и новый пароль, введенные в соответствующие поля.

Запустите проект в рантайм и нажмите кнопку **Изменить пароль и войти**. Откроется окно изменения пароля. Выберите из выпадающего списка имя учетной записи **«Иванов»**, введите текущий пароль учетной записи **«ivanov1»**, а затем придумайте и введите новый пароль. Нажмите кнопку **Изменить и войти**:



Окно изменения пароля закроется, а результат операции появится в информационном сообщении основного окна. Если операция завершится без ошибок, будет осуществлен вход с новыми учетными данными.

ОБРАТИТЕ ВНИМАНИЕ

Пароль не удастся изменить, если не вышел минимальный срок его действия. Сделать это может только пользователь с правами администратора в Конфигуратор Alpha.Security. Минимальный срок действия пароля указан в праве **Срок действия пароля** (**PasswordAge**).

1.5. Собственный конфигуратор подсистемы безопасности

В разделе продемонстрировано создание собственного конфигуратора подсистемы безопасности Alpha.Security.

Описано, как с помощью компонентов расширения Alpha.HMI.Security:

- подключиться к конкретному каталогу на определенном LDAP-сервере;
- получать данные об учетных записях пользователей, группах пользователей и приложениях, имеющихся в подсистеме безопасности Alpha.Security, из проектов Alpha.HMI;
- менять имеющиеся учетные записи, группы и приложения из проектов Alpha.HMI;
- создавать новые учетные записи, группы и приложения из проектов Alpha.HMI;
- сохранять резервные копии конфигурации Alpha.Security и восстанавливать их.

ПРИМЕЧАНИЕ

Раздел описывает расширенный сценарий применения компонентов Alpha.HMI.Security. В большинстве случаев создавать собственный конфигуратор не обязательно: для конфигурирования подсистемы безопасности можно использовать Alpha.HMI.SecurityConfigurator или Конфигуратор Alpha.Security, поставляемый в составе дистрибутива Alpha.Security.

Описанный проект-пример представляет собой простейший конфигуратор подсистемы безопасности.

1. Скачайте архив с проектом-примером `OwnConfigurator.hmi` и откройте в Дизайнер Alpha.HMI.
2. Запустите Конфигуратор Alpha.Security для просмотра конфигурации подсистемы безопасности.

ПРИМЕЧАНИЕ

Проект-пример позволяет создать новые и просматривать имеющиеся учетные записи пользователей. Создание и просмотр групп пользователей и приложений можно реализовать с помощью аналогичных компонентов, их свойств, функций и событий, указанных в примечании к каждому подразделу.

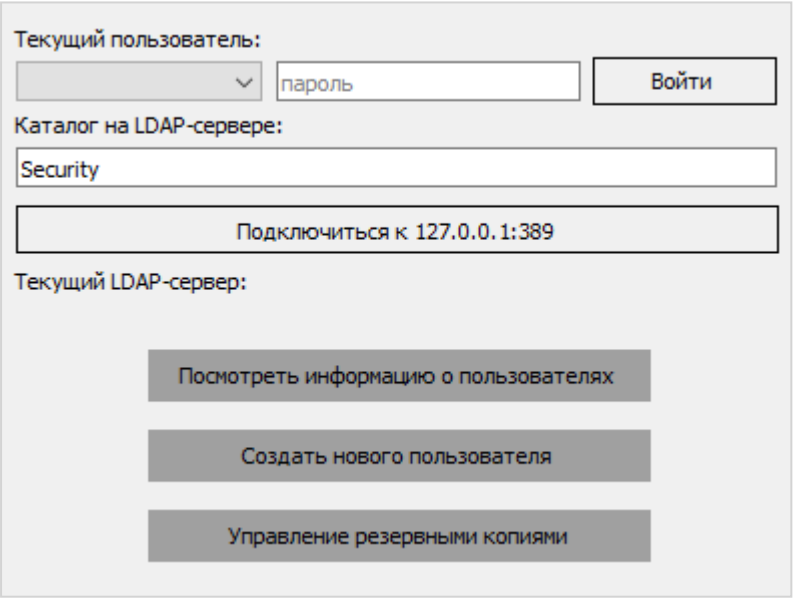
Подключение к каталогу на LDAP-сервере

Вся информация о пользователях, группах, приложениях и правах хранится в каталогах на LDAP-сервере. Подробнее об этом читайте в документации на Alpha.Security, в разделах "О продукте" и "LDAP-сервер и его настройка".

Обратите внимание, что для получения и изменения информации необязательно подключаться к LDAP-серверу, как описано ниже. По умолчанию агент безопасности подключается к каталогу и LDAP-серверу, указанным в конфигурационном файле подсистемы безопасности `alpha.security.agent.xml`. Действия, описанные ниже, выполняются для подключения к другим каталогам и LDAP-серверам.

Чтобы просмотреть или изменить информацию в конкретном каталоге на определенном LDAP-сервере, к нему необходимо подключиться. При этом потребуется предоставить агенту безопасности учетные данные администратора или пользователя, имеющего права на просмотр и изменение конфигурации.

Ознакомьтесь с основной формой проекта-примера.



На форму добавлены:

- Элемент типа **Настройка безопасности: Менеджер** - *SecurityManager*, позволяющий подключиться к конкретному каталогу на определенном LDAP-сервере.
- Экземпляры *Login* и *Password*, предназначенные для ввода учетных данных администратора. Как выполняется вход в подсистему безопасности, описано в разделе 1.3. Управление доступом в проектах (стр. 8).
- Экземпляр *CatalogName*, предназначенный для ввода имени существующего каталога на LDAP-сервере.
- Кнопка *LDAP_Enter*, предназначенная для открытия соединения с каталогом на LDAP-серерве.
- Другие кнопки, по умолчанию недоступны.

Для подключения к LDAP-серверу вызывается функция SetConfigureServer() экземпляра *SecurityManager*. Ознакомьтесь с обработчиком события ButtonPressed кнопки *LDAP_Enter*.

```
SecurityManager.SetConfigureServer
("127.0.0.1",389,false,"cn="+Login.Text+",dc=maxcrc,dc=com",
Password.Text,"ou="+CatalogName.Text+",dc=maxcrc,dc=com");
```

В качестве входных параметров функции указаны:

- адрес и порт локального LDAP-сервера: `"127.0.0.1",389`;
- защищено ли подключение: `false`;
- каталог учетной записи администратора LDAP в виде, принятом для LDAP: `"cn="+Login.Text+",dc=maxcrc,dc=com"` (логин учетной записи берется из выпадающего списка логинов *Login*);
- пароль учетной записи администратора LDAP: `Password.Text` (пароль учетной записи берется из поля ввода пароля *Password*, при передаче на LDAP-сервер значение шифруется);
- имя каталога, к которому выполняется подключение, в виде, принятом для LDAP: `"ou="+CatalogName.Text+",dc=maxcrc,dc=com"` (название каталога берется из поля ввода названия каталога *CatalogName*).

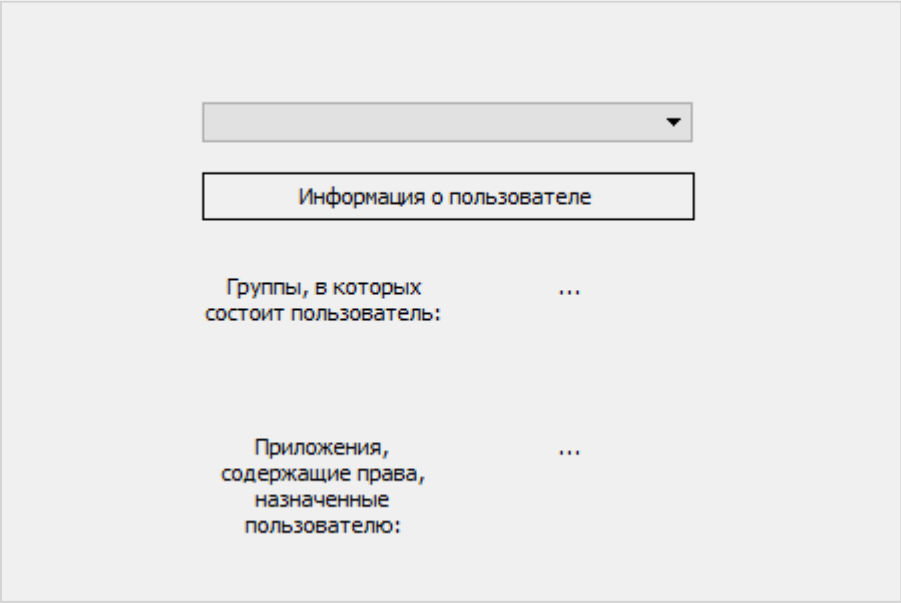
Запустите проект-пример в рантайм. Введите учетные данные администратора и нажмите **Войти**. Затем нажмите кнопку **Подключиться к 127.0.0.1:389**. По нажатию кнопки будет вызвана функция SetConfigureServer() экземпляра *SecurityManager*. Затем активируется одно из двух событий экземпляра:

- В случае успешного подключения активируется событие LDAPServerChanged. Станут доступны кнопки **Посмотреть информацию о пользователях**, **Создать нового пользователя** и **Управление резервными копиями**, а в строке под кнопкой подключения отобразится адрес текущего LDAP-сервера.
- В случае, если подключение выполнено не будет, активируется событие LDAPServerChangeFailure. В строке под кнопкой отобразится сообщение об ошибке подключения.

Ознакомьтесь с кодом обработчиков обоих событий.

Просмотр учетных записей пользователей

Откройте форму просмотра учетных записей пользователей *ViewUserForm* в проекте-примере.



На форму *ViewUserForm* добавлены:

- *SecurityManager* - элемент типа **Настройка безопасности: Менеджер** (стр. 96), позволяющий запрашивать и удалять имеющиеся учетные записи, группы и приложения;
- *SecurityManagerUser* - элемент типа **Настройка безопасности: Пользователь** (стр. 120), позволяющий добавлять новые и менять имеющиеся учетные записи пользователей.

Извлечение списка учетных записей из подсистемы безопасности

Для извлечения списка учетных записей используются возможности элемента *SecurityManager*:

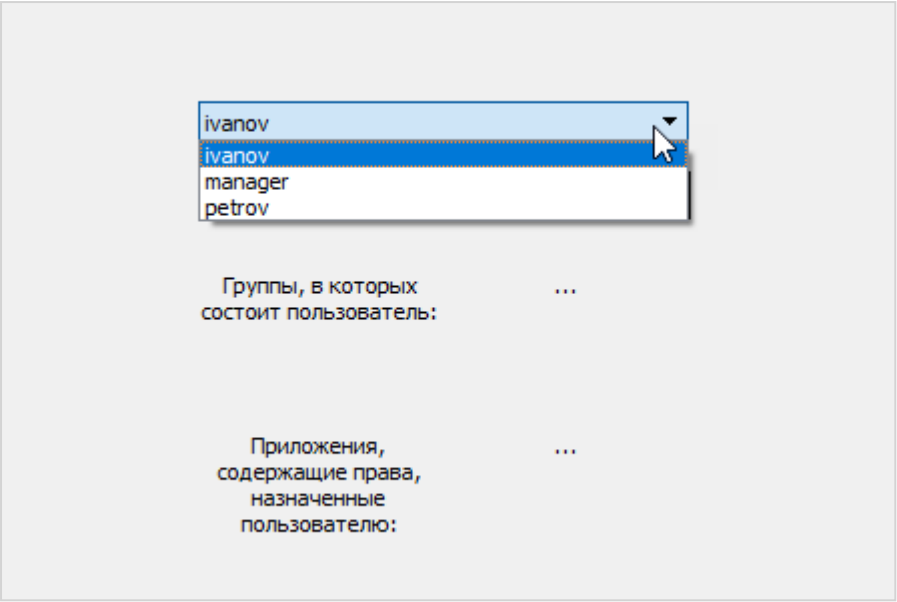
1. При запуске окна формы *ViewUserForm* вызывается функция **RequestUsersList()** (стр. 98), запрашивающая список учетных записей из подсистемы безопасности Alpha.Security.
2. Если список получен, активируется событие **RequestUsersListComplete** (стр. 104).

Результат активации события **RequestUsersListComplete** - список учетных записей пользователей - хранится в переменной JSONUsersList. Данные хранятся в формате JSON. Ознакомьтесь с кодом обработчика события **RequestUsersListComplete** элемента *SecurityManager* из проекта-примера, написанном на языке JavaScript.

```
var usrs = JSON.parse(JSONUsersList);
//преобразование данных из формата JSON в объект
for (var i = 0; i < usrs.data.length; i++)
{
 //для каждого массива внутри объекта usrs
 unit.Global.UserName[i] = usrs.data[i].symbolicId;
 //извлечение имени пользователя и помещение его в глобальную переменную
 unit.Global.UserId[i] = usrs.data[i].uid;
 //извлечение uid пользователя и помещение его в глобальную переменную
 ComboBox_Users.AddItem(unit.Global.UserName[i]);
 //добавление имени пользователя в выпадающий список
}
```

Данный скрипт извлекает из JSON-структуры логины (symbolicId) и их идентификаторы (uid). Для удобства дальнейшего использования извлеченные данные помещаются в глобальные переменные *UserName[]* и *UserId[]* соответственно.

Запустите проект-пример в рантайм. Откроется окно выбора действия, и, если было выполнено подключение к LDAP-серверу, кнопка **Посмотреть информацию о пользователях** будет доступна. Нажмите кнопку. Откроется окно просмотра информации об учетных записях пользователей. В выпадающий список попадут логины всех учетных записей из подсистемы безопасности.



ПРИМЕЧАНИЕ

Аналогично из подсистемы безопасности Alpha.Security извлекаются списки групп и приложений:

- для получения списка групп используйте функцию **RequestGroupList()** (стр. 98), событие **RequestGroupListComplete** (стр. 104) и переменную JSONGroupList элемента *SecurityManager*;
- для получения списка приложений используйте функцию **RequestAppList()** (стр. 97), событие **RequestAppListComplete** (стр. 103) и переменную JSONAppList элемента *SecurityManager*.

Просмотр информации о пользователе

Для того, чтобы просмотреть информацию о выбранной из выпадающего списка учетной записи, её необходимо сначала загрузить в элемент *SecurityManagerUser*. Это можно сделать с помощью функции **Load()** (стр. 122), входным параметром для которой является уникальный идентификатор выбранной учетной записи (uid). Ознакомьтесь с кодом обработчика события **MouseClick** кнопки **Информация о пользователе**.

```
SecurityManagerUser.Load(unit.Global.UserId[ComboBox_Users.SelectedIndex]);
//загрузка информации о выбранной учетной записи в объект SecurityManagerUser по
идентификатору
Groups_List.Text = "";
//очистка информационного поля при выборе новой УЗ
Apps_List.Text = "";
//очистка информационного поля при выборе новой УЗ
```

После загрузки данных активируется событие **LoadComplete** (стр. 130) элемента *SecurityManagerUser*. Ознакомьтесь с кодом обработчика этого события, написанном на языке JavaScript.

```
JSONGroupList = SecurityManagerUser.GetGroupsList();
//получение списка групп, в которых состоит пользователь
JSONAppList = SecurityManagerUser.GetApplicationsList();
//получения списка приложений, содержащих права пользователя
var grpslst = JSON.parse(JSONGroupList);
var applst = JSON.parse(JSONAppList);
```

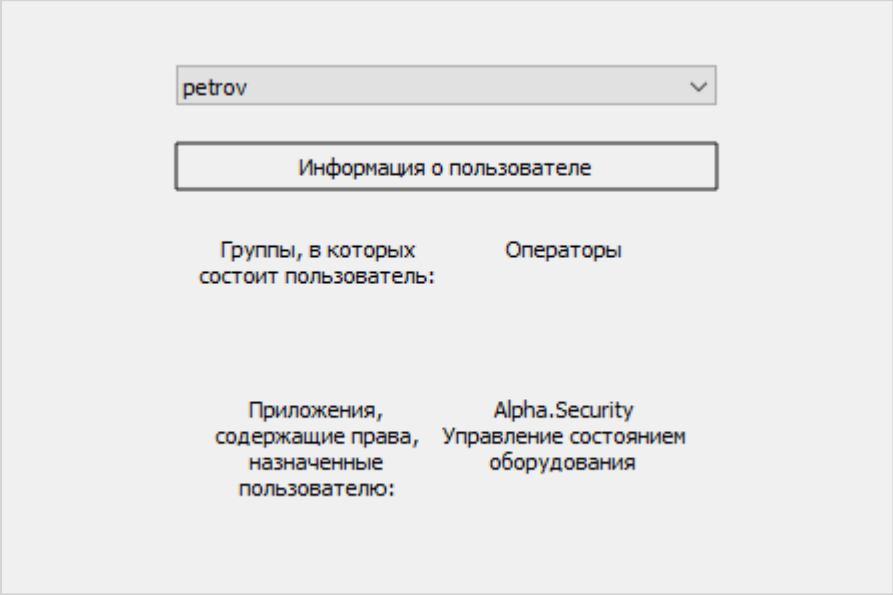
```
for (var v of grpslst.data)
//для всех элементов коллекции
{
 Groups_List.Text += v.groupName + "\n";
 //добавление названия группы в список
}
for (var v of applst.data)
//для всех элементов коллекции
{
 Apps_List.Text += v.appName + "\n";
 //добавление названия приложения в список
}
```

Данные об учетной записи извлекаются из элемента *SecurityManagerUser* путем обращения к его свойствам и функциям. Например:

- функция **GetGroupsList()** (стр. 124) возвращает данные о группах, в которых состоит пользователь, в JSON-формате;
- функция **GetApplicationsList()** (стр. 124) возвращает данные о приложениях с правами, назначенными пользователю, в JSON-формате.

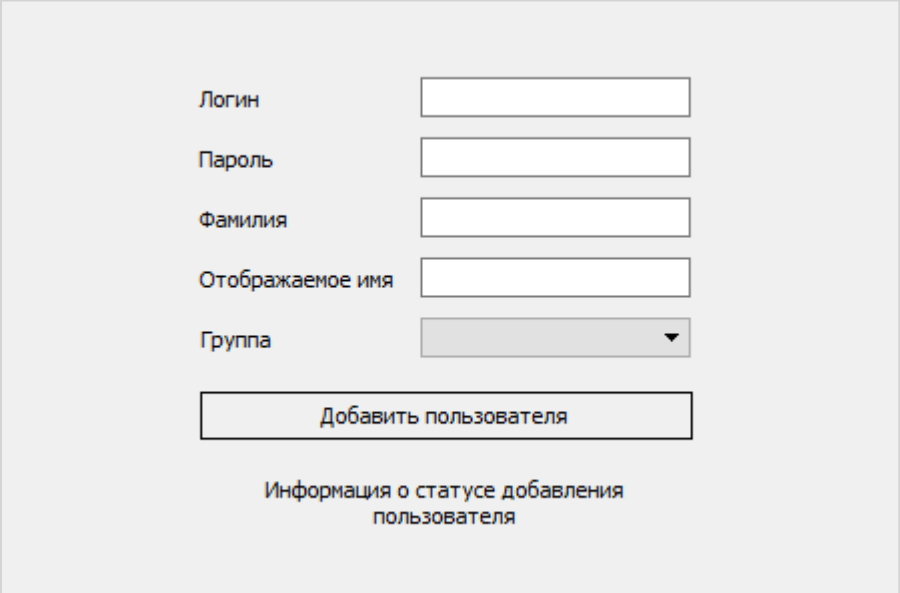
Поместив данные в переменные *JSONGroupList* и *JSONAppList*, можно извлечь из JSON-структуры необходимую информацию.

Запустите проект-пример в рантайм. Откроется окно выбора действия, и, если было выполнено подключение к LDAP-серверу, кнопка **Посмотреть информацию о пользователях** будет доступна. Нажмите кнопку. Откроется окно просмотра информации об учетных записях пользователей. Выберите любой логин из выпадающего списка и нажмите кнопку **Информация о пользователе**. В поле вывода информации появятся данные о группах и приложениях.



Создание новой учетной записи пользователя

Откройте форму создания новой учетной записи *NewUserForm* в проекте-примере.



На форму создания новой учетной записи *NewUserForm* добавлены:

- *SecurityManager* - элемент типа **Настройка безопасности: Менеджер** (стр. 96), позволяющий запрашивать и удалять имеющиеся учетные записи, группы и приложения;
- *SecurityManagerUser* - элемент типа **Настройка безопасности: Пользователь** (стр. 120), позволяющий добавлять новые и менять существующие учетные записи пользователей.

Для добавления новой учетной записи используйте функции и свойства элемента *SecurityManagerUser*. Ознакомьтесь с кодом обработчика события MouseClick кнопки **Добавить пользователя**, написанном на языке Alpha.Om.

```
SecurityManagerUser.New();
//создание новой УЗ пользователя
SecurityManagerUser.Login=TextEdit_Login;
//присвоение логина новой УЗ
SecurityManagerUser.SetPassword(TextEdit_Pswd);
//присвоение пароля новой УЗ
SecurityManagerUser.Surname=TextEdit_Surname;
//присвоение фамилии новой УЗ
SecurityManagerUser.DispalyName=TextEdit_DisplayName;
//присвоение отображаемого имени новой УЗ
SecurityManagerUser.AddGroup(unit.Global.GroupId[ComboBox_Group.SelectedIndex]);
//вызов функции AddGroup для добавления пользователя в выбранную в выпадающем
списке группу
SecurityManagerUser.Save();
//сохранение введенных данных
```

Здесь:

- функция New() (стр. 128) создает, а функция Save() (стр. 122) - сохраняет новую учетную запись в подсистеме безопасности;
- свойства элемента *SecurityManager* (Login, Surname, DisplayName (стр. 121)) получают значения, введенные в текстовые поля;
- функция SetPassword() (стр. 123) принимает введенное значение в качестве пароля;
- функция AddGroup() (стр. 128) принимает uid группы, выбранной из выпадающего списка, в качестве входного параметра и добавляет пользователя в соответствующую группу.

ПРИМЕЧАНИЕ

Пользователю можно назначить только одну группу.

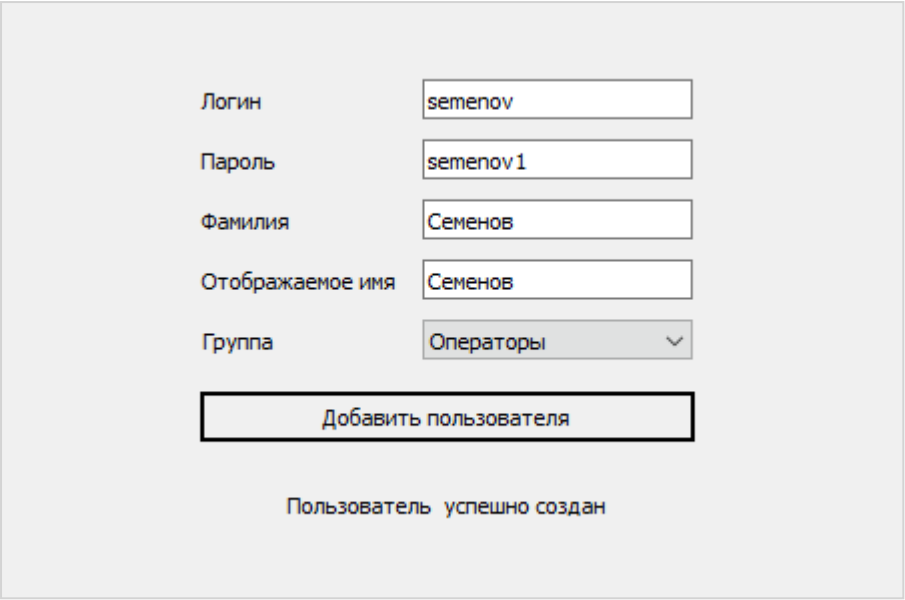
ПРИМЕЧАНИЕ

При вызове функции Save() выполнится автоматическая проверка уникальности указанного логина. Если логин не уникален, агент не сохранит учетную запись. Узнать об этой ошибке можно с помощью события SaveFailed (стр. 131) элемента *SecurityManagerUser*, подробнее об этом написано ниже.

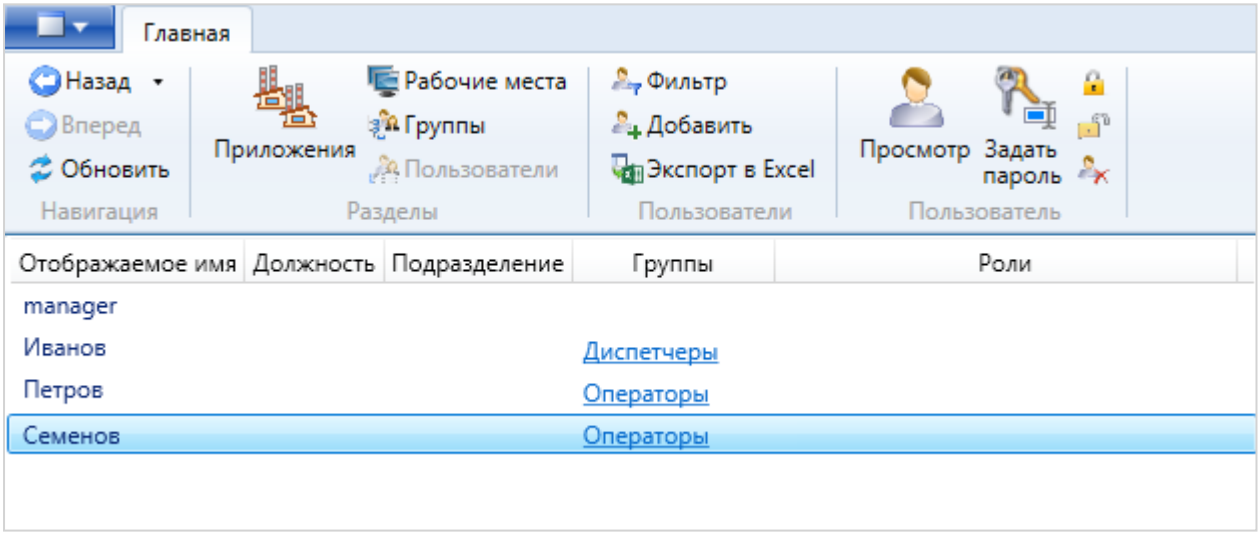
ВАЖНО

Будьте внимательны при работе с проектом-примером. Описанные ниже действия ведут к изменению конфигурации Alpha.Security.

Запустите проект-пример в рантайм. Откроется окно выбора действия, и, если было выполнено подключение к LDAP-серверу, кнопка **Создать нового пользователя** будет доступна. Нажмите кнопку. Откроется окно создания новой учетной записи. Попробуйте ввести данные новой учетной записи и нажать кнопку **Добавить пользователя**. Агент безопасности предоставит сообщение об успешности операции.



В Конфигуратор Alpha.Security нажмите кнопку **Обновить** и убедитесь в том, что новая учетная запись создана.



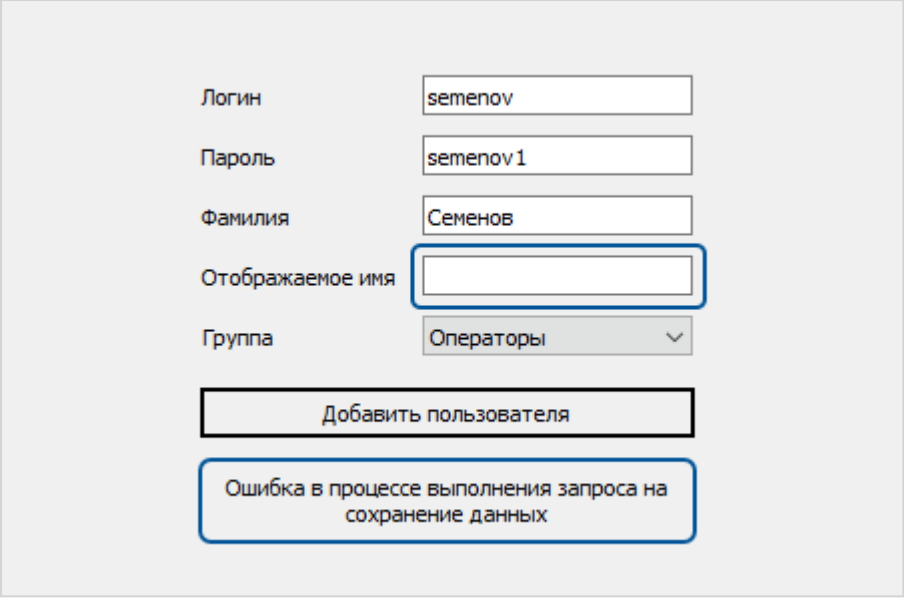
ОБРАТИТЕ ВНИМАНИЕ

Набор полей ввода и необходимость их заполнения можно определять в ходе разработки собственного конфигуратора. Для подсистемы безопасности Alpha.Security обязательными являются:

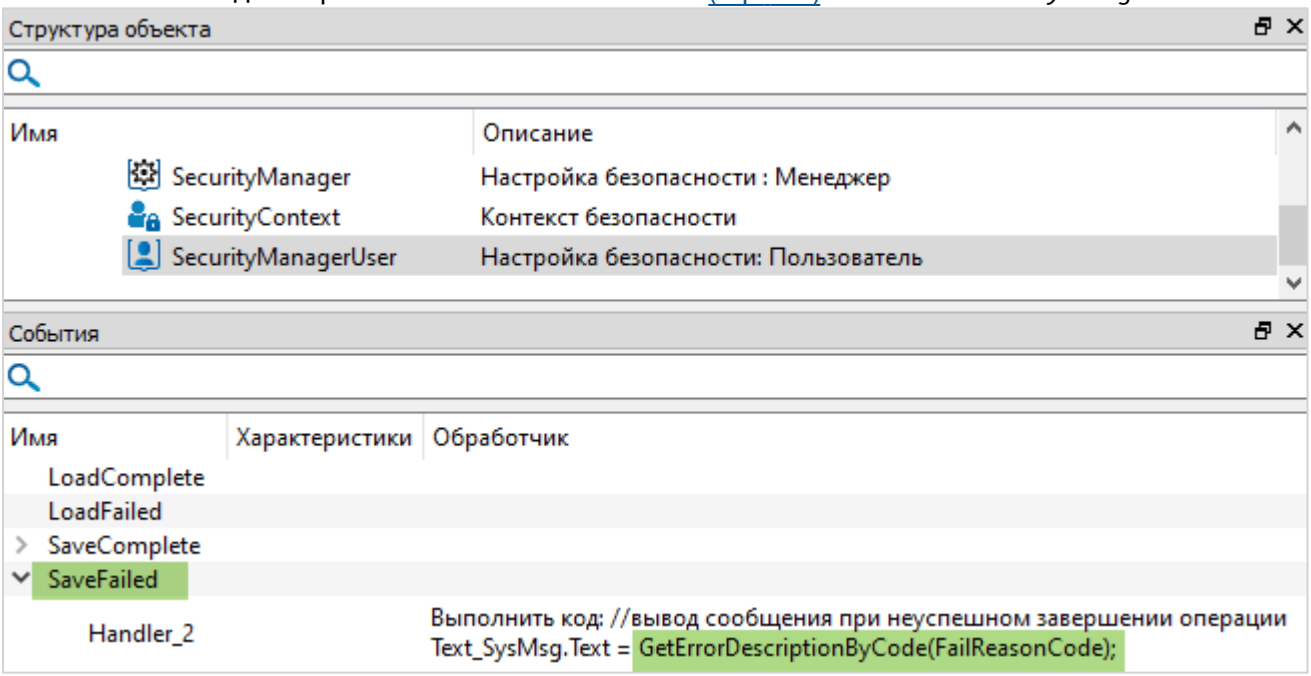
- значение свойства Login;
- значение свойства Surname;
- значение свойства DisplayName;
- результат вызова функции SetPassword().

Остальные свойства, функции и ограничения на их значения применяются исходя из требований к разрабатываемому конфигуратору.

Если при создании новой учетной записи не заполнено хотя бы одно из обязательных для подсистемы безопасности полей, или указан не уникальный логин, сохранить учетные данные не удастся.

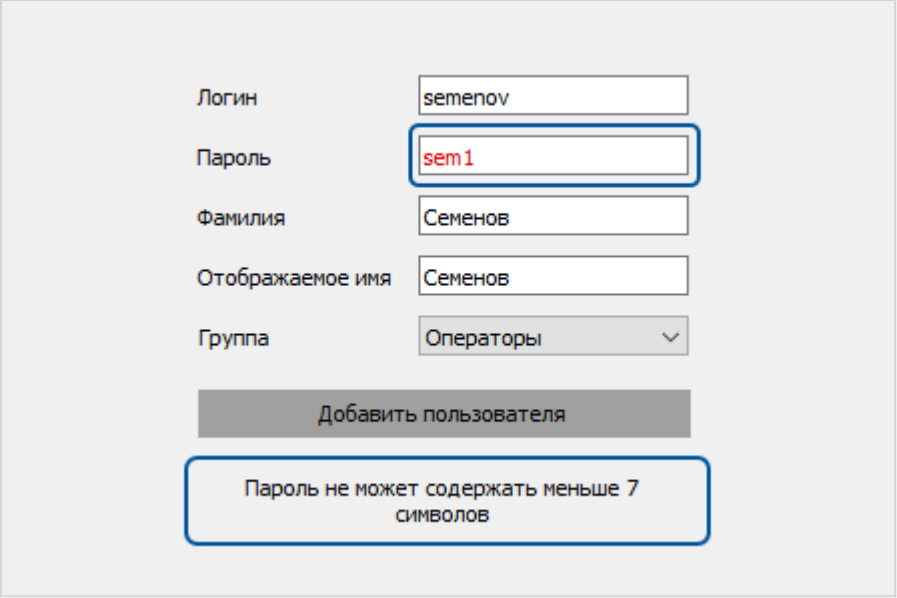


Ознакомьтесь с кодом обработчика события **SaveFailed** (стр. 131) элемента *SecurityManagerUser*.



Подсистема безопасности не принимает новую учетную запись без заполнения обязательного поля. Текст ошибки можно получить с помощью функции **GetErrorDescriptionByCode()** (стр. 130), входным параметром для которой является переменная FailReasonCode, хранящая код ошибки.

Попробуйте ввести пароль, не содержащий цифр и/или состоящий менее чем из семи символов.



Проверка требований к паролю настроена при разработке проекта-примера и выполняется в обработчике события **TextChanged** элемента *TextEdit_Pswd*. Сама подсистема безопасности не накладывает требований к содержанию пароля создаваемой учетной записи.

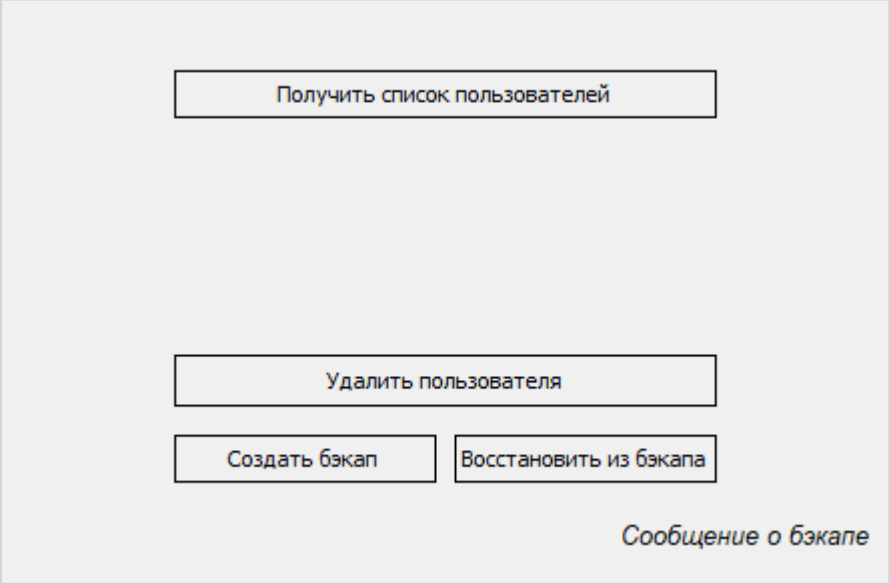
ПРИМЕЧАНИЕ

Аналогично в подсистеме безопасности Alpha.Security создаются группы и приложения:

- для создания групп пользователей используйте функции и свойства элемента типа **Настройка безопасности: Группа** (стр. 131);
- для создания приложений используйте функции и свойства элемента типа **Настройка безопасности: Приложение** (стр. 110).

Создание и восстановление резервной копии

Откройте форму управления резервными копиями *Backups* в проекте-примере.



На форму *Backups* добавлен *SecurityManager* - элемент типа **Настройка безопасности: Менеджер** (стр. 96).

Создание резервной копии

Для создания резервных копий используются возможности элемента *SecurityManager*:

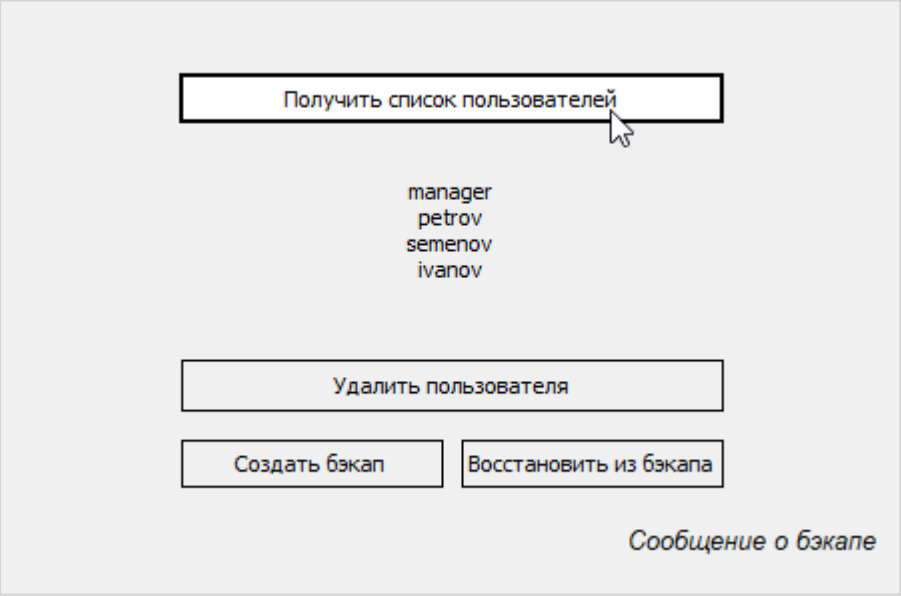
1. При нажатии кнопки *Button_CreateBckp* вызывается функция **ExportConfiguration()** (стр. 100), создающая файл резервной копии в указанном расположении. В данном случае резервная копия создается в папке текущего пользователя ОС. Ознакомьтесь с кодом обработчика **MouseClick** кнопки *Button_CreateBckp*:

```
SecurityManager.ExportConfiguration(FileSystem.CurrentUserFolder +
"/export.ldb")
```

2. После выполнения функции активируется одно из событий *SecurityManager*: в случае успешного создания резервной копии - **GetConfigurationFinished** (стр. 107), в противном случае - **GetConfigurationFailed** (стр. 108). Ознакомьтесь с обработчиками этих событий.

> **!** ВАЖНО
> Будьте внимательны при работе с проектом-примером. Описанные ниже действия ведут к изменению конфигурации Alpha.Security.

Запустите проект-пример в рантайм. Откроется окно выбора действия, и, если было выполнено подключение к LDAP-серверу, кнопка **Управление резервными копиями** будет доступна. Нажмите кнопку. Откроется окно формы *Backups*. Получите список пользователей, нажав на соответствующую кнопку.



Нажмите кнопку **Создать бэкап**. Будет вызвана функция **ExportConfiguration()**:

- в случае успешного выполнения функции создастся резервная копия и появится соответствующее сообщение;
- в случае неуспешного выполнения функции появится сообщение с текстом ошибки.

Восстановление резервной копии

Для восстановления резервных копий используются возможности элемента *SecurityManager*:

1. При нажатии кнопки *Button_OpenBckp* вызывается функция **ImportConfiguration()** (стр. 101), восстанавливающая резервную копию из указанного файла. В данном случае резервная копия

восстанавливается из файла, расположенного в папке текущего пользователя ОС. Ознакомьтесь с кодом обработчика MouseClick кнопки *Button_OpenBckp*:

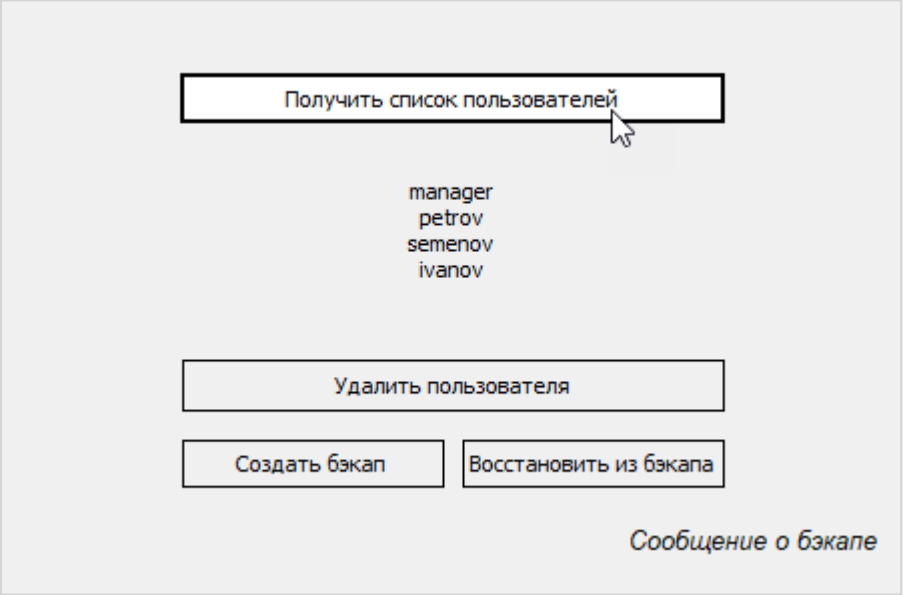
```
SecurityManager.ImportConfiguration(FileSystem.CurrentUserFolder +
"/export.ldb")
```

2. После выполнения функции активируется одно из событий *SecurityManager*: в случае успешного восстановления конфигурации - SetConfigurationFinished (стр. 108), в противном случае - SetConfigurationFailed (стр. 108). Ознакомьтесь с обработчиками этих событий.

ВАЖНО

Будьте внимательны при работе с проектом-примером. Описанные ниже действия ведут к изменению конфигурации Alpha.Security.

Запустите проект-пример в рантайм. Откроется окно выбора действия, и, если было выполнено подключение к LDAP-серверу, кнопка **Управление резервными копиями** будет доступна. Нажмите кнопку. Откроется окно формы *Backups*. Получите список пользователей, нажав на соответствующую кнопку.



ПРИМЕЧАНИЕ

Для наглядности можно удалить одного из пользователей. Для этого нажмите **Удалить пользователя** и в открывшемся окне удалите любую учетную запись, кроме записи администратора. Затем вернитесь к окну управления резервными копиями. Получите список пользователей, нажав соответствую кнопку. Убедитесь, что учетная запись была удалена.

Нажмите кнопку **Восстановить из бэкапа**. Будет вызвана функция ImportConfiguration():

- в случае успешного выполнения функции восстановится резервная копия из файла;
- в случае неуспешного выполнения функции появится сообщение с текстом ошибки.

ПРИМЕЧАНИЕ

Вновь получите список пользователей, нажав соответствую кнопку. Убедитесь, что удаленная учетная запись восстановлена.

1.6. Контроль целостности файлов и папок

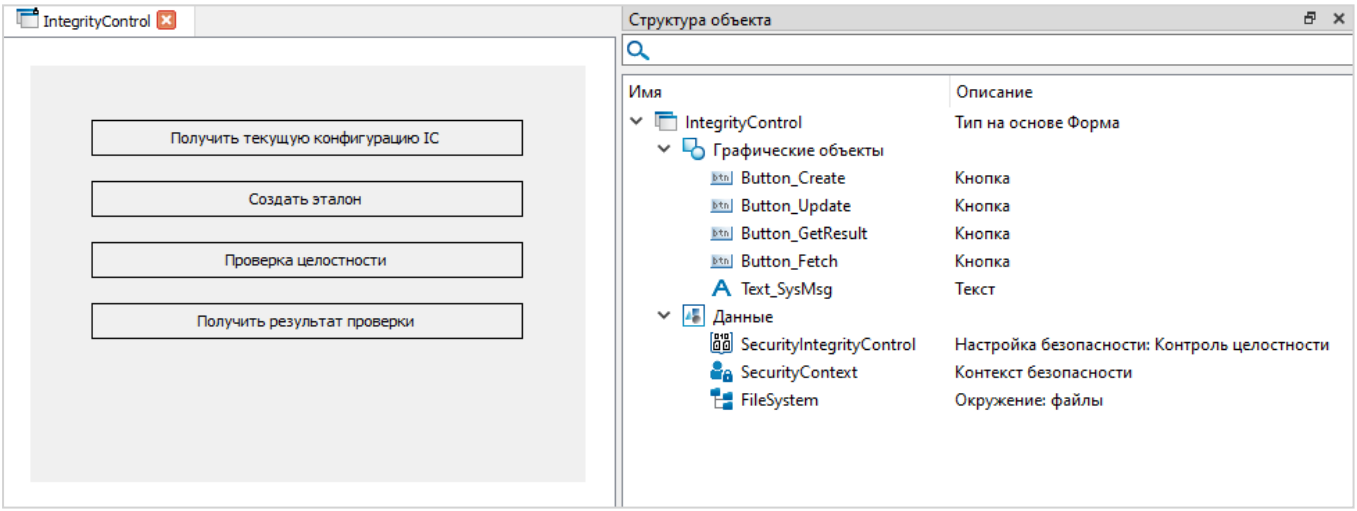
В разделе описано, как получать информацию о текущей конфигурации контроля целостности, а также об изменениях в контролируемых файлах и папках с помощью компонентов расширения Alpha.HMI.Security.

Для наглядности к разделу приложен проект-пример. Скачайте архив с проектом-примером `IntegrityControl.hmi` и откройте его в Дизайнер Alpha.HMI.

Состав проекта-примера

Единственная форма в проекте - **`IntegrityControl`**. На ней размещены:

- экземпляры типов 2.1. Контекст безопасности (SecurityContext) (стр. 50) и 2.3. Настройка безопасности: Контроль целостности (SecurityIntegrityControl) (стр. 68); первый служит для взаимодействия с агентом безопасности, второй - для получения и изменения текущих настроек контроля целостности, а также для создания эталонов, запуска проверок целостности и получения их результатов.
- вспомогательные компоненты - кнопки, текстовое поле, компонент для работы с файлами.



Назначение проекта-примера

Проект-пример в режиме рантайма представляет собой приложение, позволяющее:

- по нажатию кнопки **`Получить текущую конфигурацию IC`** получить файл, содержащий настройки активной конфигурации контроля целостности в виде JSON-строки (подробнее об этом - ниже);
- по нажатию кнопки **`Создать эталон`** создать срез текущего состояния контролируемых файлов, который будет считаться эталоном, и с которым будет сравниваться будущее состояние файлов при проверках;
- по нажатию кнопки **`Проверка целостности`** выполнить проверку целостности (сравнить текущее состояние контролируемых файлов с эталонным);
- по нажатию кнопки **`Получить результат проверки`** получить сообщения о состоянии каждого контролируемого файла.

Что представляет собой конфигурация контроля целостности

По умолчанию контроль целостности включен и выполняется для некоторых системных файлов и папок. Информация о том, целостность каких файлов и папок контролируется, с какой периодичностью и

т.п., называется конфигурацией контроля целостности, и хранится в папке `C:\Program Files\Automiq\Alpha.Security\IC` в виде зашифрованных файлов. Каждый файл представляет собой определенный набор настроек контроля целостности и называется группой контроля целостности. Агент безопасности может хранить несколько групп одновременно, однако активна может быть только одна из них.

ОБРАТИТЕ ВНИМАНИЕ

Alpha.Security версии 1.7.24 и новее при установке автоматически преобразовывает разработанную в более ранних версиях конфигурацию контроля целостности в новый формат. Ранее конфигурация представляла собой файл `alpha.security.ic.xml`, который располагался в папке на уровень выше.

Преобразованная конфигурация представляет собой группу с именем legacy. Подробнее о том, как получить имя и другие параметры групп, описано ниже и в разделе 2.3. Настройка безопасности: Контроль целостности (SecurityIntegrityControl) (стр. 68) справочного руководства.

Получение текущей конфигурации контроля целостности

Чтобы получить текущую конфигурацию контроля целостности, следует отправить JSON-запрос агенту безопасности. Ознакомьтесь с кодом обработчика события **ButtonPressed** кнопки *ButtonFetch*, написанным на языке JavaScript:

```
const request = {
 "object": "ic_api/ic_config/ic_group",
 "command": "Fetch",
};
SecurityIntegrityControl.JsonRequest(JSON.stringify(request));
```

Здесь приложение с помощью экземпляра *SecurityIntegrityControl* типа **Настройка безопасности: Контроль целостности** отправляет агенту безопасности запрос **JsonRequest()** с командой `Fetch`, служащей для получения конфигураций всех групп контроля целостности, хранимых агентом безопасности. Подробнее с принципами построения запроса и командами можно ознакомиться в разделе 2.3. Настройка безопасности: Контроль целостности (SecurityIntegrityControl) (стр. 68) справочного руководства.

ПРИМЕЧАНИЕ

Компонент позволяет не только получать, но и менять настройки контроля целостности с помощью JSON-запросов. Так, например, чтобы добавить новую группу контроля целостности, следует сначала отправить запрос с командой `Add`, затем с помощью запроса с командой `Fetch` получить ID созданной группы, а затем записать настройки контроля целостности в добавленную группу с помощью запроса с командой `Update`.

В результате выполнения запроса ответ будет отправлен в переменную response события **JsonRequestComplete** экземпляра *SecurityIntegrityControl* типа **Настройка безопасности: Контроль целостности**. Пример ответа приведен в разделе 2.3. Настройка безопасности: Контроль целостности (SecurityIntegrityControl) (стр. 68) справочного руководства. При получении ответа приложение:

- запишет этот ответ в текстовый файл `response.txt`, который создаст в папке проекта-примера;
- отобразит информацию об активной группе контроля целостности (той конфигурации, которая используется агентом для контроля целостности в данный момент) в окне приложения.

Ознакомьтесь с кодом обработчика события **JsonRequestComplete** экземпляра *SecurityIntegrityControl*, написанным на языке JavaScript: